

Curso Profesional de Entity Framework Core 8

Módulo 4 - Optimización y rendimiento

TEORÍA

AUTOR: JAIME GALLO

.NET 8 · Entity Framework Core 8 · Visual Studio Community · SQL Server LocalDB · Proyecto AceriaData

Índice

Punto 4.1 — Análisis del SQL generado: ToQueryString y logging	3
Punto 4.2 — Tracking y No Tracking	10
Punto 4.3 — AsNoTracking y AsNoTrackingWithIdentityResolution	18
Punto 4.4 — Problema N+1: identificación y causas	25
Punto 4.5 — Solución a N+1: Include, proyecciones y Split Queries	33
Punto 4.6 — Over-fetching: causas y soluciones	41
Punto 4.7 — Consultas ineficientes: traducción y frontera cliente/servidor	48
Punto 4.8 — Split Queries: cuándo y cómo usarlas	56
Punto 4.9 — Compiled Queries	64
Punto 4.10 — Paginación eficiente: Skip/Take y keyset pagination	71
Punto 4.11 — Diagnóstico con logs, métricas y herramientas	80
Punto 4.12 — Estrategias de optimización y checklist de rendimiento	87

Módulo 4 — Optimización y rendimiento

12 puntos · 6 horas · AceriaData · .NET 8 · Entity Framework Core 8 · SQL Server LocalDB

Este módulo continúa AceriaData desde el estado final del Módulo 3 y se centra en observar, medir y optimizar consultas de Entity Framework Core 8.

Mapa del módulo

Punto	Tema	Duración de referencia
4.1	Análisis del SQL generado: ToQueryString y logging	30 min
4.2	Tracking y No Tracking	30 min
4.3	AsNoTracking y AsNoTrackingWithIdentityResolution	30 min
4.4	Problema N+1: identificación y causas	30 min
4.5	Solución a N+1: Include, proyecciones y Split Queries	30 min
4.6	Over-fetching: causas y soluciones	30 min
4.7	Consultas ineficientes: traducción y frontera cliente/servidor	30 min
4.8	Split Queries: cuándo y cómo usarlas	30 min
4.9	Compiled Queries	30 min
4.10	Paginación eficiente: Skip/Take y keyset pagination	30 min
4.11	Diagnóstico con logs, métricas y herramientas	30 min
4.12	Estrategias de optimización y checklist de rendimiento	30 min

Criterio del módulo: ninguna técnica se considera optimización por su nombre; debe relacionarse con SQL, roundtrips, filas/columnas, materialización, tracking y medición.

Punto 4.1 — Análisis del SQL generado: ToQueryString y logging

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se introduce el análisis del SQL generado en el proyecto AceriaData, usando ToQueryString para inspeccionar las consultas y el logging para registrar las sentencias que EF Core ejecuta contra SQL Server LocalDB.

Objetivos de aprendizaje

- Comprender por qué es importante analizar el SQL generado por EF Core.
- Utilizar ToQueryString para inspeccionar una consulta sin ejecutarla.
- Configurar el logging de EF Core para registrar las sentencias SQL.
- Interpretar las sentencias SQL generadas por consultas LINQ.
- Detectar consultas ineficientes a partir del SQL generado.
- Comprender el impacto del filtro global de Soft Delete en el SQL.
- Analizar consultas con Include, Where, OrderBy y Select.

- Aplicar el análisis del SQL al proyecto AceriaData.

Consideraciones técnicas en EF Core 8

ToQueryString inspecciona la representación SQL sin materializar; el logging muestra los comandos realmente ejecutados. Son herramientas complementarias: una muestra la consulta prevista y la otra la ejecución real.

Desarrollo teórico

Por qué analizar el SQL generado

EF Core traduce las consultas LINQ a SQL, pero el SQL generado no siempre es el más eficiente. Analizar el SQL generado permite detectar problemas como productos cartesianos, filtros aplicados en memoria, consultas N+1 y proyecciones innecesarias. El análisis del SQL es la base de la optimización: sin saber qué SQL se ejecuta, no se puede mejorar.

Ejemplo 1 (CSHARP).

```
var consulta = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.FechaCreacion);

var sql = consulta.ToQueryString();
Console.WriteLine(sql);
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por estado. La tercera línea ordena por fecha. La cuarta línea obtiene el SQL sin ejecutarlo. La quinta línea imprime el SQL. La consulta no se ejecuta contra la base de datos.

El método ToQueryString

ToQueryString devuelve la sentencia SQL que EF Core generaría para una consulta, sin ejecutarla. Es útil para inspeccionar consultas antes de ejecutarlas y para diagnosticar problemas de traducción. Solo funciona con IQueryable, no con IEnumerable.

Ejemplo 2 (CSHARP).

```
var consulta = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Where(o => o.Cliente == "Constructora del Norte");

var sql = consulta.ToQueryString();
Console.WriteLine(sql);
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea filtra por cliente. La cuarta línea obtiene el SQL. La quinta línea imprime el SQL. El SQL incluye un LEFT JOIN con la tabla de planchas y un WHERE por cliente.

El SQL generado tiene la siguiente forma:

Ejemplo 3 (SQL).

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [p].[Id], [p].[Espesor], [p].[Peso]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[Cliente] = N'Constructora del Norte' AND [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id]
```

SQL

La primera línea selecciona las columnas de la orden y de las planchas. La segunda línea indica la tabla principal. La tercera línea combina con la tabla de planchas. La cuarta línea filtra por cliente y aplica el filtro global de Soft Delete. La quinta línea ordena por el Id de la orden y el Id de la plancha.

El logging de EF Core

El logging de EF Core registra las sentencias SQL que se ejecutan contra la base de datos. Se configura con el método LogTo en las opciones del DbContext. Los mensajes incluyen la sentencia SQL, los parámetros y el tiempo de ejecución.

Ejemplo 4 (CSHARP).

CSHARP

```
optionsBuilder
    .UseSqlServer(connectionString)
    .LogTo(
        Console.WriteLine,
        new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
        LogLevel.Information);
```

La primera línea configura el proveedor de SQL Server. La segunda línea habilita el logging. La tercera línea especifica el destino del logging. La cuarta línea especifica las categorías que se registran. La quinta línea especifica el nivel mínimo de logging. A partir de este momento, todas las sentencias SQL se escriben en la consola.

Categorías de logging

EF Core organiza los mensajes de logging en categorías. Las más habituales son `Microsoft.EntityFrameworkCore.Database.Command`, que incluye las sentencias SQL, `Microsoft.EntityFrameworkCore.Query`, que incluye información sobre la traducción de consultas, y `Microsoft.EntityFrameworkCore.Update`, que incluye información sobre las operaciones de escritura.

Ejemplo 5 (CSHARP).

CSHARP

```
optionsBuilder.LogTo(
    Console.WriteLine,
    new[]
    {
        "Microsoft.EntityFrameworkCore.Database.Command",
        "Microsoft.EntityFrameworkCore.Query",
        "Microsoft.EntityFrameworkCore.Update"
    },
    LogLevel.Information);
```

La primera línea habilita el logging. La segunda línea especifica el destino. La tercera línea inicia el array de categorías. La cuarta línea incluye la categoría de comandos. La quinta línea incluye la categoría de consultas. La sexta línea incluye la categoría de actualizaciones. La séptima línea especifica el nivel mínimo. A partir de este momento, se registran los mensajes de las tres categorías.

Niveles de logging

EF Core usa los niveles de logging estándar de .NET: Trace, Debug, Information, Warning, Error y Critical. El nivel Information incluye las sentencias SQL. El nivel Debug incluye información adicional sobre la ejecución. El nivel Warning incluye advertencias. El nivel Error incluye errores.

Ejemplo 6 (CSHARP).

CSHARP

```
optionsBuilder.LogTo(
    Console.WriteLine,
    new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
    LogLevel.Information);
```

La primera línea habilita el logging. La segunda línea especifica el destino. La tercera línea especifica las categorías. La cuarta línea especifica el nivel mínimo. Solo se muestran los mensajes de nivel Information o superior. Los mensajes de nivel Debug y Trace no se muestran.

EnableSensitiveDataLogging

Por defecto, EF Core oculta los valores de los parámetros en los mensajes de logging para evitar exponer datos sensibles. El método `EnableSensitiveDataLogging` permite mostrar los valores reales de los parámetros.

Ejemplo 7 (CSHARP).

CSHARP

```
optionsBuilder
    .UseSqlServer(connectionString)
    .EnableSensitiveDataLogging();
```

La primera línea configura el proveedor. La segunda línea habilita el logging de datos sensibles. Los parámetros de las consultas se muestran con sus valores reales. Es útil en desarrollo, pero no se recomienda en producción porque puede exponer datos confidenciales en los logs.

EnableDetailedErrors

Por defecto, EF Core no incluye información detallada en los mensajes de error para evitar exponer la estructura interna. El método `EnableDetailedErrors` permite mostrar información más detallada en los errores.

Ejemplo 8 (CSHARP).

```
optionsBuilder
    .UseSqlServer(connectionString)
    .EnableDetailedErrors();
```

CSHARP

La primera línea configura el proveedor. La segunda línea habilita los errores detallados. Los mensajes de error incluyen información sobre las propiedades y las entidades implicadas. Es útil en desarrollo, pero puede exponer información interna en producción.

Análisis de consultas con Where

El análisis de una consulta con `Where` permite comprobar que el filtro se traduce a SQL y no se aplica en memoria. El SQL generado incluye la cláusula `WHERE` con la condición.

Ejemplo 9 (CSHARP).

```
var consulta = context.OrdenesFabricacion
    .Where(o => o.Cliente == "Constructora del Norte" && o.Estado == "Pendiente");

var sql = consulta.ToQueryString();
Console.WriteLine(sql);
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por cliente y estado. La tercera línea obtiene el SQL. La cuarta línea imprime el SQL. El SQL incluye la cláusula `WHERE` con las dos condiciones unidas por `AND`.

Ejemplo 10 (SQL).

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Cliente] = N'Constructora del Norte' AND [o].[Estado] = N'Pendiente' AND [o].[IsDeleted] = CAST(0 AS bit)
```

SQL

La primera línea selecciona las columnas. La segunda línea indica la tabla. La tercera línea filtra por cliente, estado y el filtro global de `Soft Delete`.

Análisis de consultas con OrderBy

El análisis de una consulta con `OrderBy` permite comprobar que la ordenación se traduce a SQL. El SQL generado incluye la cláusula `ORDER BY` con la columna.

Ejemplo 11 (CSHARP).

```
var consulta = context.OrdenesFabricacion
    .OrderBy(o => o.FechaCreacion)
    .ThenByDescending(o => o.Cliente);

var sql = consulta.ToQueryString();
Console.WriteLine(sql);
```

CSHARP

La primera línea inicia la consulta. La segunda línea ordena por fecha ascendente. La tercera línea ordena por cliente descendente. La cuarta línea obtiene el SQL. La quinta línea imprime el SQL. El SQL incluye la cláusula `ORDER BY` con las dos columnas.

Ejemplo 12 (SQL).

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[FechaCreacion], [o].[Cliente] DESC
```

SQL

La primera línea selecciona las columnas. La segunda línea indica la tabla. La tercera línea aplica el filtro global de `Soft Delete`. La cuarta línea ordena por fecha ascendente y cliente descendente.

Análisis de consultas con Select

El análisis de una consulta con Select permite comprobar que la proyección se traduce a SQL. El SQL generado incluye solo las columnas proyectadas.

Ejemplo 13 (CSHARP).

```
var consulta = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .Select(o => new { o.NumeroOrden, o.Cliente });

var sql = consulta.ToQueryString();
Console.WriteLine(sql);
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por estado. La tercera línea proyecta a un tipo anónimo. La cuarta línea obtiene el SQL. La quinta línea imprime el SQL. El SQL incluye solo las columnas NumeroOrden y Cliente.

Ejemplo 14 (SQL).

```
SELECT [o].[NumeroOrden], [o].[Cliente]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente' AND [o].[IsDeleted] = CAST(0 AS bit)
```

SQL

La primera línea selecciona solo las columnas proyectadas. La segunda línea indica la tabla. La tercera línea filtra por estado y aplica el filtro global.

Análisis de consultas con Include

El análisis de una consulta con Include permite comprobar que la carga de entidades relacionadas se traduce a un LEFT JOIN. El SQL generado incluye todas las columnas de las entidades relacionadas.

Ejemplo 15 (CSHARP).

```
var consulta = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Where(o => o.Cliente == "Constructora del Norte");

var sql = consulta.ToQueryString();
Console.WriteLine(sql);
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea filtra por cliente. La cuarta línea obtiene el SQL. La quinta línea imprime el SQL. El SQL incluye un LEFT JOIN con la tabla de planchas y todas las columnas de las planchas.

Ejemplo 16 (SQL).

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [p].[Id], [p].[Espesor], [p].[Ancho], [p].[Largo], [p].[Peso], [p].[Activa]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[Cliente] = N'Constructora del Norte' AND [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id]
```

SQL

La primera línea selecciona todas las columnas de la orden y de las planchas. La segunda línea indica la tabla principal. La tercera línea combina con la tabla de planchas. La cuarta línea filtra por cliente y aplica el filtro global. La quinta línea ordena por el Id de la orden y el Id de la plancha.

El filtro global de Soft Delete en el SQL

El filtro global de Soft Delete configurado en el Módulo 2 se aplica automáticamente a todas las consultas. El SQL generado incluye la condición [IsDeleted] = CAST(0 AS bit) en todas las tablas que tienen el filtro.

Ejemplo 17 (CSHARP).

```
var consulta = context.OrdenesFabricacion
    .Include(o => o.Planchas);
```

CSHARP

```
var sql = consulta.ToQueryString();
Console.WriteLine(sql);
```

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea obtiene el SQL. La cuarta línea imprime el SQL. El SQL incluye el filtro global en ambas tablas.

Ejemplo 18 (SQL).

```
SELECT [o].[Id], [o].[NumeroOrden], ..., [p].[Id], [p].[Espesor], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id]
```

SQL

La primera línea selecciona las columnas. La segunda línea indica la tabla principal. La tercera línea combina con la tabla de planchas. La cuarta línea aplica el filtro global en ambas tablas. La quinta línea ordena por el Id de la orden y el Id de la plancha.

El proyecto AceriaData

En el proyecto AceriaData, el análisis del SQL generado se usa para diagnosticar problemas de rendimiento. Se añaden métodos al repositorio que devuelven el SQL generado sin ejecutarlo. Se configura el logging para registrar las sentencias SQL. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

El análisis del SQL generado permite detectar problemas de rendimiento.

ToQueryString devuelve el SQL sin ejecutarlo.

El logging de EF Core registra las sentencias SQL ejecutadas.

Las categorías de logging permiten filtrar los mensajes.

Los niveles de logging indican la importancia del mensaje.

EnableSensitiveDataLogging muestra los valores de los parámetros.

EnableDetailedErrors muestra información detallada en los errores.

El análisis de consultas con Where, OrderBy, Select e Include permite verificar la traducción.

El filtro global de Soft Delete se aplica automáticamente al SQL.

En el proyecto AceriaData se añaden métodos para analizar el SQL.

Anclaje en AceriaData

El concepto se implementa en M04/PROYECTO/4.1. En la práctica se restaura, compila y ejecuta sobre SQL Server LocalDB.

Ejemplo ejecutable del concepto

El siguiente archivo es la implementación real de Infrastructure correspondiente al punto 4.1.

```
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    public string ObtenerSqlPendientesOrdenadasM4() => _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .ToQueryString();

    public string ObtenerSqlConIncludeM4() => _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .Where(o => o.Cliente == "Constructora del Norte")
        .OrderBy(o => o.Id)
```

CSHARP

```

        .ToString();

    public string ObtenerSqlConProyeccionM4() => _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Select(o => new { o.NumeroOrden, o.Cliente })
        .ToString();
}

```

Explicación línea a línea — Rendimiento41.cs

Línea 1	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	namespace AceriaData.Infrastructure.Repositories; → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 5	public sealed partial class OrdenRepository → Declara la clase concreta que implementa la funcionalidad del punto.
Línea 6	{ → Delimita el bloque sintáctico asociado.
Línea 7	public string ObtenerSqlPendientesOrdenadasM4() => _context.OrdenesFabricacion → Define la expresión lambda que EF Core o el caso de uso empleará en esta operación.
Línea 8	.AsNoTracking() → Desactiva tracking para esta consulta de lectura.
Línea 9	.Where(o => o.Estado == "Pendiente") → Añade el predicado de filtrado a la forma de consulta.
Línea 10	.OrderBy(o => o.FechaCreacion) → Forma parte del orden determinista.
Línea 11	.ThenBy(o => o.Id) → Forma parte del orden determinista.
Línea 12	.ToString(); → Obtiene la representación SQL sin materializar la consulta.
Línea 14	public string ObtenerSqlConIncludeM4() => _context.OrdenesFabricacion → Define la expresión lambda que EF Core o el caso de uso empleará en esta operación.
Línea 15	.AsNoTracking() → Desactiva tracking para esta consulta de lectura.
Línea 16	.Include(o => o.Planchas) → Define la navegación relacionada que debe cargarse.
Línea 17	.Where(o => o.Cliente == "Constructora del Norte") → Añade el predicado de filtrado a la forma de consulta.
Línea 18	.OrderBy(o => o.Id) → Forma parte del orden determinista.
Línea 19	.ToString(); → Obtiene la representación SQL sin materializar la consulta.
Línea 21	public string ObtenerSqlConProyeccionM4() => _context.OrdenesFabricacion → Define la expresión lambda que EF Core o el caso de uso empleará en esta operación.
Línea 22	.AsNoTracking() → Desactiva tracking para esta consulta de lectura.
Línea 23	.Where(o => o.Estado == "Pendiente") → Añade el predicado de filtrado a la forma de consulta.
Línea 24	.OrderBy(o => o.FechaCreacion) → Forma parte del orden determinista.
Línea 25	.ThenBy(o => o.Id) → Forma parte del orden determinista.
Línea 26	.Select(o => new { o.NumeroOrden, o.Cliente }) → Proyecta la forma de resultado y controla datos materializados.

Línea 27	<code>.ToString();</code> → Obtiene la representación SQL sin materializar la consulta.
Línea 28	<code>}</code> → Delimita el bloque sintáctico asociado.

Qué debe observarse en ejecución

La lectura del código debe completarse con la evidencia de ejecución: SQL traducido cuando corresponda, número de comandos, cardinalidad, columnas materializadas, estado del `ChangeTracker` y las comprobaciones específicas del punto. El hecho de que el código compile no demuestra por sí solo una mejora de rendimiento; por eso cada punto termina en una comprobación observable.

Reto conceptual. Comparar el SQL de una entidad completa con el de una proyección y justificar qué columnas sobran.

Analogía operativa. `ToString` es el plano previo; el logging es el registro de lo que realmente pasó por la línea.

Criterios de salida

- Relacionar LINQ con SQL o comandos ejecutados.
- Distinguir coste de servidor, transferencia, materialización y tracking.
- Justificar técnicas aplicadas y descartadas.
- Ejecutar el proyecto del punto y obtener 4.1 OK.

Punto 4.2 — Tracking y No Tracking

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en el control del tracking en el proyecto `AceriaData`, comparando el comportamiento de las consultas con Tracking y sin Tracking, analizando el impacto en memoria y en rendimiento, y aplicando las buenas prácticas en el repositorio.

Objetivos de aprendizaje

- Comprender qué es el tracking y qué hace el `Change Tracker`.
- Diferenciar entre consultas con Tracking y sin Tracking.
- Comprender el impacto del tracking en memoria y rendimiento.
- Utilizar `AsNoTracking` en consultas de solo lectura.
- Utilizar `AsTracking` para forzar el tracking en consultas puntuales.
- Comprender el papel de la caché de identidad.
- Analizar el SQL generado por consultas con y sin Tracking.
- Medir el impacto del tracking en el número de entidades rastreadas.
- Aplicar el control del tracking al proyecto `AceriaData`.

Consideraciones técnicas en EF Core 8

Tracking y NoTracking normalmente no cambian el `SELECT`: cambian sobre todo materialización y `ChangeTracker`. Un DTO puro sin entidades no se rastrea; una proyección que contenga entidades sí puede mantener tracking de esas entidades.

Referencia oficial: <https://learn.microsoft.com/ef/core/querying/tracking>

Desarrollo teórico

Qué es el tracking

El tracking es el mecanismo por el que EF Core registra las entidades que carga desde la base de datos en el Change Tracker. Cuando una entidad se carga con Tracking, EF Core guarda una copia de sus valores originales y realiza un seguimiento de sus cambios. Al llamar a SaveChanges, EF Core compara los valores actuales con los originales y genera las sentencias SQL necesarias para persistir los cambios.

Ejemplo 1 (CSHARP).

```
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    Console.WriteLine($"{orden.NumeroOrden} - {orden.Cliente}");
}
```

CSHARP

La primera línea carga todas las órdenes con Tracking. La segunda línea itera sobre las órdenes. La tercera línea muestra los datos. Aunque no se modifica ninguna orden, EF Core ha registrado todas las entidades en el Change Tracker y ha guardado una copia de sus valores originales.

Consultas con Tracking

Las consultas con Tracking son el comportamiento por defecto de EF Core. Cada entidad cargada se registra en el Change Tracker y se guarda una copia de sus valores originales. Esto permite modificar las entidades y guardar los cambios con SaveChanges.

Ejemplo 2 (CSHARP).

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
orden!.Cliente = "Constructora del Sur";
context.SaveChanges();
```

CSHARP

La primera línea carga la orden con Tracking. La segunda línea modifica la propiedad Cliente. La tercera línea guarda los cambios. El Change Tracker detecta el cambio y genera un UPDATE con la nueva propiedad.

El SQL generado tiene la siguiente forma:

Ejemplo 3 (SQL).

```
SELECT TOP 1 [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [o].[Observaciones], [o].[IsDeleted], [o].[DeletedAt]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Id] = 1 AND [o].[IsDeleted] = CAST(0 AS bit);

UPDATE [OrdenesFabricacion]
SET [Cliente] = @p0
WHERE [Id] = @p1;
```

SQL

La primera sentencia carga la orden. La segunda sentencia actualiza solo la columna Cliente porque el Change Tracker detectó que solo esa propiedad había cambiado.

Consultas sin Tracking

Las consultas sin Tracking cargan las entidades sin registrarlas en el Change Tracker. Las entidades no se rastrean, no se guarda una copia de sus valores originales y los cambios no se guardan con SaveChanges. Evitan el trabajo del Change Tracker y pueden reducir memoria y CPU en consultas de solo lectura; el efecto temporal concreto debe medirse.

Ejemplo 4 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .AsNoTracking()
    .ToList();

foreach (var orden in ordenes)
{
    Console.WriteLine($"{orden.NumeroOrden} - {orden.Cliente}");
}
```

CSHARP

La primera línea carga todas las órdenes sin Tracking. La segunda línea itera sobre las órdenes. La tercera línea muestra los datos. Las entidades no se registran en el Change Tracker.

El SQL generado es el mismo que con Tracking:

Ejemplo 5 (SQL).

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [o].[Observaciones], [o].[IsDeleted], [o].[DeletedAt]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
```

SQL

La diferencia no está en el SQL, sino en el comportamiento en memoria. Con AsNoTracking, EF Core no crea las estructuras internas del Change Tracker para las entidades cargadas.

El impacto del tracking en memoria

El tracking consume memoria porque el Change Tracker mantiene una referencia a cada entidad cargada, una copia de sus valores originales y una entrada en la caché de identidad. En consultas con muchas filas, el consumo de memoria puede ser significativo.

Ejemplo 6 (CSHARP).

```
var ordenesConTracking = context.OrdenesFabricacion.ToList();
Console.WriteLine($"Entidades rastreadas: {context.ChangeTracker.Entries().Count()}");
```

CSHARP

La primera línea carga todas las órdenes con Tracking. La segunda línea muestra el número de entidades rastreadas. El número coincide con el número de órdenes cargadas.

Ejemplo 7 (CSHARP).

```
var ordenesSinTracking = context.OrdenesFabricacion.AsNoTracking().ToList();
Console.WriteLine($"Entidades rastreadas: {context.ChangeTracker.Entries().Count()}");
```

CSHARP

La primera línea carga todas las órdenes sin Tracking. La segunda línea muestra el número de entidades rastreadas. El número es cero porque ninguna entidad se ha registrado.

El impacto del tracking en rendimiento

El tracking consume CPU porque el Change Tracker debe registrar cada entidad, crear la caché de identidad y guardar los valores originales. En consultas de solo lectura, este trabajo es innecesario y puede degradar el rendimiento.

Ejemplo 8 (CSHARP).

```
// Con Tracking
var cronometro1 = Stopwatch.StartNew();
var ordenes1 = context.OrdenesFabricacion.ToList();
cronometro1.Stop();

// Sin Tracking
var cronometro2 = Stopwatch.StartNew();
var ordenes2 = context.OrdenesFabricacion.AsNoTracking().ToList();
cronometro2.Stop();
```

CSHARP

La primera sección mide Tracking y la segunda No Tracking. No Tracking elimina trabajo del Change Tracker, pero una medición concreta no debe darse por ganada de antemano.

El método AsNoTracking

AsNoTracking es un método de extensión que se aplica sobre IQueryable. Devuelve una nueva consulta que no registra las entidades en el Change Tracker. Se puede aplicar a cualquier consulta, antes o después de los filtros.

Ejemplo 9 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .AsNoTracking();
```

CSHARP

```
.OrderBy(o => o.FechaCreacion)
.ToList();
```

La primera línea inicia la consulta. La segunda línea filtra por estado. La tercera línea aplica AsNoTracking. La cuarta línea ordena por fecha. La quinta línea materializa la consulta. Las entidades no se registran en el Change Tracker.

El método AsTracking

AsTracking es un método de extensión que fuerza el tracking en una consulta. Es útil cuando se ha configurado NoTracking como comportamiento por defecto y se quiere forzar el tracking en una consulta concreta.

Ejemplo 10 (CSHARP).

```
optionsBuilder.UseQueryTrackingBehavior(QueryTrackingBehavior.NoTracking);

var orden = context.OrdenesFabricacion
    .AsTracking()
    .FirstOrDefault(o => o.Id == 1);
```

CSHARP

La primera línea configura NoTracking como comportamiento por defecto. La segunda línea inicia la consulta. La tercera línea aplica AsTracking para forzar el tracking. La cuarta línea carga la entidad. La entidad se registra en el Change Tracker aunque el comportamiento por defecto sea NoTracking.

El método UseQueryTrackingBehavior

UseQueryTrackingBehavior configura el comportamiento por defecto del tracking en las consultas. Se puede establecer en TrackAll (por defecto) o en NoTracking.

Ejemplo 11 (CSHARP).

```
optionsBuilder.UseQueryTrackingBehavior(QueryTrackingBehavior.NoTracking);
```

CSHARP

La primera línea establece NoTracking como comportamiento por defecto. A partir de este momento, todas las consultas se ejecutan sin Tracking a menos que se aplique AsTracking.

Ejemplo 12 (CSHARP).

```
optionsBuilder.UseQueryTrackingBehavior(QueryTrackingBehavior.TrackAll);
```

CSHARP

La primera línea establece TrackAll como comportamiento por defecto. Es el comportamiento estándar de EF Core.

La caché de identidad

La caché de identidad es una estructura interna que garantiza que, dentro de un mismo DbContext, solo existe una instancia por cada entidad con una clave primaria concreta. Con Tracking, la caché de identidad devuelve la misma instancia si se carga la misma entidad dos veces. Sin Tracking, la caché de identidad no se usa y se crean instancias nuevas cada vez.

Ejemplo 13 (CSHARP).

```
var orden1 = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
var orden2 = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);

Console.WriteLine($"Misma instancia: {ReferenceEquals(orden1, orden2)}");
```

CSHARP

La primera línea carga la orden con Tracking. La segunda línea carga la misma orden con Tracking. La tercera línea comprueba si son la misma instancia. El resultado es true porque la caché de identidad devuelve la misma instancia.

Ejemplo 14 (CSHARP).

```
var orden1 = context.OrdenesFabricacion.AsNoTracking().FirstOrDefault(o => o.Id == 1);
var orden2 = context.OrdenesFabricacion.AsNoTracking().FirstOrDefault(o => o.Id == 1);

Console.WriteLine($"Misma instancia: {ReferenceEquals(orden1, orden2)}");
```

CSHARP

La primera línea carga la orden sin Tracking. La segunda línea carga la misma orden sin Tracking. La tercera línea comprueba si son la misma instancia. El resultado es false porque la caché de identidad no se usa.

Cuándo usar AsNoTracking

AsNoTracking se usa en consultas de solo lectura donde no se van a modificar las entidades. Es el caso de las consultas de listado, las proyecciones a DTOs y las consultas de agregación. En estos escenarios, el tracking es innecesario y consume recursos.

Ejemplo 15 (CSHARP).

```
var resmenes = context.OrdenesFabricacion
    .AsNoTracking()
    .Where(o => o.Estado == "Pendiente")
    .Select(o => new OrdenResumenDto
    {
        NumeroOrden = o.NumeroOrden,
        Cliente = o.Cliente,
        Estado = o.Estado
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea aplica AsNoTracking. La tercera línea filtra por estado. La cuarta línea proyecta al DTO. La quinta línea materializa la consulta. Las entidades no se registran en el Change Tracker.

Cuándo no usar AsNoTracking

AsNoTracking no se debe usar en consultas donde se van a modificar las entidades. Si se aplica AsNoTracking y después se modifica una entidad, los cambios no se guardan porque la entidad no está registrada en el Change Tracker.

Ejemplo 16 (CSHARP).

```
var orden = context.OrdenesFabricacion
    .AsNoTracking()
    .FirstOrDefault(o => o.Id == 1);

orden!.Cliente = "Constructora del Sur";
context.SaveChanges();
```

CSHARP

La primera línea carga la orden sin Tracking. La segunda línea modifica la propiedad. La tercera línea guarda los cambios. Los cambios no se persisten porque la entidad no está registrada en el Change Tracker.

El tracking en consultas con proyección

Una proyección escalar o DTO que no contiene entidades no añade entidades al ChangeTracker; si una proyección personalizada contiene una entidad, esa entidad puede seguir siendo rastreada.

Ejemplo 17 (CSHARP).

```
var resmenes = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .Select(o => new { o.NumeroOrden, o.Cliente })
    .ToList();

Console.WriteLine($"Entidades rastreadas: {context.ChangeTracker.Entries().Count()}");
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por estado. La tercera línea proyecta a un tipo anónimo. La cuarta línea materializa la consulta. La quinta línea muestra el número de entidades rastreadas. El número es cero porque la proyección no devuelve entidades.

El tracking en consultas con Include

Las consultas con Include registran tanto la entidad principal como las entidades relacionadas en el Change Tracker. Esto puede consumir mucha memoria si se cargan muchas entidades relacionadas.

Ejemplo 18 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();

Console.WriteLine($"Entidades rastreadas: {context.ChangeTracker.Entries().Count()}");
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea materializa la consulta. La cuarta línea muestra el número de entidades rastreadas. El número incluye las órdenes y las planchas.

El tracking en consultas con AsSplitQuery

Las consultas con AsSplitQuery ejecutan varias consultas separadas. El tracking se aplica a todas las entidades cargadas en todas las consultas.

Ejemplo 19 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSplitQuery()
    .ToList();

Console.WriteLine($"Entidades rastreadas: {context.ChangeTracker.Entries().Count()}");
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la colección de entidades intermedias. La cuarta línea divide la consulta en varias. La quinta línea materializa la consulta. La sexta línea muestra el número de entidades rastreadas. El número incluye todas las entidades cargadas en las tres consultas.

El proyecto AceriaData

En el proyecto AceriaData, el control del tracking se aplica a todos los métodos del repositorio. Las consultas de solo lectura usan AsNoTracking. Las consultas de escritura usan Tracking. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

El tracking registra las entidades en el Change Tracker.

Las consultas con Tracking guardan una copia de los valores originales.

Las consultas sin Tracking no registran las entidades.

AsNoTracking se usa en consultas de solo lectura.

AsTracking fuerza el tracking en consultas puntuales.

UseQueryTrackingBehavior configura el comportamiento por defecto.

La caché de identidad garantiza una instancia por entidad con Tracking.

Las proyecciones a tipos anónimos o DTOs no registran entidades.

Las consultas con Include registran las entidades relacionadas.

En el proyecto AceriaData se aplica el control del tracking a todos los métodos.

Anclaje en AceriaData

El concepto se implementa en M04/PROYECTO/4.2. En la práctica se restaura, compila y ejecuta sobre SQL Server LocalDB.

Ejemplo ejecutable del concepto

El siguiente archivo es la implementación real de Infrastructure correspondiente al punto 4.2.

```
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    public TrackingMetricaDto MedirConsultaConTrackingM4()
    {
        _context.ChangeTracker.Clear();
        var consulta = _context.OrdenesFabricacion
            .AsTracking()
            .OrderBy(o => o.Id);
```

CSHARP

```

        var sql = consulta.ToQueryString();
        var filas = consulta.ToList().Count;
        return new TrackingMetricaDto
        {
            Filas = filas,
            EntidadesRastreadas = _context.ChangeTracker.Entries().Count(),
            Sql = sql
        };
    }

    public TrackingMetricaDto MedirConsultaSinTrackingM4()
    {
        _context.ChangeTracker.Clear();
        var consulta = _context.OrdenesFabricacion
            .AsNoTracking()
            .OrderBy(o => o.Id);
        var sql = consulta.ToQueryString();
        var filas = consulta.ToList().Count;
        return new TrackingMetricaDto
        {
            Filas = filas,
            EntidadesRastreadas = _context.ChangeTracker.Entries().Count(),
            Sql = sql
        };
    }
}

```

Explicación línea a línea — Rendimiento42.cs

Línea 1	using AceriaData.Application.Dtos; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	namespace AceriaData.Infrastructure.Repositories; → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 6	public sealed partial class OrdenRepositorio → Declara la clase concreta que implementa la funcionalidad del punto.
Línea 7	{ → Delimita el bloque sintáctico asociado.
Línea 8	public TrackingMetricaDto MedirConsultaConTrackingM4() → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 9	{ → Delimita el bloque sintáctico asociado.
Línea 10	_context.ChangeTracker.Clear(); → Limpia tracking antes de la demostración.
Línea 11	var consulta = _context.OrdenesFabricacion → Calcula y conserva el resultado que será validado o mostrado.
Línea 12	.AsTracking() → Fuerza tracking para hacer observable el ChangeTracker.
Línea 13	.OrderBy(o => o.Id); → Forma parte del orden determinista.
Línea 14	var sql = consulta.ToQueryString(); → Obtiene la representación SQL sin materializar la consulta.
Línea 15	var filas = consulta.ToList().Count; → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 16	return new TrackingMetricaDto → Devuelve el resultado calculado al llamador.
Línea 17	{ → Delimita el bloque sintáctico asociado.
Línea 18	Filas = filas, → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.

Línea 19	<code>EntidadesRastreadas = _context.ChangeTracker.Entries().Count(),</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 20	<code>Sql = sql</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 21	<code>};</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 22	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 24	<code>public TrackingMetricaDto MedirConsultaSinTrackingM4()</code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 25	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 26	<code>_context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 27	<code>var consulta = _context.OrdenesFabricacion</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 28	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 29	<code>.OrderBy(o => o.Id);</code> → Forma parte del orden determinista.
Línea 30	<code>var sql = consulta.ToQueryString();</code> → Obtiene la representación SQL sin materializar la consulta.
Línea 31	<code>var filas = consulta.ToList().Count;</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 32	<code>return new TrackingMetricaDto</code> → Devuelve el resultado calculado al llamador.
Línea 33	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 34	<code>Filas = filas,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 35	<code>EntidadesRastreadas = _context.ChangeTracker.Entries().Count(),</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 36	<code>Sql = sql</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 37	<code>};</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 38	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 39	<code>}</code> → Delimita el bloque sintáctico asociado.

Qué debe observarse en ejecución

La lectura del código debe completarse con la evidencia de ejecución: SQL traducido cuando corresponda, número de comandos, cardinalidad, columnas materializadas, estado del `ChangeTracker` y las comprobaciones específicas del punto. El hecho de que el código compile no demuestra por sí solo una mejora de rendimiento; por eso cada punto termina en una comprobación observable.

Reto conceptual. Explicar por qué dos consultas con SQL parecido pueden tener distinto coste de materialización.

Analogía operativa. Tracking es mantener una ficha viva de cada pieza; NoTracking es leerla sin abrir expediente.

Criterios de salida

- Relacionar LINQ con SQL o comandos ejecutados.
- Distinguir coste de servidor, transferencia, materialización y tracking.
- Justificar técnicas aplicadas y descartadas.

- Ejecutar el proyecto del punto y obtener 4.2 OK.

Punto 4.3 — AsNoTracking y AsNoTrackingWithIdentityResolution

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en las variantes de No Tracking en el proyecto AceriaData, comparando AsNoTracking con AsNoTrackingWithIdentityResolution, analizando el comportamiento de la caché de identidad y aplicando la variante adecuada según el escenario.

Objetivos de aprendizaje

- Comprender la diferencia entre AsNoTracking y AsNoTrackingWithIdentityResolution.
- Comprender el papel de la caché de identidad en las consultas sin tracking.
- Identificar el problema de las instancias duplicadas.
- Aplicar AsNoTrackingWithIdentityResolution cuando se cargan entidades relacionadas.
- Comprender el coste de la resolución de identidad.
- Analizar el impacto en memoria y rendimiento de cada variante.
- Aplicar estas variantes al proyecto AceriaData.

Consideraciones técnicas en EF Core 8

La resolución de identidad solo se demuestra si la misma clave aparece repetida. AceriaData usa Aleacion porque una misma aleación está relacionada con varias órdenes; PlanchaAcero pertenece a una sola orden y no es una evidencia válida.

Referencia oficial: <https://learn.microsoft.com/ef/core/querying/tracking>

Desarrollo teórico

Recordatorio del No Tracking

En el punto anterior se estudió AsNoTracking, que carga las entidades sin registrarlas en el Change Tracker. Las entidades no se rastrean, no se guarda una copia de sus valores originales y los cambios no se persisten con SaveChanges. Es la variante recomendada para consultas de solo lectura.

Ejemplo 1 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .AsNoTracking()
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea aplica AsNoTracking. La tercera línea materializa la consulta. Las entidades no se registran en el Change Tracker.

El problema de las instancias duplicadas

Cuando se usa AsNoTracking sin resolución de identidad, EF Core crea una instancia nueva por cada fila que devuelve la consulta. Si una entidad relacionada aparece varias veces en el resultado, EF Core crea varias instancias de la misma entidad. Esto provoca que la misma entidad exista varias veces en memoria, con posibles inconsistencias.

Ejemplo 2 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .AsNoTracking()
    .Include(o => o.Planchas)
    .Where(o => o.Cliente == "Constructora del Norte")
    .ToList();
```

CSHARP

```
foreach (var orden in ordenes)
{
    foreach (var plancha in orden.Planchas)
    {
        Console.WriteLine($"Plancha {plancha.Id} de la orden {plancha.OrdenId}");
    }
}
```

Nota técnica. El grafo Orden-Planchas no demuestra por sí solo identidad repetida porque cada PlanchaAcero pertenece a una sola orden. En 4.3 se usa Aleacion, que sí puede reaparecer con la misma clave.

La primera línea inicia la consulta. La segunda línea aplica AsNoTracking. La tercera línea incluye la colección de planchas. La cuarta línea filtra por cliente. La quinta línea materializa la consulta. El bucle itera sobre las órdenes y sus planchas. AsNoTracking no hace resolución de identidad. Este grafo con PlanchaAcero no demuestra por sí solo repetición de clave; AceriaData usa Aleacion compartida entre relaciones para observar el caso.

Qué es la resolución de identidad

La resolución de identidad es el mecanismo que garantiza que, dentro de una misma consulta, solo existe una instancia por cada entidad con una clave primaria concreta. Es el comportamiento de la caché de identidad, pero aplicado a consultas sin tracking. Se activa con AsNoTrackingWithIdentityResolution.

Ejemplo 3 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .AsNoTrackingWithIdentityResolution()
    .Include(o => o.Planchas)
    .Where(o => o.Cliente == "Constructora del Norte")
    .ToList();
```

CSHARP

Nota técnica. El grafo Orden-Planchas no demuestra por sí solo identidad repetida porque cada PlanchaAcero pertenece a una sola orden. En 4.3 se usa Aleacion, que sí puede reaparecer con la misma clave.

La primera línea inicia la consulta. La segunda línea aplica AsNoTrackingWithIdentityResolution. La tercera línea incluye la colección de planchas. La cuarta línea filtra por cliente. La quinta línea materializa la consulta. Aunque las entidades no se registran en el Change Tracker, EF Core mantiene una caché de identidad temporal para garantizar que solo haya una instancia por entidad.

El método AsNoTrackingWithIdentityResolution

AsNoTrackingWithIdentityResolution es un método de extensión que se aplica sobre IQueryable. Devuelve una nueva consulta que no registra las entidades en el Change Tracker, pero mantiene una caché de identidad temporal para resolver las referencias duplicadas.

Ejemplo 4 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .AsNoTrackingWithIdentityResolution()
    .Include(o => o.Planchas)
    .ToList();
```

CSHARP

Nota técnica. El grafo Orden-Planchas no demuestra por sí solo identidad repetida porque cada PlanchaAcero pertenece a una sola orden. En 4.3 se usa Aleacion, que sí puede reaparecer con la misma clave.

La primera línea inicia la consulta. La segunda línea aplica AsNoTrackingWithIdentityResolution. La tercera línea incluye la colección de planchas. La cuarta línea materializa la consulta. Las entidades no se registran en el Change Tracker. La resolución de identidad solo produce una diferencia observable cuando una misma clave reaparece en el resultado; AceriaData lo demuestra con Aleacion.

Diferencia entre AsNoTracking y AsNoTrackingWithIdentityResolution

La diferencia principal es el uso de la caché de identidad. AsNoTracking no realiza resolución de identidad: si una misma clave aparece varias veces en el resultado, pueden materializarse instancias distintas.

AsNoTrackingWithIdentityResolution usa una caché de identidad temporal: cada entidad con la misma clave primaria se resuelve a la misma instancia.

Ejemplo 5 (CSHARP).

```
// AsNoTracking: puede crear instancias distintas si una misma clave reaparece
var ordenes1 = context.OrdenesFabricacion
    .AsNoTracking()
    .Include(o => o.Planchas)
    .ToList();

// IdentityResolution: reutiliza una instancia por clave dentro de esta consulta
var ordenes2 = context.OrdenesFabricacion
    .AsNoTrackingWithIdentityResolution()
    .Include(o => o.Planchas)
    .ToList();
```

CSHARP

Nota técnica. El grafo Orden-Planchas no demuestra por sí solo identidad repetida porque cada PlanchaAcero pertenece a una sola orden. En 4.3 se usa Aleacion, que sí puede reaparecer con la misma clave.

La primera consulta usa AsNoTracking. La segunda consulta usa AsNoTrackingWithIdentityResolution. Ambas devuelven las mismas órdenes con sus planchas, pero la segunda garantiza que las entidades compartidas que aparecen varias veces sean la misma instancia.

Cuándo usar AsNoTracking

AsNoTracking se usa cuando las entidades no se van a modificar y no hay relaciones que puedan provocar instancias duplicadas. Es el caso de las consultas que devuelven una sola entidad por clave primaria o de las consultas que proyectan a DTOs.

Ejemplo 6 (CSHARP).

```
var resúmenes = context.OrdenesFabricacion
    .AsNoTracking()
    .Where(o => o.Estado == "Pendiente")
    .Select(o => new OrdenResumenDto
    {
        NumeroOrden = o.NumeroOrden,
        Cliente = o.Cliente,
        Estado = o.Estado
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea aplica AsNoTracking. La tercera línea filtra por estado. La cuarta línea proyecta al DTO. La quinta línea materializa la consulta. Como la proyección devuelve DTOs, no hay entidades que puedan duplicarse.

Cuándo usar AsNoTrackingWithIdentityResolution

AsNoTrackingWithIdentityResolution se usa cuando las entidades no se van a modificar pero hay relaciones que pueden provocar instancias duplicadas. Es el caso de las consultas con Include que cargan colecciones de navegación.

Ejemplo 7 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .AsNoTrackingWithIdentityResolution()
    .Include(o => o.Planchas)
    .Include(o => o.Detalle)
    .ToList();
```

CSHARP

Nota técnica. El grafo Orden-Planchas no demuestra por sí solo identidad repetida porque cada PlanchaAcero pertenece a una sola orden. En 4.3 se usa Aleacion, que sí puede reaparecer con la misma clave.

La primera línea inicia la consulta. La segunda línea aplica `AsNoTrackingWithIdentityResolution`. La tercera línea incluye la colección de planchas. La cuarta línea incluye la referencia al detalle. La quinta línea materializa la consulta. Las entidades no se registran en el Change Tracker, pero las planchas y el detalle se resuelven a instancias únicas.

El coste de la resolución de identidad

La resolución de identidad tiene un coste. EF Core debe mantener una caché de identidad temporal durante la materialización de la consulta. Esto consume memoria y CPU. En consultas con muchas filas, el coste puede ser significativo.

Ejemplo 8 (CSHARP).

```
var cronometro1 = Stopwatch.StartNew();
var ordenes1 = context.OrdenesFabricacion
    .AsNoTracking()
    .Include(o => o.Planchas)
    .ToList();
cronometro1.Stop();

var cronometro2 = Stopwatch.StartNew();
var ordenes2 = context.OrdenesFabricacion
    .AsNoTrackingWithIdentityResolution()
    .Include(o => o.Planchas)
    .ToList();
cronometro2.Stop();
```

CSHARP

Nota técnica. El grafo Orden-Planchas no demuestra por sí solo identidad repetida porque cada `PlanchaAcero` pertenece a una sola orden. En 4.3 se usa `Aleacion`, que sí puede reaparecer con la misma clave.

La primera sección mide el tiempo de `AsNoTracking`. La segunda sección mide el tiempo de `AsNoTrackingWithIdentityResolution`. La resolución de identidad añade trabajo de materialización; el impacto real debe medirse y no se presupone una diferencia temporal fija.

La caché de identidad temporal

La caché de identidad temporal se crea durante la materialización de la consulta y se destruye al finalizar. No persiste entre consultas. Esto significa que dos consultas distintas pueden devolver instancias distintas de la misma entidad, aunque se use `AsNoTrackingWithIdentityResolution`.

Ejemplo 9 (CSHARP).

```
var orden1 = context.OrdenesFabricacion
    .AsNoTrackingWithIdentityResolution()
    .FirstOrDefault(o => o.Id == 1);

var orden2 = context.OrdenesFabricacion
    .AsNoTrackingWithIdentityResolution()
    .FirstOrDefault(o => o.Id == 1);

Console.WriteLine($"Misma instancia: {ReferenceEquals(orden1, orden2)}");
```

CSHARP

La primera línea carga la orden con `AsNoTrackingWithIdentityResolution`. La segunda línea carga la misma orden con `AsNoTrackingWithIdentityResolution`. La tercera línea comprueba si son la misma instancia. El resultado es `false` porque la caché de identidad temporal se destruye al finalizar cada consulta.

La caché de identidad con Tracking

Con Tracking, la caché de identidad persiste durante toda la vida del `DbContext`. Si se carga la misma entidad dos veces con Tracking, EF Core devuelve la misma instancia.

Ejemplo 10 (CSHARP).

```
var orden1 = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
var orden2 = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);

Console.WriteLine($"Misma instancia: {ReferenceEquals(orden1, orden2)}");
```

CSHARP

La primera línea carga la orden con Tracking. La segunda línea carga la misma orden con Tracking. La tercera línea comprueba si son la misma instancia. El resultado es `true` porque la caché de identidad del Change Tracker devuelve la misma instancia.

El uso de AsNoTrackingWithIdentityResolution en proyecciones

AsNoTrackingWithIdentityResolution no tiene efecto en las proyecciones a tipos anónimos o DTOs. Las proyecciones devuelven objetos nuevos que no se registran en ninguna caché de identidad.

Ejemplo 11 (CSHARP).

```
var resmenes = context.OrdenesFabricacion
    .AsNoTrackingWithIdentityResolution()
    .Select(o => new OrdenResumenDto
    {
        NumeroOrden = o.NumeroOrden,
        Cliente = o.Cliente,
        Estado = o.Estado
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea aplica AsNoTrackingWithIdentityResolution. La tercera línea proyecta al DTO. La cuarta línea materializa la consulta. La resolución de identidad no tiene efecto porque la proyección devuelve DTOs.

El uso de AsNoTrackingWithIdentityResolution en consultas con Include

AsNoTrackingWithIdentityResolution es útil en consultas con Include que cargan colecciones de navegación. Garantiza que las entidades relacionadas que aparecen en varias entidades principales sean la misma instancia.

Ejemplo 12 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .AsNoTrackingWithIdentityResolution()
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .ThenInclude(oa => oa.Aleacion)
    .ToList();
```

CSHARP

Nota técnica. El grafo Orden-Planchas no demuestra por sí solo identidad repetida porque cada PlanchaAcero pertenece a una sola orden. En 4.3 se usa Aleacion, que sí puede reaparecer con la misma clave.

La primera línea inicia la consulta. La segunda línea aplica AsNoTrackingWithIdentityResolution. La tercera línea incluye la colección de planchas. La cuarta línea incluye la colección de entidades intermedias. La quinta línea incluye la aleación de cada entidad intermedia. La sexta línea materializa la consulta. Las aleaciones que aparecen en varias órdenes se resuelven a la misma instancia.

El proyecto AceriaData

En el proyecto AceriaData, se aplican las variantes de No Tracking según el escenario. Las consultas de solo lectura sin relaciones usan AsNoTracking. Las consultas de solo lectura con relaciones que pueden provocar duplicados usan AsNoTrackingWithIdentityResolution. Las consultas de escritura usan Tracking. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

AsNoTracking carga las entidades sin registrarlas en el Change Tracker.

AsNoTrackingWithIdentityResolution mantiene una caché de identidad temporal.

La caché de identidad garantiza una instancia por entidad con la misma clave.

AsNoTracking no realiza resolución de identidad; si una misma clave aparece varias veces pueden materializarse instancias distintas.

AsNoTrackingWithIdentityResolution resuelve las instancias duplicadas.

La resolución de identidad tiene un coste en memoria y CPU.

La caché de identidad temporal se destruye al finalizar la consulta.

Las proyecciones a DTOs no se benefician de la resolución de identidad.

En el proyecto AceriaData se aplican las variantes según el escenario.

Anclaje en AceriaData

El concepto se implementa en M04/PROYECTO/4.3. En la práctica se restaura, compila y ejecuta sobre SQL Server LocalDB.

Ejemplo ejecutable del concepto

El siguiente archivo es la implementación real de Infrastructure correspondiente al punto 4.3.

```
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    public IdentityResolutionMetricaDto MedirNoTrackingSinResolucionM4()
    {
        _context.ChangeTracker.Clear();
        var entidades = _context.OrdenesAleaciones
            .AsNoTracking()
            .OrderBy(oa => oa.OrdenFabricacionId)
            .Select(oa => oa.Aleacion)
            .ToList();

        return new IdentityResolutionMetricaDto
        {
            Filas = entidades.Count,
            ClavesUnicas = entidades.Select(a => a.Id).Distinct().Count(),
            InstanciasUnicas = entidades.Distinct(ReferenceEqualityComparer.Instance).Count()
        };
    }

    public IdentityResolutionMetricaDto MedirNoTrackingConResolucionM4()
    {
        _context.ChangeTracker.Clear();
        var entidades = _context.OrdenesAleaciones
            .AsNoTrackingWithIdentityResolution()
            .OrderBy(oa => oa.OrdenFabricacionId)
            .Select(oa => oa.Aleacion)
            .ToList();

        return new IdentityResolutionMetricaDto
        {
            Filas = entidades.Count,
            ClavesUnicas = entidades.Select(a => a.Id).Distinct().Count(),
            InstanciasUnicas = entidades.Distinct(ReferenceEqualityComparer.Instance).Count()
        };
    }
}
```

CSHARP

Explicación línea a línea — Rendimiento43.cs

Línea 1	using AceriaData.Application.Dtos; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	namespace AceriaData.Infrastructure.Repositories; → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 6	public sealed partial class OrdenRepositorio → Declara la clase concreta que implementa la funcionalidad del punto.
Línea 7	{ → Delimita el bloque sintáctico asociado.
Línea 8	public IdentityResolutionMetricaDto MedirNoTrackingSinResolucionM4() → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 9	{ → Delimita el bloque sintáctico asociado.
Línea 10	_context.ChangeTracker.Clear(); → Limpia tracking antes de la demostración.

Línea 11	<code>var entidades = _context.OrdenesAleaciones</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 12	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 13	<code>.OrderBy(oa => oa.OrdenFabricacionId)</code> → Forma parte del orden determinista.
Línea 14	<code>.Select(oa => oa.Aleacion)</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 15	<code>.ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 17	<code>return new IdentityResolutionMetricaDto</code> → Devuelve el resultado calculado al llamador.
Línea 18	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 19	<code>Filas = entidades.Count,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 20	<code>ClavesUnicas = entidades.Select(a => a.Id).Distinct().Count(),</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 21	<code>InstanciasUnicas = entidades.Distinct(ReferenceEqualityComparer.Instance).Count()</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 22	<code>};</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 23	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 25	<code>public IdentityResolutionMetricaDto MedirNoTrackingConResolucionM4()</code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 26	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 27	<code>_context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 28	<code>var entidades = _context.OrdenesAleaciones</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 29	<code>.AsNoTrackingWithIdentityResolution()</code> → Activa NoTracking con resolución temporal de identidad.
Línea 30	<code>.OrderBy(oa => oa.OrdenFabricacionId)</code> → Forma parte del orden determinista.
Línea 31	<code>.Select(oa => oa.Aleacion)</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 32	<code>.ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 34	<code>return new IdentityResolutionMetricaDto</code> → Devuelve el resultado calculado al llamador.
Línea 35	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 36	<code>Filas = entidades.Count,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 37	<code>ClavesUnicas = entidades.Select(a => a.Id).Distinct().Count(),</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 38	<code>InstanciasUnicas = entidades.Distinct(ReferenceEqualityComparer.Instance).Count()</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 39	<code>};</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.

Línea 40	} → Delimita el bloque sintáctico asociado.
Línea 41	} → Delimita el bloque sintáctico asociado.

Qué debe observarse en ejecución

La lectura del código debe completarse con la evidencia de ejecución: SQL traducido cuando corresponda, número de comandos, cardinalidad, columnas materializadas, estado del ChangeTracker y las comprobaciones específicas del punto. El hecho de que el código compile no demuestra por sí solo una mejora de rendimiento; por eso cada punto termina en una comprobación observable.

Reto conceptual. Predecir cuántas instancias habrá cuando cuatro relaciones apunten a dos aleaciones distintas.

Analogía operativa. La resolución de identidad evita crear dos fichas físicas para la misma clave dentro de una consulta.

Criterios de salida

- Relacionar LINQ con SQL o comandos ejecutados.
- Distinguir coste de servidor, transferencia, materialización y tracking.
- Justificar técnicas aplicadas y descartadas.
- Ejecutar el proyecto del punto y obtener 4.3 OK.

Punto 4.4 — Problema N+1: identificación y causas

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en el problema N+1 en el proyecto *AceriaData*, identificando sus causas, observando su impacto en el rendimiento y analizando el SQL generado por las consultas que lo provocan.

Objetivos de aprendizaje

- Comprender qué es el problema N+1 y por qué es un problema de rendimiento.
- Identificar las causas del problema N+1 en EF Core.
- Reconocer los patrones de código que provocan el problema N+1.
- Analizar el número de consultas ejecutadas con y sin el problema.
- Medir el impacto del problema en el tiempo de ejecución.
- Comprender la relación entre N+1 y la carga Lazy.
- Comprender la relación entre N+1 y las consultas en bucle.
- Aplicar la identificación del problema N+1 al proyecto *AceriaData*.

Consideraciones técnicas en EF Core 8

En *AceriaData*, Lazy Loading está desactivado. Para estudiar N+1 se provoca de forma explícita: una consulta para órdenes y una adicional por orden. Un interceptor cuenta los DbCommand reales.

Desarrollo teórico

Qué es el problema N+1

El problema N+1 es un problema de rendimiento que ocurre cuando se ejecuta una consulta para cargar las entidades principales y después una consulta adicional por cada entidad principal para cargar sus entidades relacionadas. Si hay N entidades principales, se ejecutan N+1 consultas: una para las principales y N para las relacionadas. El problema se agrava cuando N es grande, porque el número de consultas crece linealmente con el número de entidades.

Ejemplo 1 (CSHARP).

CSHARP

```
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    Console.WriteLine($"Orden {orden.NumeroOrden}: {orden.Planchas.Count} planchas");
}
```

Nota técnica. En AceriaData, Lazy Loading está desactivado. Acceder por sí solo a una navegación no cargada no dispara SQL. Este patrón solo produce N+1 si Lazy Loading está habilitado; en 4.4 se provoca N+1 explícitamente mediante una consulta relacionada dentro del bucle y se cuentan los comandos reales.

La primera línea carga todas las órdenes. El bucle itera sobre las órdenes. En cada iteración, se accede a orden.Planchas. Si la carga Lazy está habilitada, cada acceso provoca una consulta adicional. Si hay cien órdenes, se ejecutan ciento una consultas: una para las órdenes y cien para las planchas.

Por qué es un problema de rendimiento

Cada consulta adicional tiene un coste: se abre una conexión, se envía la sentencia SQL, se ejecuta en el servidor, se leen los resultados y se cierra la conexión. Aunque el coste de una sola consulta sea pequeño, el coste acumulado de N consultas puede ser significativo. En aplicaciones con muchas peticiones concurrentes, el problema N+1 puede saturar la base de datos y degradar el tiempo de respuesta.

Ejemplo 2 (CSHARP).

CSHARP

```
var ordenes = context.OrdenesFabricacion.ToList();

var cronometro = Stopwatch.StartNew();
foreach (var orden in ordenes)
{
    var total = orden.Planchas.Count;
}
cronometro.Stop();

Console.WriteLine($"Tiempo: {cronometro.ElapsedMilliseconds} ms para {ordenes.Count} órdenes");
```

Nota técnica. En AceriaData, Lazy Loading está desactivado. Acceder por sí solo a una navegación no cargada no dispara SQL. Este patrón solo produce N+1 si Lazy Loading está habilitado; en 4.4 se provoca N+1 explícitamente mediante una consulta relacionada dentro del bucle y se cuentan los comandos reales.

La primera línea carga todas las órdenes. La segunda línea inicia el cronómetro. El bucle accede a las planchas de cada orden. La penúltima línea detiene el cronómetro. La última línea muestra el tiempo. El tiempo crece linealmente con el número de órdenes.

La diferencia con una sola consulta

Una solución habitual es la carga anticipada con Include, que evita una consulta por cada entidad principal. Con una colección y el comportamiento por defecto puede resolverse con un único comando; con SplitQuery puede usar varios comandos acotados sin convertirse en N+1.

Ejemplo 3 (CSHARP).

CSHARP

```
// Con N+1: N+1 consultas
var ordenes1 = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes1)
{
    var total = orden.Planchas.Count;
}

// Sin N+1: 1 consulta
var ordenes2 = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();
foreach (var orden in ordenes2)
{
    var total = orden.Planchas.Count;
}
```

Nota técnica. En AceriaData, Lazy Loading está desactivado. Acceder por sí solo a una navegación no cargada no dispara SQL. Este patrón solo produce N+1 si Lazy Loading está habilitado; en 4.4 se provoca N+1 explícitamente mediante una consulta relacionada dentro del bucle y se cuentan los comandos reales.

La primera sección solo ejecutaría N+1 por el acceso a la navegación si Lazy Loading estuviera habilitado. La segunda usa carga anticipada y evita consultas por entidad; el número exacto de comandos depende de Single/Split Query.

Causas del problema N+1

El problema N+1 tiene varias causas. La primera es la carga Lazy: al acceder a una propiedad de navegación, EF Core ejecuta una consulta adicional. La segunda es ejecutar explícitamente una consulta relacionada dentro de un bucle; acceder a una navegación no cargada no dispara SQL cuando Lazy Loading está desactivado. Una proyección correlacionada no es por sí misma una causa de N+1 en EF Core 8; debe comprobarse la traducción y el número real de comandos. La cuarta es el uso de FirstOrDefault dentro de un bucle.

Ejemplo 4 (CSHARP).

```
// Causa 1: carga Lazy
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    var planchas = orden.Planchas;
}

// Causa 2: acceso en bucle sin Include
var ordenes2 = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes2)
{
    var total = orden.Planchas.Count;
}

// Causa 3: proyección sin ToList
var resultado = context.OrdenesFabricacion
    .Select(o => new
    {
        o.NumeroOrden,
        Planchas = o.Planchas.Select(p => p.Espesor)
    })
    .ToList();

// Causa 4: FirstOrDefault en bucle
var ordenes4 = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes4)
{
    var detalle = context.DetallesOrden.FirstOrDefault(d => d.OrdenId == orden.Id);
}
```

CSHARP

Nota técnica. En AceriaData, Lazy Loading está desactivado. Acceder por sí solo a una navegación no cargada no dispara SQL. Este patrón solo produce N+1 si Lazy Loading está habilitado; en 4.4 se provoca N+1 explícitamente mediante una consulta relacionada dentro del bucle y se cuentan los comandos reales.

La primera sección muestra la carga Lazy. La segunda sección muestra acceso a navegación; solo implicaría consultas adicionales con Lazy Loading habilitado. La tercera sección muestra una proyección correlacionada que debe analizarse por su SQL, no etiquetarse automáticamente como N+1. La cuarta sección muestra el FirstOrDefault en bucle.

El problema N+1 con carga Lazy

La carga Lazy es la causa más habitual del problema N+1. Al acceder a una propiedad de navegación, EF Core ejecuta una consulta adicional para cargarla. Si el acceso se produce dentro de un bucle, se ejecuta una consulta por cada iteración.

Ejemplo 5 (CSHARP).

```
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
```

CSHARP

```
Console.WriteLine($"Orden {orden.NumeroOrden}: {orden.Planchas.Count} planchas");
}
```

Nota técnica. En AceriaData, Lazy Loading está desactivado. Acceder por sí solo a una navegación no cargada no dispara SQL. Este patrón solo produce N+1 si Lazy Loading está habilitado; en 4.4 se provoca N+1 explícitamente mediante una consulta relacionada dentro del bucle y se cuentan los comandos reales.

La primera línea carga los órdenes. El bucle itera sobre los órdenes. En cada iteración, se accede a `orden.Planchas`. Si la carga Lazy está habilitada, se ejecuta una consulta adicional por cada orden.

El SQL generado tiene la siguiente forma:

Ejemplo 6 (SQL).

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit);

SELECT [p].[Id], [p].[Espesor], [p].[Peso], ...
FROM [PlanchasAcero] AS [p]
WHERE [p].[OrdenId] = 1 AND [p].[IsDeleted] = CAST(0 AS bit);

SELECT [p].[Id], [p].[Espesor], [p].[Peso], ...
FROM [PlanchasAcero] AS [p]
WHERE [p].[OrdenId] = 2 AND [p].[IsDeleted] = CAST(0 AS bit);

-- ... una consulta por cada orden
```

SQL

La primera sentencia carga los órdenes. Las siguientes sentencias cargan las planchas de cada orden. Se ejecutan tantas sentencias de planchas como órdenes haya.

El problema N+1 sin carga Lazy

El problema N+1 también puede ocurrir sin carga Lazy. Si se accede a una propiedad de navegación en un bucle sin Include y sin carga Lazy, la propiedad está vacía o es null. Si se usa `FirstOrDefault` dentro del bucle para cargar la entidad relacionada, se ejecuta una consulta por cada iteración.

Ejemplo 7 (CSHARP).

```
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    var planchas = context.PlanchasAcero
        .Where(p => p.OrdenId == orden.Id)
        .ToList();
    Console.WriteLine($"Orden {orden.NumeroOrden}: {planchas.Count} planchas");
}
```

CSHARP

La primera línea carga los órdenes. El bucle itera sobre los órdenes. En cada iteración, se ejecuta una consulta para cargar las planchas de la orden. Se ejecutan tantas consultas de planchas como órdenes haya.

El problema N+1 con FirstOrDefault

El uso de `FirstOrDefault` dentro de un bucle es otra causa del problema N+1. Se ejecuta una consulta por cada iteración para cargar una sola entidad.

Ejemplo 8 (CSHARP).

```
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    var detalle = context.DetallesOrden
        .FirstOrDefault(d => d.OrdenId == orden.Id);
    Console.WriteLine($"Orden {orden.NumeroOrden}: detalle {detalle?.ComposicionQuimica}");
}
```

CSHARP

La primera línea carga los órdenes. El bucle itera sobre los órdenes. En cada iteración, se ejecuta una consulta para cargar el detalle. Se ejecutan tantas consultas de detalle como órdenes haya.

El problema N+1 con proyección

Una proyección de colección puede traducirse a SQL en EF Core 8 y no debe clasificarse automáticamente como N+1. La evidencia válida es el SQL generado y el número de comandos ejecutados.

Ejemplo 9 (CSHARP).

```
var resultado = context.OrdenesFabricacion
    .Select(o => new
    {
        o.NumeroOrden,
        Planchas = o.Planchas.Select(p => p.Espesor)
    })
    .ToList();

foreach (var item in resultado)
{
    Console.WriteLine($"Orden {item.NumeroOrden}: {item.Planchas.Count()} planchas");
}
```

CSHARP

Nota técnica. En AceriaData, Lazy Loading está desactivado. Acceder por sí solo a una navegación no cargada no dispara SQL. Este patrón solo produce N+1 si Lazy Loading está habilitado; en 4.4 se provoca N+1 explícitamente mediante una consulta relacionada dentro del bucle y se cuentan los comandos reales.

La consulta proyecta una colección correlacionada y después materializa el resultado. En EF Core 8 debe observarse la traducción concreta; Count sobre la colección ya proyectada no implica por sí mismo una nueva consulta por orden.

El problema N+1 con Include

Include evita la carga relacionada mediante una consulta por cada principal. El acceso posterior a otra navegación solo generará SQL adicional si existe un mecanismo de carga como Lazy Loading o una consulta explícita.

Ejemplo 10 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();

foreach (var orden in ordenes)
{
    foreach (var plancha in orden.Planchas)
    {
        Console.WriteLine($"Plancha {plancha.Id} de la orden {plancha.Orden.NumeroOrden}");
    }
}
```

CSHARP

Nota técnica. En AceriaData, Lazy Loading está desactivado. Acceder por sí solo a una navegación no cargada no dispara SQL. Este patrón solo produce N+1 si Lazy Loading está habilitado; en 4.4 se provoca N+1 explícitamente mediante una consulta relacionada dentro del bucle y se cuentan los comandos reales.

La orden principal ya forma parte del grafo materializado y EF Core puede realizar relationship fixup de la referencia inversa. Este ejemplo no demuestra N+1 con la configuración actual; para demostrarlo se debe consultar explícitamente una relación dentro del bucle o habilitar Lazy Loading.

La identificación del problema N+1

El problema N+1 se identifica analizando el número de consultas ejecutadas. El logging de EF Core registra cada consulta. Si se ve un patrón de una consulta seguida de N consultas idénticas con parámetros distintos, hay un problema N+1.

Ejemplo 11 (CSHARP).

```
optionsBuilder.LogTo(
    Console.WriteLine,
    new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
    LogLevel.Information);
```

CSHARP

La primera línea habilita el logging. La segunda línea especifica el destino. La tercera línea especifica las categorías. La cuarta línea especifica el nivel. A partir de este momento, se registra cada consulta ejecutada.

El log muestra el siguiente patrón:

Ejemplo 12 (TEXT).

```
Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='60']
SELECT [o].[Id], [o].[NumeroOrden], ... FROM [OrdenesFabricacion] AS [o] WHERE ...

Executed DbCommand (1ms) [Parameters=[@__ordenId_0='1'], CommandType='Text', CommandTimeout='60']
SELECT [p].[Id], [p].[Espesor], ... FROM [PlanchasAcero] AS [p] WHERE [p].[OrdenId] = @__ordenId_0

Executed DbCommand (1ms) [Parameters=[@__ordenId_0='2'], CommandType='Text', CommandTimeout='60']
SELECT [p].[Id], [p].[Espesor], ... FROM [PlanchasAcero] AS [p] WHERE [p].[OrdenId] = @__ordenId_0
```

TEXT

El primer comando carga las órdenes. Los siguientes comandos cargan las planchas de cada orden con un parámetro distinto. Este patrón es el problema N+1.

El impacto del problema N+1

El impacto del problema N+1 crece con el número de entidades principales. Con diez órdenes, se ejecutan once consultas. Con cien órdenes, se ejecutan ciento una. Con mil órdenes, se ejecutan mil una. El tiempo de ejecución crece linealmente con el número de entidades.

Ejemplo 13 (CSHARP).

```
var cronometro = Stopwatch.StartNew();
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    var planchas = context.PlanchasAcero
        .Where(p => p.OrdenId == orden.Id)
        .ToList();
}
cronometro.Stop();
Console.WriteLine($"Tiempo: {cronometro.ElapsedMilliseconds} ms para {ordenes.Count} órdenes");
```

CSHARP

La primera línea inicia el cronómetro. La segunda línea carga las órdenes. El bucle carga las planchas de cada orden. La penúltima línea detiene el cronómetro. La última línea muestra el tiempo. El tiempo es proporcional al número de órdenes.

El problema N+1 en aplicaciones web

El problema N+1 es especialmente problemático en aplicaciones web porque el tiempo de respuesta es crítico. Cada consulta adicional añade latencia y consume recursos de la base de datos. En una aplicación con muchas peticiones concurrentes, el problema N+1 puede provocar la saturación de la base de datos.

Ejemplo 14 (CSHARP).

```
public IActionResult ObtenerOrdenesConPlanchas()
{
    var ordenes = _context.OrdenesFabricacion.ToList();
    foreach (var orden in ordenes)
    {
        var planchas = _context.PlanchasAcero
            .Where(p => p.OrdenId == orden.Id)
            .ToList();
    }
    return Ok(ordenes);
}
```

CSHARP

La primera línea declara el método. La segunda línea carga las órdenes. El bucle carga las planchas de cada orden. La última línea devuelve las órdenes. Cada petición ejecuta N+1 consultas.

El proyecto AceriaData

En el proyecto AceriaData, se identifica el problema N+1 en este punto. Se añaden métodos al repositorio que provocan el problema N+1 y métodos que lo evitan con Include. Se analiza el número de consultas ejecutadas en cada caso y el tiempo de ejecución. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

El problema N+1 ejecuta N+1 consultas por cargar N entidades principales y sus relacionadas.

Cada consulta adicional tiene un coste en conexión, ejecución y lectura.

El problema N+1 crece linealmente con el número de entidades.

La carga Lazy es la causa más habitual del problema N+1.

El acceso a una navegación dentro de un bucle provoca N+1 cuando existe Lazy Loading; sin él, se necesita una consulta explícita por iteración para producir N+1.

El uso de FirstOrDefault en un bucle lo provoca.

Una proyección correlacionada no se clasifica como N+1 sin observar primero su traducción y sus comandos.

Include evita consultas relacionadas por cada principal al cargar la navegación anticipadamente.

ThenInclude permite cargar anticipadamente navegaciones de niveles posteriores; su necesidad depende del grafo y de cómo se acceda después.

El logging de EF Core permite identificar el problema N+1.

En el proyecto AceriaData se identifica el problema N+1 en este punto.

Anclaje en AceriaData

El concepto se implementa en M04/PROYECTO/4.4. En la práctica se restaura, compila y ejecuta sobre SQL Server LocalDB.

Ejemplo ejecutable del concepto

El siguiente archivo es la implementación real de Infrastructure correspondiente al punto 4.4.

```
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepository
{
    public NMasUnoMetricaDto EjecutarNMasUnoM4()
    {
        SqlCommandCounterInterceptor.Instance.Reset();

        var ordenes = _context.OrdenesFabricacion
            .AsNoTracking()
            .OrderBy(o => o.Id)
            .Select(o => new { o.Id, o.NumeroOrden })
            .ToList();

        var totalPlanchas = 0;
        foreach (var orden in ordenes)
        {
            totalPlanchas += _context.PlanchasAcero
                .AsNoTracking()
                .Count(p => p.OrdenId == orden.Id);
        }

        return new NMasUnoMetricaDto
        {
            Ordenes = ordenes.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            Planchas = totalPlanchas
        };
    }
}
```

CSHARP

Explicación línea a línea — Rendimiento44.cs

Línea 1

using AceriaData.Application.Dtos; → Importa tipos o extensiones requeridos por esta implementación.

Línea 2	<code>using Microsoft.EntityFrameworkCore;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	<code>namespace AceriaData.Infrastructure.Repositories;</code> → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 6	<code>public sealed partial class OrdenRepositorio</code> → Declara la clase concreta que implementa la funcionalidad del punto.
Línea 7	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 8	<code>public NMasUnoMetricaDto EjecutarNMasUnoM4()</code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 9	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 10	<code>SqlCommandCounterInterceptor.Instance.Reset();</code> → Mide comandos SQL reales ejecutados.
Línea 12	<code>var ordenes = _context.OrdenesFabricacion</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 13	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 14	<code>.OrderBy(o => o.Id)</code> → Forma parte del orden determinista.
Línea 15	<code>.Select(o => new { o.Id, o.NumeroOrden })</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 16	<code>.ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 18	<code>var totalPlanchas = 0;</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 19	<code>foreach (var orden in ordenes)</code> → Recorre los elementos materializados para observar o validar cada resultado.
Línea 20	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 21	<code>totalPlanchas += _context.PlanchasAcero</code> → Continúa la implementación del punto con esta expresión: <code>totalPlanchas += _context.PlanchasAcero</code>
Línea 22	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 23	<code>.Count(p => p.OrdenId == orden.Id);</code> → Continúa la composición fluida invocando Count sobre el resultado de la línea anterior.
Línea 24	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 26	<code>return new NMasUnoMetricaDto</code> → Devuelve el resultado calculado al llamador.
Línea 27	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 28	<code>Ordenes = ordenes.Count,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 29	<code>ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),</code> → Mide comandos SQL reales ejecutados.
Línea 30	<code>Planchas = totalPlanchas</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 31	<code>};</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.

Línea 32	} → Delimita el bloque sintáctico asociado.
Línea 33	} → Delimita el bloque sintáctico asociado.

Qué debe observarse en ejecución

La lectura del código debe completarse con la evidencia de ejecución: SQL traducido cuando corresponda, número de comandos, cardinalidad, columnas materializadas, estado del ChangeTracker y las comprobaciones específicas del punto. El hecho de que el código compile no demuestra por sí solo una mejora de rendimiento; por eso cada punto termina en una comprobación observable.

Reto conceptual. Calcular y después medir cuántos comandos se producen para N órdenes.

Analogía operativa. N+1 es pedir una lista y volver a la ventanilla una vez por cada elemento.

Criterios de salida

- Relacionar LINQ con SQL o comandos ejecutados.
- Distinguir coste de servidor, transferencia, materialización y tracking.
- Justificar técnicas aplicadas y descartadas.
- Ejecutar el proyecto del punto y obtener 4.4 OK.

Punto 4.5 — Solución a N+1: Include, proyecciones y Split Queries

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en las soluciones al problema N+1 en el proyecto *AceriaData*, aplicando **Include**, **ThenInclude**, **proyecciones** y **Split Queries** para reducir el número de consultas y mejorar el rendimiento.

Objetivos de aprendizaje

- Comprender las técnicas para resolver el problema N+1.
- Aplicar **Include** para cargar colecciones de navegación en una sola consulta.
- Aplicar **ThenInclude** para cargar relaciones de segundo nivel.
- Aplicar proyecciones para reducir el volumen de datos y evitar consultas adicionales.
- Aplicar **AsSplitQuery** cuando se incluyen varias colecciones.
- Comparar el número de consultas y el tiempo de ejecución de cada técnica.
- Analizar el SQL generado por cada solución.
- Aplicar estas soluciones al proyecto *AceriaData*.

Consideraciones técnicas en EF Core 8

No existe una solución universal al N+1. **Include** sirve para grafos; una proyección cuando solo se necesitan campos concretos; **SplitQuery** puede reducir explosión cartesiana con varias colecciones a costa de más roundtrips.

Desarrollo teórico

Recordatorio del problema N+1

En el punto anterior se estudió el problema N+1: una consulta para las entidades principales y N consultas adicionales para las entidades relacionadas. El problema crece linealmente con el número de entidades principales y degrada el rendimiento de forma significativa. La solución consiste en cargar todas las entidades relacionadas en el menor número de consultas posible.

Ejemplo 1 (CSHARP).

CSHARP

```
// Problema N+1: N+1 consultas
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    var planchas = context.PlanchasAcero
        .Where(p => p.OrdenId == orden.Id)
        .ToList();
}
```

La primera línea carga las órdenes. El bucle carga las planchas de cada orden. Se ejecutan N+1 consultas.

Solución con Include

Una solución directa es usar Include para carga anticipada. En modo Single Query, una colección suele resolverse mediante JOIN en un único comando; en modo Split Query, EF separa la colección en un comando adicional.

Ejemplo 2 (CSHARP).

CSHARP

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();

foreach (var orden in ordenes)
{
    Console.WriteLine($"Orden {orden.NumeroOrden}: {orden.Planchas.Count} planchas");
}
```

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea materializa la consulta. El bucle accede a las planchas sin ejecutar consultas adicionales. Se ejecuta una sola consulta.

El SQL generado tiene la siguiente forma:

Ejemplo 3 (SQL).

SQL

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [p].[Id], [p].[Espesor], [p].[Peso]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id]
```

La primera línea selecciona las columnas de ambas tablas. La segunda línea indica la tabla principal. La tercera línea combina con la tabla de planchas. La cuarta línea aplica el filtro global. La quinta línea ordena por el Id de la orden y el Id de la plancha.

Solución con ThenInclude

Cuando se necesita cargar una relación de segundo nivel, se usa ThenInclude después de Include. Esto permite cargar las entidades relacionadas de las entidades relacionadas en una sola consulta.

Ejemplo 4 (CSHARP).

CSHARP

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.OrdenesAleaciones)
    .ThenInclude(oa => oa.Aleacion)
    .ToList();

foreach (var orden in ordenes)
{
    foreach (var ordenAleacion in orden.OrdenesAleaciones)
    {
        Console.WriteLine($"Orden {orden.NumeroOrden}: aleación {ordenAleacion.Aleacion.Nombre}");
    }
}
```

La primera línea inicia la consulta. La segunda línea incluye la colección de entidades intermedias. La tercera línea incluye la aleación de cada entidad intermedia. La cuarta línea materializa la consulta. Los bucles acceden a las aleaciones sin ejecutar consultas adicionales. Se ejecuta una sola consulta.

El SQL generado tiene la siguiente forma:

Ejemplo 5 (SQL).

SQL

```
SELECT [o].[Id], [o].[NumeroOrden], ..., [oa].[OrdenFabricacionId], [oa].[AleacionId], ..., [a].[Id], [a].[Nombre], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [OrdenesAleaciones] AS [oa] ON [o].[Id] = [oa].[OrdenFabricacionId]
LEFT JOIN [Aleaciones] AS [a] ON [oa].[AleacionId] = [a].[Id]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [oa].[OrdenFabricacionId], [oa].[AleacionId]
```

La primera línea selecciona las columnas de las tres tablas. La segunda línea indica la tabla principal. Las siguientes líneas combinan con las tablas relacionadas. La penúltima línea aplica el filtro global. La última línea ordena por las claves.

Solución con proyecciones

Las proyecciones permiten seleccionar solo las columnas necesarias y pueden evitar consultas por entidad cuando toda la forma se traduce al servidor. Debe verificarse la traducción y el número de comandos reales.

Ejemplo 6 (CSHARP).

CSHARP

```
var resmenes = context.OrdenesFabricacion
    .Select(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        Planchas = o.Planchas.Select(p => new { p.Espesor, p.Peso }).ToList()
    })
    .ToList();
```

La primera línea inicia la consulta. La segunda línea proyecta cada orden a un tipo anónimo. La tercera línea incluye el número de orden. La cuarta línea incluye el cliente. La quinta línea proyecta la colección de planchas. La sexta línea materializa la consulta. Se ejecuta una sola consulta con las columnas proyectadas.

El SQL generado tiene la siguiente forma:

Ejemplo 7 (SQL).

SQL

```
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Espesor], [p].[Peso], [p].[Id]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id]
```

La primera línea selecciona solo las columnas proyectadas. La segunda línea indica la tabla principal. La tercera línea combina con la tabla de planchas. La cuarta línea aplica el filtro global. La quinta línea ordena por el Id de la orden y el Id de la plancha.

Solución con AsSplitQuery

Cuando se incluyen varias colecciones en la misma consulta, EF Core genera un producto cartesiano que multiplica las filas. AsSplitQuery divide la consulta en varias consultas separadas, evitando el producto cartesiano.

Ejemplo 8 (CSHARP).

CSHARP

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .ThenInclude(oa => oa.Aleacion)
    .AsSplitQuery()
    .ToList();
```

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la colección de entidades intermedias. La cuarta línea incluye la aleación de cada entidad intermedia. La quinta línea divide la consulta en varias. La sexta línea materializa la consulta. Se ejecutan tres consultas: una para las órdenes, una para las planchas y una para las entidades intermedias con sus aleaciones.

El SQL generado incluye tres consultas:

Ejemplo 9 (SQL).

SQL

```
-- Consulta 1: órdenes
SELECT [o].[Id], [o].[NumeroOrden], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit);

-- Consulta 2: planchas
SELECT [p].[Id], [p].[Espesor], ..., [o].[Id]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit);

-- Consulta 3: entidades intermedias con aleaciones
SELECT [oa].[OrdenFabricacionId], [oa].[AleacionId], ..., [a].[Id], [a].[Nombre], ...
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [OrdenesAleaciones] AS [oa] ON [o].[Id] = [oa].[OrdenFabricacionId]
INNER JOIN [Aleaciones] AS [a] ON [oa].[AleacionId] = [a].[Id]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
```

La primera consulta carga las órdenes. La segunda consulta carga las planchas. La tercera consulta carga las entidades intermedias con sus aleaciones. EF Core combina los resultados en memoria.

Comparación entre Include y AsSplitQuery

Include con una sola colección en modo Single Query usa normalmente un comando con JOIN. AsSplitQuery con una colección genera el comando de principales y otro para la colección. Con varias colecciones hermanas, Single Query puede sufrir explosión cartesiana y SplitQuery añade un comando por colección.

Ejemplo 10 (CSHARP).

CSHARP

```
// Include con una colección: 1 consulta
var ordenes1 = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();

// AsSplitQuery con una colección: principal + colección (2 comandos)
var ordenes2 = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .AsSplitQuery()
    .ToList();

// Include con dos colecciones: 1 consulta con producto cartesiano
var ordenes3 = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .ToList();

// AsSplitQuery con dos colecciones: 3 consultas sin producto cartesiano
var ordenes4 = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSplitQuery()
    .ToList();
```

La primera consulta usa Single Query con una colección. La segunda usa Split Query y requiere dos comandos. La tercera incluye dos colecciones hermanas y puede generar explosión cartesiana. La cuarta usa SplitQuery y ejecuta tres comandos: principal más uno por colección.

Comparación entre Include y proyecciones

Include carga las entidades completas, incluyendo todas sus columnas. Las proyecciones cargan solo las columnas proyectadas. Las proyecciones son más eficientes cuando no se necesitan todas las columnas.

Ejemplo 11 (CSHARP).

CSHARP

```
// Include: carga todas las columnas
var ordenes1 = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();

// Proyección: carga solo las columnas proyectadas
var ordenes2 = context.OrdenesFabricacion
    .Select(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        Planchas = o.Planchas.Select(p => new { p.Espesor, p.Peso }).ToList()
    })
    .ToList();
```

```

    })
    .ToList();

```

La primera consulta carga todas las columnas de las órdenes y las planchas. La segunda consulta carga solo las columnas proyectadas. La segunda transfiere menos columnas; el impacto total debe medirse junto con cardinalidad, materialización y plan del servidor.

Cuándo usar cada solución

La elección de la solución depende del escenario. Include es adecuado cuando se necesitan las entidades completas y solo hay una colección. ThenInclude es adecuado cuando se necesitan relaciones de segundo nivel. Las proyecciones son adecuadas cuando solo se necesitan algunas columnas. AsSplitQuery es adecuado cuando se incluyen varias colecciones y se quiere evitar el producto cartesiano.

Ejemplo 12 (CSHARP).

```

// Include: entidades completas, una colección
var ordenes1 = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();

// Proyección: solo algunas columnas
var ordenes2 = context.OrdenesFabricacion
    .Select(o => new { o.NumeroOrden, o.Cliente })
    .ToList();

// AsSplitQuery: varias colecciones
var ordenes3 = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSplitQuery()
    .ToList();

```

CSHARP

La primera consulta usa Include. La segunda usa proyección. La tercera usa AsSplitQuery.

La combinación de soluciones

Las soluciones se pueden combinar. Por ejemplo, se puede usar AsNoTracking con Include, o AsNoTrackingWithIdentityResolution con AsSplitQuery, o proyecciones con AsNoTracking. La combinación adecuada depende del escenario.

Ejemplo 13 (CSHARP).

```

var ordenes = context.OrdenesFabricacion
    .AsNoTrackingWithIdentityResolution()
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .ThenInclude(oa => oa.Aleacion)
    .AsSplitQuery()
    .ToList();

```

CSHARP

La primera línea inicia la consulta. La segunda línea aplica AsNoTrackingWithIdentityResolution. La tercera línea incluye la colección de planchas. La cuarta línea incluye la colección de entidades intermedias. La quinta línea incluye la aleación de cada entidad intermedia. La sexta línea divide la consulta en varias. La séptima línea materializa la consulta.

El proyecto AceriaData

En el proyecto AceriaData, se aplican las soluciones al problema N+1 en este punto. Se añaden métodos al repositorio que usan Include, ThenInclude, proyecciones y AsSplitQuery. Se comparan el número de consultas y el tiempo de ejecución de cada técnica. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

La solución al problema N+1 es cargar las entidades relacionadas en menos consultas.

Include carga las entidades relacionadas en una sola consulta.

ThenInclude carga las relaciones de segundo nivel.

Las proyecciones cargan solo las columnas necesarias.

AsSplitQuery divide la consulta en varias para evitar el producto cartesiano.

Include con una colección ejecuta una sola consulta.

Include con varias colecciones genera un producto cartesiano.

AsSplitQuery con varias colecciones ejecuta varias consultas sin producto cartesiano.

Las soluciones se pueden combinar con AsNoTracking y AsNoTrackingWithIdentityResolution.

En el proyecto AceriaData se aplican las soluciones en este punto.

Anclaje en AceriaData

El concepto se implementa en M04/PROYECTO/4.5. En la práctica se restaura, compila y ejecuta sobre SQL Server LocalDB.

Ejemplo ejecutable del concepto

El siguiente archivo es la implementación real de Infrastructure correspondiente al punto 4.5.

CSHARP

```
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepository
{
    public SolucionNMasUnoMetricaDto EjecutarIncludeContraNMasUnoM4()
    {
        SqlCommandCounterInterceptor.Instance.Reset();
        var ordenes = _context.OrdenesFabricacion
            .AsNoTracking()
            .Include(o => o.Planchas)
            .OrderBy(o => o.Id)
            .ToList();

        return new SolucionNMasUnoMetricaDto
        {
            Ordenes = ordenes.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            ElementosRelacionados = ordenes.Sum(o => o.Planchas.Count)
        };
    }

    public SolucionNMasUnoMetricaDto EjecutarProyeccionContraNMasUnoM4()
    {
        SqlCommandCounterInterceptor.Instance.Reset();
        var ordenes = _context.OrdenesFabricacion
            .AsNoTracking()
            .OrderBy(o => o.Id)
            .Select(o => new { o.Id, TotalPlanchas = o.Planchas.Count })
            .ToList();

        return new SolucionNMasUnoMetricaDto
        {
            Ordenes = ordenes.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            ElementosRelacionados = ordenes.Sum(o => o.TotalPlanchas)
        };
    }

    public SolucionNMasUnoMetricaDto EjecutarSplitQueryContraNMasUnoM4()
    {
        SqlCommandCounterInterceptor.Instance.Reset();
        var ordenes = _context.OrdenesFabricacion
            .AsNoTrackingWithIdentityResolution()
            .Include(o => o.Planchas)
            .Include(o => o.OrdenesAleaciones)
            .ThenInclude(oa => oa.Aleacion)
            .AsSplitQuery()
            .OrderBy(o => o.Id)
            .ToList();

        return new SolucionNMasUnoMetricaDto
        {
            Ordenes = ordenes.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            ElementosRelacionados = ordenes.Sum(o => o.Planchas.Count + o.OrdenesAleaciones.Count)
        };
    }
}
```

```
}
}
```

Explicación línea a línea — Rendimiento45.cs

Línea 1	<code>using AceriaData.Application.Dtos;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	<code>using Microsoft.EntityFrameworkCore;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	<code>namespace AceriaData.Infrastructure.Repositories;</code> → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 6	<code>public sealed partial class OrdenRepositorio</code> → Declara la clase concreta que implementa la funcionalidad del punto.
Línea 7	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 8	<code>public SolucionNMasUnoMetricaDto EjecutarIncludeContraNMasUnoM4()</code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 9	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 10	<code>SqlCommandCounterInterceptor.Instance.Reset();</code> → Mide comandos SQL reales ejecutados.
Línea 11	<code>var ordenes = _context.OrdenesFabricacion</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 12	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 13	<code>.Include(o => o.Planchas)</code> → Define la navegación relacionada que debe cargarse.
Línea 14	<code>.OrderBy(o => o.Id)</code> → Forma parte del orden determinista.
Línea 15	<code>.ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 17	<code>return new SolucionNMasUnoMetricaDto</code> → Devuelve el resultado calculado al llamador.
Línea 18	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 19	<code>Ordenes = ordenes.Count,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 20	<code>ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),</code> → Mide comandos SQL reales ejecutados.
Línea 21	<code>ElementosRelacionados = ordenes.Sum(o => o.Planchas.Count)</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 22	<code>};</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 23	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 25	<code>public SolucionNMasUnoMetricaDto EjecutarProyeccionContraNMasUnoM4()</code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 26	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 27	<code>SqlCommandCounterInterceptor.Instance.Reset();</code> → Mide comandos SQL reales ejecutados.
Línea 28	<code>var ordenes = _context.OrdenesFabricacion</code> → Calcula y conserva el resultado que será validado o mostrado.

Línea 29	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 30	<code>.OrderBy(o => o.Id)</code> → Forma parte del orden determinista.
Línea 31	<code>.Select(o => new { o.Id, TotalPlanchas = o.Planchas.Count })</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 32	<code>.ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 34	<code>return new SolucionNMasUnoMetricaDto</code> → Devuelve el resultado calculado al llamador.
Línea 35	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 36	<code>Ordenes = ordenes.Count,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 37	<code>ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),</code> → Mide comandos SQL reales ejecutados.
Línea 38	<code>ElementosRelacionados = ordenes.Sum(o => o.TotalPlanchas)</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 39	<code>};</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 40	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 42	<code>public SolucionNMasUnoMetricaDto EjecutarSplitQueryContraNMasUnoM4()</code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 43	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 44	<code>SqlCommandCounterInterceptor.Instance.Reset();</code> → Mide comandos SQL reales ejecutados.
Línea 45	<code>var ordenes = _context.OrdenesFabricacion</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 46	<code>.AsNoTrackingWithIdentityResolution()</code> → Activa NoTracking con resolución temporal de identidad.
Línea 47	<code>.Include(o => o.Planchas)</code> → Define la navegación relacionada que debe cargarse.
Línea 48	<code>.Include(o => o.OrdenesAleaciones)</code> → Define la navegación relacionada que debe cargarse.
Línea 49	<code>.ThenInclude(oa => oa.Aleacion)</code> → Define la navegación relacionada que debe cargarse.
Línea 50	<code>.AsSplitQuery()</code> → Divide la carga relacionada en varios comandos SQL.
Línea 51	<code>.OrderBy(o => o.Id)</code> → Forma parte del orden determinista.
Línea 52	<code>.ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 54	<code>return new SolucionNMasUnoMetricaDto</code> → Devuelve el resultado calculado al llamador.
Línea 55	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 56	<code>Ordenes = ordenes.Count,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 57	<code>ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),</code> → Mide comandos SQL reales ejecutados.

Línea 58	<code>ElementosRelacionados = ordenes.Sum(o => o.Planchas.Count + o.OrdenesAleaciones.Count) → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.</code>
Línea 59	<code>}; → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.</code>
Línea 60	<code>} → Delimita el bloque sintáctico asociado.</code>
Línea 61	<code>} → Delimita el bloque sintáctico asociado.</code>

Qué debe observarse en ejecución

La lectura del código debe completarse con la evidencia de ejecución: SQL traducido cuando corresponda, número de comandos, cardinalidad, columnas materializadas, estado del `ChangeTracker` y las comprobaciones específicas del punto. El hecho de que el código compile no demuestra por sí solo una mejora de rendimiento; por eso cada punto termina en una comprobación observable.

Reto conceptual. Elegir entre `Include`, proyección o `SplitQuery` para tres escenarios y justificar el coste dominante.

Analogía operativa. Optimizar N+1 es decidir si conviene traer el expediente completo, un resumen o varios lotes coordinados.

Criterios de salida

- Relacionar LINQ con SQL o comandos ejecutados.
- Distinguir coste de servidor, transferencia, materialización y tracking.
- Justificar técnicas aplicadas y descartadas.
- Ejecutar el proyecto del punto y obtener 4.5 OK.

Punto 4.6 — Over-fetching: causas y soluciones

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en el over-fetching en el proyecto `AceriaData`, identificando sus causas, observando su impacto en el rendimiento y aplicando proyecciones y otras técnicas para reducirlo.

Objetivos de aprendizaje

- Comprender qué es el over-fetching y por qué es un problema de rendimiento.
- Identificar las causas del over-fetching en EF Core.
- Diferenciar entre over-fetching de columnas y over-fetching de filas.
- Aplicar proyecciones para reducir el over-fetching de columnas.
- Aplicar filtros y paginación para reducir el over-fetching de filas.
- Analizar el SQL generado por consultas con y sin over-fetching.
- Medir el impacto del over-fetching en el tiempo de ejecución y en el volumen de datos.
- Aplicar estas técnicas al proyecto `AceriaData`.

Consideraciones técnicas en EF Core 8

Over-fetching se diagnostica observando la forma real del `SELECT`. La práctica compara igual cardinalidad con entidad completa frente a proyección DTO.

Desarrollo teórico

Qué es el over-fetching

El over-fetching es el problema de rendimiento que ocurre cuando se cargan más datos de los necesarios. Puede ocurrir en dos dimensiones: en columnas, cuando se cargan todas las columnas de una entidad aunque solo se necesiten algunas; y en filas, cuando se cargan todas las filas de una tabla aunque solo se necesiten algunas. El over-fetching consume ancho de banda, memoria y CPU, tanto en el servidor como en el cliente.

Ejemplo 1 (CSHARP).

```
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    Console.WriteLine($"{orden.NumeroOrden} - {orden.Cliente}");
}
```

CSHARP

La primera línea carga todas las órdenes con todas sus columnas. El bucle solo usa las propiedades NumeroOrden y Cliente. Las columnas Id, FechaCreacion, Estado, Observaciones, IsDeleted y DeletedAt se cargan pero no se usan. Esto es over-fetching de columnas.

Over-fetching de columnas

El over-fetching de columnas ocurre cuando se cargan todas las columnas de una entidad aunque solo se necesiten algunas. Es habitual cuando se cargan entidades completas para mostrarlas en una lista o para calcular un resumen. La solución es proyectar solo las columnas necesarias.

Ejemplo 2 (CSHARP).

```
var resmenes = context.OrdenesFabricacion
    .Select(o => new { o.NumeroOrden, o.Cliente })
    .ToList();

foreach (var resumen in resmenes)
{
    Console.WriteLine($"{resumen.NumeroOrden} - {resumen.Cliente}");
}
```

CSHARP

La primera línea inicia la consulta. La segunda línea proyecta solo las columnas NumeroOrden y Cliente. La tercera línea materializa la consulta. El bucle itera sobre los resúmenes. El SQL generado solo incluye las columnas proyectadas.

El SQL generado tiene la siguiente forma:

Ejemplo 3 (SQL).

```
SELECT [o].[NumeroOrden], [o].[Cliente]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
```

SQL

La primera línea selecciona solo las dos columnas proyectadas. La segunda línea indica la tabla. La tercera línea aplica el filtro global. Las demás columnas no se cargan.

Over-fetching de filas

El over-fetching de filas ocurre cuando se cargan todas las filas de una tabla aunque solo se necesiten algunas. Es habitual cuando se cargan todas las entidades para filtrarlas o paginarlas en memoria. La solución es aplicar filtros y paginación en la consulta.

Ejemplo 4 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.FechaCreacion)
    .Skip(0)
    .Take(10)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por estado. La tercera línea ordena por fecha. La cuarta línea salta las primeras filas. La quinta línea toma las primeras diez filas. La sexta línea materializa la consulta. El SQL generado incluye las cláusulas WHERE, ORDER BY, OFFSET y FETCH.

El SQL generado tiene la siguiente forma:

Ejemplo 5 (SQL).

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [o].[Observaciones], [o].[IsDeleted], [o].[DeletedAt]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente' AND [o].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[FechaCreacion]
OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY
```

SQL

La primera línea selecciona todas las columnas. La segunda línea indica la tabla. La tercera línea filtra por estado. La cuarta línea ordena por fecha. La quinta línea limita a diez filas. Solo se transfieren diez filas aunque la tabla tenga más.

Over-fetching de columnas y filas combinado

El over-fetching puede ocurrir en columnas y filas a la vez. Se cargan todas las columnas de todas las filas aunque solo se necesiten algunas columnas de algunas filas. La solución es combinar proyecciones con filtros y paginación.

Ejemplo 6 (CSHARP).

```
var resúmenes = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.FechaCreacion)
    .Skip(0)
    .Take(10)
    .Select(o => new { o.NumeroOrden, o.Cliente })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por estado. La tercera línea ordena por fecha. La cuarta línea salta las primeras filas. La quinta línea toma las primeras diez filas. La sexta línea proyecta solo dos columnas. La séptima línea materializa la consulta. El SQL generado solo incluye las columnas proyectadas y las filas filtradas.

El SQL generado tiene la siguiente forma:

Ejemplo 7 (SQL).

```
SELECT [o].[NumeroOrden], [o].[Cliente]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente' AND [o].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[FechaCreacion]
OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY
```

SQL

La primera línea selecciona solo las columnas proyectadas. La segunda línea indica la tabla. La tercera línea filtra por estado. La cuarta línea ordena por fecha. La quinta línea limita a diez filas. Solo se transfieren diez filas con dos columnas cada una.

Causas del over-fetching

El over-fetching tiene varias causas. La primera es cargar entidades completas para mostrarlas en una lista. La segunda es cargar todas las filas para filtrarlas o paginarlas en memoria. La tercera es usar Include con entidades relacionadas aunque no se necesiten. La cuarta es cargar columnas de tipo nvarchar(max) que ocupan mucho espacio.

Ejemplo 8 (CSHARP).

```
// Causa 1: entidad completa para una lista
var ordenes = context.OrdenesFabricacion.ToList();

// Causa 2: todas las filas para filtrar en memoria
var pendientes = context.OrdenesFabricacion.ToList().Where(o => o.Estado == "Pendiente");

// Causa 3: Include innecesario
var ordenesConPlanchas = context.OrdenesFabricacion.Include(o => o.Planchas).ToList();
```

CSHARP

```
// Causa 4: columnas de tipo nvarchar(max)
public string Observaciones { get; set; } = string.Empty;
```

La primera sección carga entidades completas para una lista. La segunda sección carga todas las filas para filtrar en memoria. La tercera sección incluye planchas aunque no se necesiten. La cuarta sección declara una columna de tipo `nvarchar(max)`.

Soluciones al over-fetching de columnas

La solución al over-fetching de columnas es proyectar solo las columnas necesarias. Se usan proyecciones a tipos anónimos o DTOs. Las proyecciones reducen el volumen de datos transferidos y el tiempo de materialización.

Ejemplo 9 (CSHARP).

```
var resmenes = context.OrdenesFabricacion
    .Select(o => new OrdenResumenDto
    {
        NumeroOrden = o.NumeroOrden,
        Cliente = o.Cliente,
        Estado = o.Estado
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea proyecta al DTO. Las siguientes líneas asignan las propiedades. La penúltima línea cierra la proyección. La última línea materializa la consulta. El SQL generado solo incluye las columnas proyectadas.

Soluciones al over-fetching de filas

La solución al over-fetching de filas es aplicar filtros y paginación en la consulta. Se usan `Where` para filtrar, `OrderBy` para ordenar, `Skip` y `Take` para paginar. Los filtros y la paginación se aplican en el servidor, no en memoria.

Ejemplo 10 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.FechaCreacion)
    .Skip(0)
    .Take(10)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por estado. La tercera línea ordena por fecha. La cuarta línea salta las primeras filas. La quinta línea toma las primeras diez filas. La sexta línea materializa la consulta. El SQL generado incluye las cláusulas `WHERE`, `ORDER BY`, `OFFSET` y `FETCH`.

Soluciones al over-fetching con Include

El over-fetching con `Include` ocurre cuando se cargan entidades relacionadas aunque no se necesiten. La solución es aplicar `Include` solo cuando se necesiten las entidades relacionadas, y usar proyecciones cuando solo se necesiten algunas columnas de las entidades relacionadas.

Ejemplo 11 (CSHARP).

```
// Include innecesario: carga todas las planchas
var ordenesConPlanchas = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();

// Proyección: carga solo el número de planchas
var ordenesConNumeroPlanchas = context.OrdenesFabricacion
    .Select(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        TotalPlanchas = o.Planchas.Count()
    })
    .ToList();
```

CSHARP

La primera sección carga todas las planchas aunque solo se necesite el número. La segunda sección proyecta solo el número de planchas. La segunda es más eficiente porque no transfiere todas las columnas de las planchas.

El impacto del over-fetching

El impacto del over-fetching crece con el número de columnas y de filas. Cuantas más columnas y filas se carguen innecesariamente, mayor es el consumo de ancho de banda, memoria y CPU. En aplicaciones con tablas grandes, el over-fetching puede degradar el rendimiento de forma significativa.

Ejemplo 12 (CSHARP).

```
var cronometro1 = Stopwatch.StartNew();
var ordenes = context.OrdenesFabricacion.ToList();
cronometro1.Stop();

var cronometro2 = Stopwatch.StartNew();
var resmenes = context.OrdenesFabricacion
    .Select(o => new { o.NumeroOrden, o.Cliente })
    .ToList();
cronometro2.Stop();

Console.WriteLine($"Entidades completas: {cronometro1.ElapsedMilliseconds} ms");
Console.WriteLine($"Proyección: {cronometro2.ElapsedMilliseconds} ms");
```

CSHARP

La primera sección mide el tiempo de cargar entidades completas. La segunda sección mide el tiempo de cargar la proyección. La segunda es más rápida porque transfiere menos datos.

El over-fetching en aplicaciones web

El over-fetching es especialmente problemático en aplicaciones web porque el ancho de banda es limitado y el tiempo de respuesta es crítico. Cada byte innecesario que se transfiere añade latencia y consume recursos. En una aplicación con muchas peticiones concurrentes, el over-fetching puede provocar la saturación del servidor.

Ejemplo 13 (CSHARP).

```
public IActionResult ObtenerOrdenes()
{
    var ordenes = _context.OrdenesFabricacion.ToList();
    return Ok(ordenes);
}
```

CSHARP

La primera línea declara el método. La segunda línea carga todas las órdenes con todas sus columnas. La tercera línea devuelve las órdenes en la respuesta. Cada petición transfiere todas las columnas de todas las órdenes.

El over-fetching y las columnas de tipo nvarchar(max)

Las columnas de tipo nvarchar(max) ocupan mucho espacio porque no tienen límite de longitud. Cargar estas columnas innecesariamente provoca un over-fetching significativo. La solución es proyectar solo las columnas necesarias o configurar una longitud máxima.

Ejemplo 14 (CSHARP).

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string? Observaciones { get; set; }
}

// Configuración con longitud máxima
modelBuilder.Entity<OrdenFabricacion>()
    .Property(o => o.Observaciones)
    .HasMaxLength(500);
```

CSHARP

La primera sección declara la entidad con la propiedad Observaciones. La segunda sección configura la propiedad con longitud máxima de 500 caracteres. La columna se crea como nvarchar(500) en lugar de nvarchar(max).

El proyecto AceriaData

En el proyecto AceriaData, se identifica y se resuelve el over-fetching en este punto. Se añaden métodos al repositorio que cargan entidades completas y métodos que proyectan solo las columnas necesarias. Se comparan el volumen de datos y el tiempo de ejecución. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

El over-fetching carga más datos de los necesarios.

El over-fetching puede ser de columnas o de filas.

Las proyecciones resuelven el over-fetching de columnas.

Los filtros y la paginación resuelven el over-fetching de filas.

Incluir innecesario provoca over-fetching de columnas y filas.

Las columnas de tipo nvarchar(max) agravan el over-fetching.

El over-fetching consume ancho de banda, memoria y CPU.

El over-fetching es especialmente problemático en aplicaciones web.

En el proyecto AceriaData se identifica y se resuelve el over-fetching.

Anclaje en AceriaData

El concepto se implementa en M04/PROYECTO/4.6. En la práctica se restaura, compila y ejecuta sobre SQL Server LocalDB.

Ejemplo ejecutable del concepto

El siguiente archivo es la implementación real de Infrastructure correspondiente al punto 4.6.

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    private IQueryable<OrdenFabricacion> PendientesM4() => _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id);

    public List<OrdenFabricacion> ObtenerPendientesEntidadCompletaM4() =>
        PendientesM4().ToList();

    public List<OrdenResumenDto> ObtenerPendientesProyectadasM4() =>
        PendientesM4()
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();

    public string ObtenerSqlPendientesEntidadCompletaM4() =>
        PendientesM4().ToQueryString();

    public string ObtenerSqlPendientesProyectadasM4() =>
        PendientesM4()
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToQueryString();
}
```

CSHARP

Explicación línea a línea — Rendimiento46.cs

Línea 1	using AceriaData.Application.Dtos; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.Domain.Entities; → Importa tipos o extensiones requeridos por esta implementación.

Línea 3	<code>using Microsoft.EntityFrameworkCore;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	<code>namespace AceriaData.Infrastructure.Repositories;</code> → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 7	<code>public sealed partial class OrdenRepositorio</code> → Declara la clase concreta que implementa la funcionalidad del punto.
Línea 8	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 9	<code>private IQueryable<OrdenFabricacion> PendientesM4() => _context.OrdenesFabricacion</code> → Define la expresión lambda que EF Core o el caso de uso empleará en esta operación.
Línea 10	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 11	<code>.Where(o => o.Estado == "Pendiente")</code> → Añade el predicado de filtrado a la forma de consulta.
Línea 12	<code>.OrderBy(o => o.FechaCreacion)</code> → Forma parte del orden determinista.
Línea 13	<code>.ThenBy(o => o.Id);</code> → Forma parte del orden determinista.
Línea 15	<code>public List<OrdenFabricacion> ObtenerPendientesEntidadCompletaM4() =></code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 16	<code>PendientesM4().ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 18	<code>public List<OrdenResumenDto> ObtenerPendientesProyectadasM4() =></code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 19	<code>PendientesM4()</code> → Continúa la implementación del punto con esta expresión: <code>PendientesM4()</code>
Línea 20	<code>.Select(o => new OrdenResumenDto</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 21	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 22	<code>NumeroOrden = o.NumeroOrden,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 23	<code>Cliente = o.Cliente,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 24	<code>Estado = o.Estado,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 25	<code>FechaCreacion = o.FechaCreacion</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 26	<code>})</code> → Continúa la implementación del punto con esta expresión: <code>})</code>
Línea 27	<code>.ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 29	<code>public string ObtenerSqlPendientesEntidadCompletaM4() =></code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 30	<code>PendientesM4().ToQueryString();</code> → Obtiene la representación SQL sin materializar la consulta.
Línea 32	<code>public string ObtenerSqlPendientesProyectadasM4() =></code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 33	<code>PendientesM4()</code> → Continúa la implementación del punto con esta expresión: <code>PendientesM4()</code>

Línea 34	<code>.Select(o => new OrdenResumenDto →</code> Proyecta la forma de resultado y controla datos materializados.
Línea 35	<code>{ →</code> Delimita el bloque sintáctico asociado.
Línea 36	<code>NumeroOrden = o.NumeroOrden, →</code> Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 37	<code>Cliente = o.Cliente, →</code> Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 38	<code>Estado = o.Estado, →</code> Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 39	<code>FechaCreacion = o.FechaCreacion →</code> Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 40	<code>}) →</code> Continúa la implementación del punto con esta expresión: <code>})</code>
Línea 41	<code>.ToString(); →</code> Obtiene la representación SQL sin materializar la consulta.
Línea 42	<code>} →</code> Delimita el bloque sintáctico asociado.

Qué debe observarse en ejecución

La lectura del código debe completarse con la evidencia de ejecución: SQL traducido cuando corresponda, número de comandos, cardinalidad, columnas materializadas, estado del `ChangeTracker` y las comprobaciones específicas del punto. El hecho de que el código compile no demuestra por sí solo una mejora de rendimiento; por eso cada punto termina en una comprobación observable.

Reto conceptual. Identificar en el SQL qué columnas desaparecen al proyectar y relacionarlo con transferencia y materialización.

Analogía operativa. Over-fetching es mover un palé entero cuando la siguiente estación solo necesita cuatro piezas.

Criterios de salida

- Relacionar LINQ con SQL o comandos ejecutados.
- Distinguir coste de servidor, transferencia, materialización y tracking.
- Justificar técnicas aplicadas y descartadas.
- Ejecutar el proyecto del punto y obtener 4.6 OK.

Punto 4.7 — Consultas ineficientes: traducción y frontera cliente/servidor

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en las consultas ineficientes del proyecto `AceriaData`, identificando los filtros no traducibles a SQL, las funciones en `Where` que impiden el uso de índices y las técnicas para reescribir las consultas de forma más eficiente.

Objetivos de aprendizaje

- Comprender qué es una consulta ineficiente y por qué es un problema.
- Identificar filtros que EF Core no puede traducir a SQL.
- Comprender el impacto de las funciones aplicadas sobre columnas en `Where`.
- Conocer las funciones que EF Core traduce a SQL y las que no.

- Reescribir consultas para que se traduzcan completamente a SQL.
- Detectar consultas no traducibles y distinguirlas de la evaluación en memoria elegida explícitamente.
- Analizar el SQL generado por consultas eficientes e ineficientes.
- Aplicar estas técnicas al proyecto AceriaData.

Consideraciones técnicas en EF Core 8

En EF Core 8 un predicado no traducible dentro de Where no se evalúa silenciosamente en cliente: falla. La evaluación cliente exige una frontera explícita como AsEnumerable. Las funciones sobre columnas pueden perjudicar sargabilidad y deben medirse.

Referencia oficial: <https://learn.microsoft.com/ef/core/querying/client-eval>

Desarrollo teórico

Qué es una consulta ineficiente

Una consulta ineficiente es aquella que obtiene el resultado correcto pero consume más recursos de los necesarios. Puede ser ineficiente por varios motivos: porque carga más datos de los necesarios, porque ejecuta más consultas de las necesarias o porque aplica filtros en memoria en lugar de en el servidor. En este punto se estudian las consultas que no se traducen completamente a SQL: en EF Core 8 un predicado no traducible dentro de Where falla, salvo que el desarrollador establezca explícitamente una frontera hacia evaluación cliente.

Ejemplo 1 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Where(o => MiMetodoPersonalizado(o.Cliente))
    .ToList();
```

CSHARP

Nota técnica. En EF Core 8, si este predicado no puede traducirse y está dentro de Where, la consulta lanza InvalidOperationException. La evaluación en cliente solo aparece tras una frontera explícita como AsEnumerable().

La primera línea inicia la consulta. La segunda línea filtra por un método personalizado. La tercera línea materializa la consulta. EF Core no puede traducir MiMetodoPersonalizado a SQL. En EF Core 8 este Where no traducible provoca InvalidOperationException. Para filtrar en memoria debe establecerse una frontera explícita, por ejemplo con AsEnumerable().

La traducción de consultas a SQL

EF Core traduce las expresiones LINQ a SQL mediante un árbol de expresión. El proveedor analiza el árbol y genera la sentencia SQL correspondiente. Sin embargo, no todas las expresiones tienen equivalente en SQL. Cuando EF Core 8 encuentra una expresión no traducible fuera de la proyección superior permitida, lanza una excepción. Para continuar en cliente hay que establecer una frontera explícita.

Ejemplo 2 (CSHARP).

```
// Se traduce a SQL: comparación de columnas
var ordenes1 = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .ToList();

// No se traduce a SQL: método personalizado
var ordenes2 = context.OrdenesFabricacion
    .Where(o => EsPendiente(o.Estado))
    .ToList();
```

CSHARP

Nota técnica. En EF Core 8, si este predicado no puede traducirse y está dentro de Where, la consulta lanza InvalidOperationException. La evaluación en cliente solo aparece tras una frontera explícita como AsEnumerable().

La primera consulta se traduce a SQL porque la comparación `o.Estado == "Pendiente"` tiene equivalente en SQL. La segunda consulta no se traduce porque `EsPendiente` es un método personalizado que EF Core no conoce. La segunda consulta provoca una excepción de traducción en EF Core 8 mientras el método personalizado permanezca dentro de `Where`.

Filtros no traducibles

Un filtro no traducible es una condición que EF Core no puede convertir a SQL. Los casos más habituales son: llamadas a métodos personalizados, uso de métodos de .NET que no tienen equivalente en SQL, uso de expresiones regulares y uso de `ToLower/ToUpper` en combinación con otras funciones.

Ejemplo 3 (CSHARP).

```
// Método personalizado
var ordenes1 = context.OrdenesFabricacion
    .Where(o => EsPendiente(o.Estado))
    .ToList();

// Método de .NET sin equivalente en SQL
var ordenes2 = context.OrdenesFabricacion
    .Where(o => o.NumeroOrden.IsNormalized())
    .ToList();

// Expresión regular
var ordenes3 = context.OrdenesFabricacion
    .Where(o => Regex.IsMatch(o.NumeroOrden, @"^0F-\d{4}-\d{4}$"))
    .ToList();
```

CSHARP

Nota técnica. En EF Core 8, si este predicado no puede traducirse y está dentro de `Where`, la consulta lanza `InvalidOperationException`. La evaluación en cliente solo aparece tras una frontera explícita como `AsEnumerable()`.

La primera consulta usa un método personalizado. La segunda usa un método de .NET sin equivalente en SQL. La tercera usa una expresión regular. Ninguna de las tres se traduce completamente a SQL.

Error común: cuando EF Core no puede traducir una expresión, lanza una excepción indicando que la expresión no se pudo traducir. Esto ocurre cuando la expresión está en una posición que EF Core no puede evaluar en el cliente. Si una expresión no traducible está dentro de `Where`, EF Core 8 falla; solo después de una frontera cliente explícita el filtro pasa a LINQ to Objects.

Funciones en Where

Las funciones aplicadas sobre columnas en `Where` pueden impedir el uso de índices. Cuando se aplica una función sobre una columna, SQL Server no puede usar el índice de esa columna porque el valor indexado no coincide con el valor de la función. Esto provoca un table scan en lugar de un index seek.

Ejemplo 4 (CSHARP).

```
// Comparación directa: conserva mejor la sargabilidad; verificar el plan
var ordenes1 = context.OrdenesFabricacion
    .Where(o => o.Cliente == "Constructora del Norte")
    .ToList();

// Función sobre columna: puede dificultar un index seek; verificar el plan
var ordenes2 = context.OrdenesFabricacion
    .Where(o => o.Cliente.ToLower() == "constructora del norte")
    .ToList();
```

CSHARP

Nota técnica. Aplicar una función a la columna puede reducir la sargabilidad, pero el uso real de índices depende del esquema, la collation, los índices y el plan de SQL Server; debe verificarse con el plan de ejecución.

La primera consulta compara la columna directamente. SQL Server puede usar el índice sobre `Cliente`. La segunda consulta aplica `LOWER` sobre la columna; esto puede reducir la sargabilidad. El uso efectivo del índice debe verificarse en el plan de SQL Server.

El impacto de las funciones en Where

El impacto de las funciones en Where depende del tamaño de la tabla y de la selectividad del filtro. En tablas pequeñas, el impacto es pequeño. En tablas grandes, el impacto puede ser significativo porque SQL Server recorre todas las filas en lugar de usar el índice.

Ejemplo 5 (CSHARP).

```
var cronometro1 = Stopwatch.StartNew();
var ordenes1 = context.OrdenesFabricacion
    .Where(o => o.Cliente == "Constructora del Norte")
    .ToList();
cronometro1.Stop();

var cronometro2 = Stopwatch.StartNew();
var ordenes2 = context.OrdenesFabricacion
    .Where(o => o.Cliente.ToLower() == "constructora del norte")
    .ToList();
cronometro2.Stop();
```

CSHARP

Nota técnica. Aplicar una función a la columna puede reducir la sargabilidad, pero el uso real de índices depende del esquema, la collation, los índices y el plan de SQL Server; debe verificarse con el plan de ejecución.

La primera sección mide el tiempo de la consulta sin función. La segunda sección mide el tiempo de la consulta con función. La segunda puede tener un plan menos eficiente; el resultado temporal debe medirse y no se presupone.

Funciones que EF Core traduce a SQL

EF Core traduce varias funciones de .NET a funciones SQL. Las más habituales son StartsWith, EndsWith, Contains, ToLower, ToUpper, Trim, Length, Substring, Replace y las funciones de fecha como Year, Month, Day, Hour, Minute, Second.

Ejemplo 6 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Cliente.StartsWith("Constructora"))
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por clientes que empiezan por "Constructora". La tercera línea materializa la consulta. StartsWith se traduce a LIKE 'Constructora%' en SQL.

El SQL generado tiene la siguiente forma:

Ejemplo 7 (SQL).

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Cliente] LIKE N'Constructora%' AND [o].[IsDeleted] = CAST(0 AS bit)
```

SQL

La primera línea selecciona las columnas. La segunda línea indica la tabla. La tercera línea filtra por el patrón LIKE. La posibilidad de usar un índice depende de la collation, el patrón, el esquema y el plan de ejecución.

Funciones que EF Core no traduce a SQL

EF Core no traduce varias funciones de .NET a SQL. Las más habituales son los métodos personalizados, las expresiones regulares, los métodos de normalización de cadenas y los métodos de conversión que no tienen equivalente directo.

Ejemplo 8 (CSHARP).

```
// No se traduce: método personalizado
var ordenes1 = context.OrdenesFabricacion
    .Where(o => EsPendiente(o.Estado))
    .ToList();

// No se traduce: expresión regular
var ordenes2 = context.OrdenesFabricacion
```

CSHARP

```
.Where(o => Regex.IsMatch(o.NumeroOrden, @"^0F-\d{4}-\d{4}$"))
.ToList();
```

Nota técnica. En EF Core 8, si este predicado no puede traducirse y está dentro de Where, la consulta lanza `InvalidOperationException`. La evaluación en cliente solo aparece tras una frontera explícita como `AsEnumerable()`.

La primera consulta usa un método personalizado. La segunda usa una expresión regular. Ninguna de las dos se traduce completamente a SQL.

Reescribir consultas no traducibles

Las consultas no traducibles se pueden reescribir para que se traduzcan completamente a SQL. La estrategia consiste en sustituir las expresiones no traducibles por expresiones equivalentes que EF Core sí traduce.

Ejemplo 9 (CSHARP).

```
// No traducible: método personalizado
var ordenes1 = context.OrdenesFabricacion
    .Where(o => EsPendiente(o.Estado))
    .ToList();

// Traducible: comparación directa
var ordenes2 = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .ToList();
```

CSHARP

Nota técnica. En EF Core 8, si este predicado no puede traducirse y está dentro de Where, la consulta lanza `InvalidOperationException`. La evaluación en cliente solo aparece tras una frontera explícita como `AsEnumerable()`.

La primera consulta usa un método personalizado. La segunda consulta usa una comparación directa. La segunda se traduce completamente a SQL.

Evitar funciones en Where

Las funciones aplicadas sobre columnas en Where se deben evitar siempre que sea posible. Si se necesita comparar sin distinguir mayúsculas y minúsculas, se puede usar la collation de la columna en lugar de `ToLower`. Si se necesita comparar por una parte de la cadena, se puede usar `StartsWith`, `EndsWith` o `Contains`, que se traducen a LIKE.

Ejemplo 10 (CSHARP).

```
// Función sobre columna: puede dificultar un index seek; verificar el plan
var ordenes1 = context.OrdenesFabricacion
    .Where(o => o.Cliente.ToLower() == "constructora del norte")
    .ToList();

// Comparación directa: conserva mejor la sargabilidad; verificar el plan
var ordenes2 = context.OrdenesFabricacion
    .Where(o => o.Cliente == "Constructora del Norte")
    .ToList();
```

CSHARP

Nota técnica. Aplicar una función a la columna puede reducir la sargabilidad, pero el uso real de índices depende del esquema, la collation, los índices y el plan de SQL Server; debe verificarse con el plan de ejecución.

La primera consulta aplica `ToLower` sobre la columna. La segunda consulta compara directamente. La segunda usa el índice.

El uso de la collation

La collation de una columna determina cómo se comparan las cadenas. Si la collation es insensible a mayúsculas y minúsculas, no es necesario usar `ToLower`. La collation por defecto de SQL Server es `SQL_Latin1_General_CP1_CI_AS`, que es insensible a mayúsculas y minúsculas.

Ejemplo 11 (CSHARP).

```
modelBuilder.Entity<OrdenFabricacion>()
    .Property(o => o.Cliente)
    .UseCollation("SQL_Latin1_General_CP1_CI_AS");
```

CSHARP

La primera línea selecciona la entidad. La segunda línea selecciona la propiedad. La tercera línea establece la collation. A partir de este momento, las comparaciones sobre Cliente son insensibles a mayúsculas y minúsculas sin necesidad de ToLower.

El uso de filtros en memoria

Si una consulta no se puede traducir completamente a SQL, EF Core materializa la consulta antes de tiempo y aplica el filtro en memoria. Esto provoca que se carguen todas las filas y que el filtro se aplique en el cliente.

Ejemplo 12 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .AsEnumerable()
    .Where(o => EsPendiente(o.Estado))
    .ToList();
```

CSHARP

Nota técnica. En EF Core 8, si este predicado no puede traducirse y está dentro de Where, la consulta lanza InvalidOperationException. La evaluación en cliente solo aparece tras una frontera explícita como AsEnumerable().

La primera línea inicia la consulta. La segunda línea convierte la consulta a IEnumerable. La tercera línea filtra en memoria. La cuarta línea materializa la lista. Todas las órdenes se cargan en memoria antes de filtrar.

Detectar consultas no traducibles

Las consultas no traducibles se detectan con el logging de EF Core. Cuando una consulta no se traduce completamente, EF Core emite un warning indicando que parte de la consulta se evaluará en el cliente. También se puede detectar con ToQueryString, que falla si la consulta no se puede traducir.

Ejemplo 13 (CSHARP).

```
optionsBuilder.LogTo(
    Console.WriteLine,
    new[] { "Microsoft.EntityFrameworkCore.Query" },
    LogLevel.Warning);
```

CSHARP

La primera línea habilita el logging. La segunda línea especifica el destino. La tercera línea especifica las categorías. La cuarta línea especifica el nivel mínimo. A partir de este momento, se registran los warnings de traducción de consultas.

El proyecto AceriaData

En el proyecto AceriaData, se identifican y se resuelven las consultas ineficientes en este punto. Se añaden métodos al repositorio que usan filtros no traducibles y funciones en Where. Se reescriben las consultas para que se traduzcan completamente a SQL. Se comparan el SQL generado y el tiempo de ejecución. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

Una consulta ineficiente obtiene el resultado correcto pero consume más recursos.

Un predicado no traducible dentro de una parte que debe ejecutarse en servidor provoca una excepción en EF Core 8; la evaluación cliente debe elegirse explícitamente.

Las funciones en Where impiden el uso de índices.

EF Core traduce StartsWith, EndsWith, Contains, ToLower y funciones de fecha.

EF Core no traduce métodos personalizados ni expresiones regulares.

Las consultas no traducibles se reescriben con expresiones equivalentes.

La collation permite comparaciones insensibles sin ToLower.

El logging de EF Core detecta consultas no traducibles.

En el proyecto AceriaData se identifican y se resuelven las consultas ineficientes.

Anclaje en AceriaData

El concepto se implementa en M04/PROYECTO/4.7. En la práctica se restaura, compila y ejecuta sobre SQL Server LocalDB.

Ejemplo ejecutable del concepto

El siguiente archivo es la implementación real de Infrastructure correspondiente al punto 4.7.

```
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    private static bool EstadoCoincideM4(string actual, string buscado) =>
        string.Equals(actual, buscado, StringComparison.OrdinalIgnoreCase);

    public bool FiltroPersonalizadoNoTraducibleFallaM4(string estado)
    {
        try
        {
            _ = _context.OrdenesFabricacion
                .AsNoTracking()
                .Where(o => EstadoCoincideM4(o.Estado, estado))
                .ToList();
            return false;
        }
        catch (InvalidOperationException)
        {
            return true;
        }
    }

    public int ContarConEvaluacionClienteExplicitaM4(string estado) =>
        _context.OrdenesFabricacion
            .AsNoTracking()
            .AsEnumerable()
            .Count(o => EstadoCoincideM4(o.Estado, estado));

    public string ObtenerSqlClienteConFuncionM4(string cliente)
    {
        var normalizado = cliente.ToLowerInvariant();
        return _context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o => o.Cliente.ToLower() == normalizado)
            .ToQueryString();
    }

    public string ObtenerSqlClienteDirectoM4(string cliente) =>
        _context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o => o.Cliente == cliente)
            .ToQueryString();
}
```

CSHARP

Explicación línea a línea — Rendimiento47.cs

Línea 1	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	namespace AceriaData.Infrastructure.Repositories; → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 5	public sealed partial class OrdenRepositorio → Declara la clase concreta que implementa la funcionalidad del punto.
Línea 6	{ → Delimita el bloque sintáctico asociado.

Línea 7	<code>private static bool EstadoCoincideM4(string actual, string buscado) =></code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 8	<code>string.Equals(actual, buscado, StringComparison.OrdinalIgnoreCase);</code> → Continúa la implementación del punto con esta expresión: <code>string.Equals(actual, buscado, StringComparison.OrdinalIgnoreCase);</code>
Línea 10	<code>public bool FiltroPersonalizadoNoTraducibleFallaM4(string estado)</code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 11	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 12	<code>try</code> → Abre el bloque protegido cuya excepción forma parte de la evidencia del escenario.
Línea 13	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 14	<code>_ = _context.OrdenesFabricacion</code> → Fuerza la ejecución y descarta el valor porque en este bloque interesa medir el coste de la operación.
Línea 15	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 16	<code>.Where(o => EstadoCoincideM4(o.Estado, estado))</code> → Añade el predicado de filtrado a la forma de consulta.
Línea 17	<code>.ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 18	<code>return false;</code> → Devuelve el resultado calculado al llamador.
Línea 19	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 20	<code>catch (InvalidOperationException)</code> → Captura explícitamente la excepción esperada para distinguir el fallo de traducción.
Línea 21	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 22	<code>return true;</code> → Devuelve el resultado calculado al llamador.
Línea 23	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 24	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 26	<code>public int ContarConEvaluacionClienteExplicitaM4(string estado) =></code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 27	<code>_context.OrdenesFabricacion</code> → Continúa la implementación del punto con esta expresión: <code>_context.OrdenesFabricacion</code>
Línea 28	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 29	<code>.AsEnumerable()</code> → Establece explícitamente la frontera hacia LINQ to Objects.
Línea 30	<code>.Count(o => EstadoCoincideM4(o.Estado, estado));</code> → Continúa la composición fluida invocando Count sobre el resultado de la línea anterior.
Línea 32	<code>public string ObtenerSqlClienteConFuncionM4(string cliente)</code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 33	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 34	<code>var normalizado = cliente.ToLowerInvariant();</code> → Calcula y conserva el resultado que será validado o mostrado.

Línea 35	<code>return _context.OrdenesFabricacion</code> → Devuelve el resultado calculado al llamador.
Línea 36	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 37	<code>.Where(o => o.Cliente.ToLower() == normalizado)</code> → Añade el predicado de filtrado a la forma de consulta.
Línea 38	<code>.ToString();</code> → Obtiene la representación SQL sin materializar la consulta.
Línea 39	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 41	<code>public string ObtenerSqlClienteDirectoM4(string cliente) =></code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 42	<code>_context.OrdenesFabricacion</code> → Continúa la implementación del punto con esta expresión: <code>_context.OrdenesFabricacion</code>
Línea 43	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 44	<code>.Where(o => o.Cliente == cliente)</code> → Añade el predicado de filtrado a la forma de consulta.
Línea 45	<code>.ToString();</code> → Obtiene la representación SQL sin materializar la consulta.
Línea 46	<code>}</code> → Delimita el bloque sintáctico asociado.

Qué debe observarse en ejecución

La lectura del código debe completarse con la evidencia de ejecución: SQL traducido cuando corresponda, número de comandos, cardinalidad, columnas materializadas, estado del `ChangeTracker` y las comprobaciones específicas del punto. El hecho de que el código compile no demuestra por sí solo una mejora de rendimiento; por eso cada punto termina en una comprobación observable.

Reto conceptual. Comparar el SQL con `LOWER(columna)` frente a comparación directa y explicar qué debe medirse en SQL Server.

Analogía operativa. Una frontera cliente explícita es sacar las piezas de la máquina y continuar manualmente: se puede hacer, pero debe ser consciente.

Criterios de salida

- Relacionar LINQ con SQL o comandos ejecutados.
- Distinguir coste de servidor, transferencia, materialización y tracking.
- Justificar técnicas aplicadas y descartadas.
- Ejecutar el proyecto del punto y obtener 4.7 OK.

Punto 4.8 — Split Queries: cuándo y cómo usarlas

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en las Split Queries del proyecto `AceriaData`, comprendiendo el problema del producto cartesiano, cuándo usar `AsSplitQuery`, cómo se comporta frente a `AsSingleQuery` y qué implicaciones tiene en el rendimiento y la coherencia de los datos.

Objetivos de aprendizaje

- Comprender qué es una Split Query y qué problema resuelve.
- Comprender el producto cartesiano que generan varias colecciones incluidas.
- Diferenciar entre `AsSplitQuery` y `AsSingleQuery`.

- Aplicar AsSplitQuery cuando se incluyen varias colecciones.
- Comprender el impacto de AsSplitQuery en el número de consultas.
- Comprender la coherencia de los datos en las Split Queries.
- Analizar el SQL generado por AsSplitQuery.
- Configurar el comportamiento por defecto de las Split Queries.
- Aplicar estas técnicas al proyecto AceriaData.

Consideraciones técnicas en EF Core 8

SplitQuery ejecuta varios comandos y puede evitar explosión cartesiana. No implica una transacción independiente por subconsulta. Sin aislamiento adecuado puede no existir una instantánea consistente frente a cambios concurrentes.

Referencia oficial: <https://learn.microsoft.com/ef/core/querying/single-split-queries>

Desarrollo teórico

Qué es una Split Query

Una Split Query es una consulta que EF Core divide en varias consultas separadas para cargar las entidades principales y las entidades relacionadas. En lugar de generar una sola consulta con varios LEFT JOIN, EF Core genera una consulta para la entidad principal y una consulta por cada colección incluida. Esto evita el producto cartesiano que se produce cuando se incluyen varias colecciones en una sola consulta.

Ejemplo 1 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSplitQuery()
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la colección de entidades intermedias. La cuarta línea aplica AsSplitQuery. La quinta línea materializa la consulta. EF Core genera tres consultas: una para las órdenes, una para las planchas y una para las entidades intermedias.

El problema del producto cartesiano

Cuando se incluyen varias colecciones en una sola consulta, EF Core genera un LEFT JOIN por cada colección. El resultado es un producto cartesiano: cada fila de la primera colección se combina con cada fila de la segunda colección. Si una orden tiene dos planchas y dos aleaciones, el resultado incluye cuatro filas para esa orden. Si tiene tres planchas y tres aleaciones, el resultado incluye nueve filas. El número de filas crece de forma multiplicativa.

Ejemplo 2 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la colección de entidades intermedias. La cuarta línea materializa la consulta. EF Core genera una sola consulta con dos LEFT JOIN. El resultado incluye un producto cartesiano de planchas y entidades intermedias.

El SQL generado tiene la siguiente forma:

Ejemplo 3 (SQL).

```
SELECT [o].[Id], [o].[NumeroOrden], ..., [p].[Id], [p].[Espesor], ..., [oa].[OrdenFabricacionId], [oa].[AleacionId], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
LEFT JOIN [OrdenesAleaciones] AS [oa] ON [o].[Id] = [oa].[OrdenFabricacionId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id], [oa].[OrdenFabricacionId], [oa].[AleacionId]
```

SQL

La primera línea selecciona las columnas de las tres tablas. La segunda línea indica la tabla principal. La tercera línea combina con la tabla de planchas. La cuarta línea combina con la tabla de entidades intermedias. La quinta línea aplica el filtro global. La sexta línea ordena por las claves.

Si una orden tiene dos planchas y dos aleaciones, el resultado incluye cuatro filas para esa orden. EF Core agrupa las filas en memoria y construye las colecciones. El resultado final es correcto, pero el número de filas transferidas es mayor que el número de planchas más el número de aleaciones.

El problema del producto cartesiano en tablas grandes

El producto cartesiano se agrava cuando las colecciones tienen muchas filas. Si una orden tiene cien planchas y cien aleaciones, el resultado incluye diez mil filas para esa orden. Aunque EF Core agrupa las filas en memoria, el volumen de datos transferidos y el tiempo de procesamiento son significativos.

Ejemplo 4 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSplitQuery()
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la colección de entidades intermedias. La cuarta línea aplica AsSplitQuery. La quinta línea materializa la consulta. EF Core genera tres consultas separadas: una para las órdenes, una para las planchas y una para las entidades intermedias. El número de filas transferidas es la suma de las filas de cada tabla, no el producto.

La solución con AsSplitQuery

AsSplitQuery divide la consulta en varias consultas separadas. Cada consulta carga una colección. EF Core combina los resultados en memoria usando las claves primarias y foráneas. El resultado final es el mismo que con una sola consulta, pero el número de filas transferidas es menor.

Ejemplo 5 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSplitQuery()
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la colección de entidades intermedias. La cuarta línea aplica AsSplitQuery. La quinta línea materializa la consulta. EF Core genera tres consultas separadas.

El SQL generado tiene la siguiente forma:

Ejemplo 6 (SQL).

```
-- Consulta 1: órdenes
SELECT [o].[Id], [o].[NumeroOrden], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit);

-- Consulta 2: planchas
SELECT [p].[Id], [p].[OrdenId], [p].[Espesor], ..., [o].[Id]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit);

-- Consulta 3: entidades intermedias
SELECT [oa].[OrdenFabricacionId], [oa].[AleacionId], ..., [o].[Id]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [OrdenesAleaciones] AS [oa] ON [o].[Id] = [oa].[OrdenFabricacionId]
WHERE [o].[IsDeleted] = CAST(0 AS bit);
```

SQL

La primera consulta carga las órdenes. La segunda consulta carga las planchas. La tercera consulta carga las entidades intermedias. EF Core combina los resultados en memoria usando la clave primaria de la orden.

El método AsSingleQuery

AsSingleQuery fuerza que una consulta se ejecute en una sola consulta, incluso si se han incluido varias colecciones. Es el comportamiento por defecto. Se usa cuando se quiere forzar una sola consulta y se acepta el producto cartesiano.

Ejemplo 7 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSingleQuery()
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la colección de entidades intermedias. La cuarta línea fuerza una sola consulta. La quinta línea materializa la consulta. EF Core genera una sola consulta con dos LEFT JOIN. El resultado incluye el producto cartesiano.

Comparación entre AsSplitQuery y AsSingleQuery

La diferencia entre AsSplitQuery y AsSingleQuery está en el número de consultas y en el volumen de datos transferidos. AsSingleQuery genera una sola consulta con un producto cartesiano. AsSplitQuery genera varias consultas sin producto cartesiano.

Ejemplo 8 (CSHARP).

```
// AsSingleQuery: 1 consulta con producto cartesiano
var ordenes1 = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSingleQuery()
    .ToList();

// AsSplitQuery: 3 consultas sin producto cartesiano
var ordenes2 = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSplitQuery()
    .ToList();
```

CSHARP

La primera consulta genera una sola consulta con producto cartesiano. La segunda genera tres consultas sin producto cartesiano.

Cuándo usar AsSplitQuery

AsSplitQuery se usa cuando se incluyen varias colecciones y se quiere evitar el producto cartesiano. Es especialmente útil cuando las colecciones tienen muchas filas o cuando las filas son anchas. También es útil cuando se quiere reducir el volumen de datos transferidos.

Ejemplo 9 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSplitQuery()
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la colección de entidades intermedias. La cuarta línea aplica AsSplitQuery. La quinta línea materializa la consulta.

Cuándo no usar AsSplitQuery

Con una sola colección no existe explosión cartesiana entre colecciones; el beneficio típico de SplitQuery suele ser menor, pero la decisión depende de volumen y roundtrips. Si se necesita coherencia entre los comandos, debe elegirse explícitamente una estrategia transaccional y un nivel de aislamiento adecuados. Tampoco se debe usar cuando el número de consultas adicionales es mayor que el coste del producto cartesiano.

Ejemplo 10 (CSHARP).

```
// Una sola colección: el beneficio típico es menor; medir volumen y roundtrips
var ordenes1 = context.OrdenesFabricacion
```

CSHARP

```
.Include(o => o.Planchas)
.AsSplitQuery()
.ToList();

// Usar AsSplitQuery con varias colecciones
var ordenes2 = context.OrdenesFabricacion
.Include(o => o.Planchas)
.Include(o => o.OrdenesAleaciones)
.AsSplitQuery()
.ToList();
```

La primera consulta usa AsSplitQuery con una sola colección. No hay explosión cartesiana entre colecciones; el posible beneficio o coste debe medirse. La segunda usa AsSplitQuery con dos colecciones. Evita el producto cartesiano.

La coherencia de los datos en AsSplitQuery

Una Split Query ejecuta varios comandos. Sin una transacción con aislamiento adecuado no existe garantía de que todos observen la misma instantánea frente a cambios concurrentes. Esto significa que los datos pueden cambiar entre una consulta y la siguiente. Si otra transacción modifica los datos entre la primera y la segunda consulta, el resultado puede ser inconsistente. Si se necesita una instantánea consistente, debe elegirse una transacción y un nivel de aislamiento que proporcionen esa garantía.

Ejemplo 11 (CSHARP).

```
using var transaction = context.Database.BeginTransaction(System.Data.IsolationLevel.Serializable);

var ordenes = context.OrdenesFabricacion
.Include(o => o.Planchas)
.Include(o => o.OrdenesAleaciones)
.AsSplitQuery()
.ToList();

transaction.Commit();
```

CSHARP

Nota técnica. Una transacción explícita con aislamiento Serializable se usa aquí para ilustrar consistencia entre varios comandos; el aislamiento tiene coste y debe elegirse según el escenario.

La primera línea inicia una transacción. La segunda línea inicia la consulta. La tercera línea incluye la colección de planchas. La cuarta línea incluye la colección de entidades intermedias. La quinta línea aplica AsSplitQuery. La sexta línea materializa la consulta. La séptima línea confirma la transacción. Las tres consultas se ejecutan dentro de la misma transacción.

La configuración global de Split Queries

El comportamiento por defecto de las consultas con varios Include se puede configurar globalmente en el DbContext con UseSqlServer y la opción UseQuerySplittingBehavior.

Ejemplo 12 (CSHARP).

```
optionsBuilder.UseSqlServer(
    connectionString,
    sqlOptions => sqlOptions.UseQuerySplittingBehavior(QuerySplittingBehavior.SplitQuery));
```

CSHARP

La primera línea configura el proveedor de SQL Server. La segunda línea establece el comportamiento por defecto como SplitQuery. A partir de este momento, todas las consultas con varios Include se dividen en varias consultas por defecto.

Ejemplo 13 (CSHARP).

```
optionsBuilder.UseSqlServer(
    connectionString,
    sqlOptions => sqlOptions.UseQuerySplittingBehavior(QuerySplittingBehavior.SingleQuery));
```

CSHARP

La primera línea configura el proveedor de SQL Server. La segunda línea establece el comportamiento por defecto como SingleQuery. Es el comportamiento estándar.

La advertencia de producto cartesiano

EF Core emite una advertencia cuando detecta que una consulta con varias colecciones incluidas puede generar un producto cartesiano. La advertencia se puede convertir en error con ConfigureWarnings.

Ejemplo 14 (CSHARP).

```
optionsBuilder.ConfigureWarnings(warnings =>
    warnings.Throw(RelationalEventId.MultipleCollectionIncludeWarning));
```

CSHARP

La primera línea configura las advertencias. La segunda línea convierte la advertencia `MultipleCollectionIncludeWarning` en excepción. A partir de este momento, cualquier consulta que incluya varias colecciones sin `AsSplitQuery` lanza una excepción.

El proyecto AceriaData

En el proyecto `AceriaData`, se profundiza en las Split Queries en este punto. Se añaden métodos al repositorio que usan `AsSplitQuery` y `AsSingleQuery`. Se comparan el número de consultas, el volumen de datos y el tiempo de ejecución. La base de datos sigue siendo `AceriaDB` en `SQL Server LocalDB`.

Resumen de la teoría

Una Split Query divide una consulta en varias consultas separadas.

El producto cartesiano aparece cuando se incluyen varias colecciones.

El producto cartesiano multiplica el número de filas transferidas.

`AsSplitQuery` evita el producto cartesiano.

`AsSingleQuery` fuerza una sola consulta con producto cartesiano.

`AsSplitQuery` puede ser útil cuando varias colecciones producen duplicación o explosión cartesiana; su coste en roundtrips también debe medirse.

Con una sola colección no existe explosión cartesiana entre colecciones, por lo que el beneficio típico de `SplitQuery` es menor; la decisión sigue dependiendo de volumen, duplicación y roundtrips.

Una Split Query ejecuta varios comandos. Sin una transacción con aislamiento adecuado no existe garantía de que todos observen la misma instantánea frente a cambios concurrentes.

La coherencia se garantiza con una transacción explícita.

El comportamiento por defecto se configura con `UseQuerySplittingBehavior`.

La advertencia de producto cartesiano se puede convertir en error.

En el proyecto `AceriaData` se comparan `AsSplitQuery` y `AsSingleQuery`.

Anclaje en AceriaData

El concepto se implementa en `M04/PROYECTO/4.8`. En la práctica se restaura, compila y ejecuta sobre `SQL Server LocalDB`.

Ejemplo ejecutable del concepto

El siguiente archivo es la implementación real de Infrastructure correspondiente al punto 4.8.

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    private IQueryable<OrdenFabricacion> ConsultaDosColeccionesM4() =>
        _context.OrdenesFabricacion
            .AsNoTrackingWithIdentityResolution()
            .Include(o => o.Planchas)
            .Include(o => o.OrdenesAleaciones)
            .ThenInclude(oa => oa.Aleacion)
            .OrderBy(o => o.Id);

    public SplitQueryMetricaDto MedirSingleQueryM4()
    {
        SqlCommandCounterInterceptor.Instance.Reset();
        var ordenes = ConsultaDosColeccionesM4().AsSingleQuery().ToList();
    }
}
```

CSHARP

```

        return new SplitQueryMetricaDto
        {
            Ordenes = ordenes.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            Planchas = ordenes.Sum(o => o.Planchas.Count),
            RelacionesAleacion = ordenes.Sum(o => o.OrdenesAleaciones.Count)
        };
    }

    public SplitQueryMetricaDto MedirSplitQueryM4()
    {
        SqlCommandCounterInterceptor.Instance.Reset();
        var ordenes = ConsultaDosColeccionesM4().AsSplitQuery().ToList();

        return new SplitQueryMetricaDto
        {
            Ordenes = ordenes.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            Planchas = ordenes.Sum(o => o.Planchas.Count),
            RelacionesAleacion = ordenes.Sum(o => o.OrdenesAleaciones.Count)
        };
    }

    public string ObtenerSqlSingleQueryM4() =>
        ConsultaDosColeccionesM4().AsSingleQuery().ToQueryString();

    public string ObtenerSqlSplitQueryM4() =>
        ConsultaDosColeccionesM4().AsSplitQuery().ToQueryString();
}

```

Explicación línea a línea — Rendimiento48.cs

Línea 1	using AceriaData.Application.Dtos; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.Domain.Entities; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	namespace AceriaData.Infrastructure.Repositories; → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 7	public sealed partial class OrdenRepositorio → Declara la clase concreta que implementa la funcionalidad del punto.
Línea 8	{ → Delimita el bloque sintáctico asociado.
Línea 9	private IQueryable<OrdenFabricacion> ConsultaDosColeccionesM4() => → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 10	_context.OrdenesFabricacion → Continúa la implementación del punto con esta expresión: _context.OrdenesFabricacion
Línea 11	.AsNoTrackingWithIdentityResolution() → Activa NoTracking con resolución temporal de identidad.
Línea 12	.Include(o => o.Planchas) → Define la navegación relacionada que debe cargarse.
Línea 13	.Include(o => o.OrdenesAleaciones) → Define la navegación relacionada que debe cargarse.
Línea 14	.ThenInclude(oa => oa.Aleacion) → Define la navegación relacionada que debe cargarse.
Línea 15	.OrderBy(o => o.Id); → Forma parte del orden determinista.
Línea 17	public SplitQueryMetricaDto MedirSingleQueryM4() → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 18	{ → Delimita el bloque sintáctico asociado.

Línea 19	<code>SqlCommandCounterInterceptor.Instance.Reset();</code> → Mide comandos SQL reales ejecutados.
Línea 20	<code>var ordenes = ConsultaDosColeccionesM4().AsSingleQuery().ToList();</code> → Fuerza un único comando para la comparación.
Línea 22	<code>return new SplitQueryMetricaDto</code> → Devuelve el resultado calculado al llamador.
Línea 23	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 24	<code>Ordenes = ordenes.Count,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 25	<code>ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),</code> → Mide comandos SQL reales ejecutados.
Línea 26	<code>Planchas = ordenes.Sum(o => o.Planchas.Count),</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 27	<code>RelacionesAleacion = ordenes.Sum(o => o.OrdenesAleaciones.Count)</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 28	<code>};</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 29	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 31	<code>public SplitQueryMetricaDto MedirSplitQueryM4()</code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 32	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 33	<code>SqlCommandCounterInterceptor.Instance.Reset();</code> → Mide comandos SQL reales ejecutados.
Línea 34	<code>var ordenes = ConsultaDosColeccionesM4().AsSplitQuery().ToList();</code> → Divide la carga relacionada en varios comandos SQL.
Línea 36	<code>return new SplitQueryMetricaDto</code> → Devuelve el resultado calculado al llamador.
Línea 37	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 38	<code>Ordenes = ordenes.Count,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 39	<code>ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),</code> → Mide comandos SQL reales ejecutados.
Línea 40	<code>Planchas = ordenes.Sum(o => o.Planchas.Count),</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 41	<code>RelacionesAleacion = ordenes.Sum(o => o.OrdenesAleaciones.Count)</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 42	<code>};</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 43	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 45	<code>public string ObtenerSqlSingleQueryM4() =></code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 46	<code>ConsultaDosColeccionesM4().AsSingleQuery().ToQueryString();</code> → Fuerza un único comando para la comparación.

Línea 48	<code>public string ObtenerSqlSplitQueryM4() =></code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 49	<code>ConsultaDosColeccionesM4().AsSplitQuery().ToQueryString();</code> → Divide la carga relacionada en varios comandos SQL.
Línea 50	<code>}</code> → Delimita el bloque sintáctico asociado.

Qué debe observarse en ejecución

La lectura del código debe completarse con la evidencia de ejecución: SQL traducido cuando corresponda, número de comandos, cardinalidad, columnas materializadas, estado del `ChangeTracker` y las comprobaciones específicas del punto. El hecho de que el código compile no demuestra por sí solo una mejora de rendimiento; por eso cada punto termina en una comprobación observable.

Reto conceptual. Explicar por qué dos colecciones multiplican filas en `SingleQuery` y por qué `SplitQuery` intercambia volumen por roundtrips.

Analogía operativa. `SingleQuery` mezcla lotes en una hoja grande; `SplitQuery` los trae por separado y los ensambla por claves.

Criterios de salida

- Relacionar LINQ con SQL o comandos ejecutados.
- Distinguir coste de servidor, transferencia, materialización y tracking.
- Justificar técnicas aplicadas y descartadas.
- Ejecutar el proyecto del punto y obtener 4.8 OK.

Punto 4.9 — Compiled Queries

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en las Compiled Queries del proyecto `AceriaData`, comprendiendo qué son, cómo se compilan, cuándo aportan beneficios y cómo se integran en el repositorio.

Objetivos de aprendizaje

- Comprender qué es una Compiled Query y qué problema resuelve.
- Comprender el coste de compilar una consulta LINQ a SQL.
- Utilizar `EF.CompileQuery` y `EF.CompileAsyncQuery`.
- Diferenciar entre consultas compiladas síncronas y asíncronas.
- Comprender el ciclo de vida de una consulta compilada.
- Medir el impacto de las Compiled Queries en el rendimiento.
- Comprender cuándo usar Compiled Queries y cuándo no.
- Aplicar estas técnicas al proyecto `AceriaData`.

Consideraciones técnicas en EF Core 8

EF Core ya cachea consultas por forma. `EF.CompileQuery` evita parte del trabajo de búsqueda y preparación de EF; no almacena el plan de ejecución de SQL Server. Debe medirse en hot paths y no se exige ganar una microprueba aislada.

Referencia oficial: <https://learn.microsoft.com/ef/core/performance/advanced-performance-topics>

Desarrollo teórico

Qué es una Compiled Query

Una Compiled Query es un delegado LINQ compilado explícitamente que puede invocarse muchas veces, evitando la búsqueda por forma en la caché interna de consultas de EF. EF Core procesa la forma de la consulta y almacena en caché la salida de compilación. Una compiled query crea un delegado explícito que evita la búsqueda por forma en la caché interna; no crea ni almacena el plan de ejecución de SQL Server.

Ejemplo 1 (CSHARP).

```
var consultaCompilada = EF.CompileQuery((AceriaDbContext context, string estado) =>
    context.OrdenesFabricacion
        .Where(o => o.Estado == estado)
        .OrderBy(o => o.FechaCreacion)
        .Select(o => o)
    );

var pendientes = consultaCompilada(context, "Pendiente");
var enProceso = consultaCompilada(context, "EnProceso");
```

CSHARP

Nota técnica. La consulta compilada omite la búsqueda en la caché de forma de consulta de EF. No almacena el plan de ejecución de SQL Server y debe medirse en el hot path real.

La primera línea compila la consulta y la almacena en la variable `consultaCompilada`. La segunda línea declara la consulta con un parámetro `estado`. Las siguientes líneas filtran y ordenan. La penúltima línea cierra la consulta. La última línea ejecuta la consulta compilada con el parámetro "Pendiente" y "EnProceso".

El coste de compilar una consulta

EF Core mantiene una caché por forma de consulta. En una consulta normal todavía debe comparar el árbol de expresión con las formas cacheadas; una compiled query permite omitir ese trabajo de búsqueda. En aplicaciones que ejecutan la misma consulta muchas veces, el coste de compilación se acumula.

Ejemplo 2 (CSHARP).

```
var cronometro = Stopwatch.StartNew();
for (int i = 0; i < 1000; i++)
{
    var ordenes = context.OrdenesFabricacion
        .Where(o => o.Estado == "Pendiente")
        .ToList();
}
cronometro.Stop();
```

CSHARP

El bucle ejecuta la misma forma muchas veces. EF Core reutiliza su caché interna; la medición sirve para observar el coste total, no para afirmar que la consulta se recompila por completo en cada iteración.

La caché de consultas de EF Core

EF Core almacena en caché la salida de compilación asociada a la forma de la consulta. El plan de ejecución pertenece a SQL Server y se gestiona independientemente. Sin embargo, la caché tiene un límite y las consultas con parámetros dinámicos pueden no coincidir exactamente con las consultas cacheadas.

Ejemplo 3 (CSHARP).

```
var ordenes1 = context.OrdenesFabricacion.Where(o => o.Estado == "Pendiente").ToList();
var ordenes2 = context.OrdenesFabricacion.Where(o => o.Estado == "Pendiente").ToList();
```

CSHARP

La primera línea ejecuta la consulta. La segunda línea ejecuta la misma consulta. EF Core detecta que la consulta es la misma y reutiliza el SQL de la caché. La segunda puede beneficiarse de la caché interna de EF y de las cachés del servidor, pero no se presupone una ventaja temporal fija sin medir.

Cuándo las Compiled Queries aportan beneficios

Las Compiled Queries aportan beneficios cuando la misma consulta se ejecuta muchas veces con parámetros distintos. En estos casos, el coste de compilación se paga una sola vez y se reutiliza en todas las ejecuciones. También aportan beneficios cuando la consulta es compleja y su compilación es costosa.

Ejemplo 4 (CSHARP).

```
var consultaCompilada = EF.CompileQuery((AceriaDbContext context, string cliente, string estado) =>
    context.OrdenesFabricacion
        .Where(o => o.Cliente == cliente && o.Estado == estado)
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado
        })
    );

var resultado1 = consultaCompilada(context, "Constructora del Norte", "Pendiente");
var resultado2 = consultaCompilada(context, "Constructora del Sur", "EnProceso");
```

CSHARP

Nota técnica. La consulta compilada omite la búsqueda en la caché de forma de consulta de EF. No almacena el plan de ejecución de SQL Server y debe medirse en el hot path real.

La primera línea compila la consulta con dos parámetros. La segunda línea declara la consulta. Las siguientes líneas filtran, ordenan y proyectan. La penúltima línea cierra la consulta. La última línea ejecuta la consulta compilada con dos combinaciones de parámetros.

El método EF.CompileQuery

EF.CompileQuery es el método que compila una consulta LINQ. Acepta una expresión lambda que devuelve un `IEnumerable<T>` o `IQueryable<T>`. Devuelve un delegado que se puede invocar con el contexto y los parámetros.

Ejemplo 5 (CSHARP).

```
var consultaCompilada = EF.CompileQuery((AceriaDbContext context, string estado) =>
    context.OrdenesFabricacion
        .Where(o => o.Estado == estado)
    );
```

CSHARP

Nota técnica. La consulta compilada omite la búsqueda en la caché de forma de consulta de EF. No almacena el plan de ejecución de SQL Server y debe medirse en el hot path real.

La primera línea compila la consulta. La segunda línea declara la consulta con el contexto y el parámetro estado. Las siguientes líneas filtran y materializan. La penúltima línea cierra la consulta. La última línea cierra la compilación.

El método EF.CompileAsyncQuery

EF.CompileAsyncQuery es la versión asíncrona de EF.CompileQuery. Devuelve un delegado que devuelve un `Task<T>` o `IAsyncEnumerable<T>`. Se usa en aplicaciones que necesitan liberar el hilo mientras se espera la respuesta de la base de datos.

Ejemplo 6 (CSHARP).

```
var consultaCompiladaAsync = EF.CompileAsyncQuery(
    (AceriaDbContext context, string estado) =>
        context.OrdenesFabricacion
            .Where(o => o.Estado == estado));

await foreach (var orden in consultaCompiladaAsync(context, "Pendiente"))
{
    Console.WriteLine(orden.NumeroOrden);
}
```

CSHARP

Nota técnica. La consulta compilada omite la búsqueda en la caché de forma de consulta de EF. No almacena el plan de ejecución de SQL Server y debe medirse en el hot path real.

La primera línea compila la consulta asíncrona. La segunda línea declara la consulta con el contexto y el parámetro estado. Las siguientes líneas filtran y materializan de forma asíncrona. La penúltima línea cierra la consulta. La última línea ejecuta la consulta compilada asíncrona.

El ciclo de vida de una Compiled Query

Una Compiled Query suele conservarse en un campo estático o equivalente y se invoca con parámetros distintos. El delegado pertenece a EF; el SQL concreto y el plan de ejecución son responsabilidades separadas de la ejecución y de SQL Server.

Ejemplo 7 (CSHARP).

```
public static class OrdenConsultasCompiladas
{
    public static readonly Func<AceriaDbContext, string, IEnumerable<OrdenFabricacion>> ObtenerPorEstado =
        EF.CompileQuery((AceriaDbContext context, string estado) =>
            context.OrdenesFabricacion
                .Where(o => o.Estado == estado)
                .OrderBy(o => o.FechaCreacion)
                .Select(o => o)
        );
}
```

CSHARP

Nota técnica. La consulta compilada omite la búsqueda en la caché de forma de consulta de EF. No almacena el plan de ejecución de SQL Server y debe medirse en el hot path real.

La primera línea declara la clase estática. La segunda línea declara el campo estático con la consulta compilada. La tercera línea compila la consulta. Las siguientes líneas declaran la consulta. La penúltima línea cierra la consulta. La última línea cierra la compilación. La consulta compilada se almacena en el campo estático y se reutiliza durante toda la vida de la aplicación.

Las Compiled Queries y los parámetros

Las Compiled Queries aceptan parámetros que se pasan en la invocación. Los parámetros se usan en la consulta y se traducen a parámetros SQL. Los parámetros pueden ser de cualquier tipo: cadenas, números, fechas, etc.

Ejemplo 8 (CSHARP).

```
var consultaCompilada = EF.CompileQuery((AceriaDbContext context, string cliente, DateTime desde) =>
    context.OrdenesFabricacion
        .Where(o => o.Cliente == cliente && o.FechaCreacion >= desde)
    );

var resultado = consultaCompilada(context, "Constructora del Norte", new DateTime(2024, 1, 1));
```

CSHARP

Nota técnica. La consulta compilada omite la búsqueda en la caché de forma de consulta de EF. No almacena el plan de ejecución de SQL Server y debe medirse en el hot path real.

La primera línea compila la consulta con dos parámetros. La segunda línea declara la consulta. Las siguientes líneas filtran por cliente y fecha. La penúltima línea cierra la consulta. La última línea ejecuta la consulta compilada con los dos parámetros.

Las Compiled Queries y las proyecciones

Las Compiled Queries se pueden combinar con proyecciones para reducir el volumen de datos transferidos. La proyección se aplica dentro de la consulta compilada y se traduce a SQL.

Ejemplo 9 (CSHARP).

```
var consultaCompilada = EF.CompileQuery((AceriaDbContext context, string estado) =>
    context.OrdenesFabricacion
        .Where(o => o.Estado == estado)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
```

CSHARP

```
Estado = o.Estado
}))
);
```

Nota técnica. La consulta compilada omite la búsqueda en la caché de forma de consulta de EF. No almacena el plan de ejecución de SQL Server y debe medirse en el hot path real.

La primera línea compila la consulta. La segunda línea declara la consulta con el contexto y el parámetro estado. Las siguientes líneas filtran y proyectan. La penúltima línea cierra la consulta. La última línea cierra la compilación.

Las Compiled Queries y el tracking

Las Compiled Queries respetan el comportamiento de tracking del contexto. Si se aplica AsNoTracking dentro de la consulta, las entidades no se registran en el Change Tracker. Si no se aplica, se registran.

Ejemplo 10 (CSHARP).

```
var consultaCompilada = EF.CompileQuery((AceriaDbContext context, string estado) =>
    context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == estado)
    );
```

CSHARP

Nota técnica. La consulta compilada omite la búsqueda en la caché de forma de consulta de EF. No almacena el plan de ejecución de SQL Server y debe medirse en el hot path real.

La primera línea compila la consulta. La segunda línea declara la consulta con el contexto y el parámetro estado. Las siguientes líneas aplican AsNoTracking y filtran. La penúltima línea cierra la consulta. La última línea cierra la compilación.

Cuándo no usar Compiled Queries

Las Compiled Queries no aportan beneficios cuando la consulta se ejecuta pocas veces. El coste de compilación se paga una sola vez, pero si la consulta solo se ejecuta una vez, el beneficio es nulo. Tampoco aportan beneficios cuando la consulta es muy simple y su compilación es rápida. En estos casos, la caché de consultas de EF Core es suficiente.

Ejemplo 11 (CSHARP).

```
// Consulta simple ejecutada una vez: normalmente no es candidata prioritaria; medir
var ordenes = context.OrdenesFabricacion.ToList();

// Aporta beneficio: consulta compleja ejecutada muchas veces
var consultaCompilada = EF.CompileQuery((AceriaDbContext context, string estado) =>
    context.OrdenesFabricacion
        .Where(o => o.Estado == estado)
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new { o.NumeroOrden, o.Cliente })
    );
```

CSHARP

Nota técnica. La consulta compilada omite la búsqueda en la caché de forma de consulta de EF. No almacena el plan de ejecución de SQL Server y debe medirse en el hot path real.

La primera sección ejecuta una consulta simple una vez. La segunda sección compila una consulta compleja que se ejecutará muchas veces.

Las Compiled Queries en aplicaciones de alta concurrencia

En aplicaciones de alta concurrencia, las Compiled Queries aportan beneficios porque reducen el coste de compilación por petición. Cada petición reutiliza la consulta compilada en lugar de compilarla de nuevo. Esto reduce el uso de CPU y mejora el tiempo de respuesta.

Ejemplo 12 (CSHARP).

CSHARP

```

public class OrdenServicio
{
    private static readonly Func<AceriaDbContext, string, IEnumerable<OrdenFabricacion>> _consultaCompilada =
        EF.CompileQuery((AceriaDbContext context, string estado) =>
            context.OrdenesFabricacion
                .Where(o => o.Estado == estado)
                );

    public IEnumerable<OrdenFabricacion> ObtenerPorEstado(AceriaDbContext context, string estado)
    {
        return _consultaCompilada(context, estado);
    }
}

```

Nota técnica. La consulta compilada omite la búsqueda en la caché de forma de consulta de EF. No almacena el plan de ejecución de SQL Server y debe medirse en el hot path real.

La primera línea declara la clase. La segunda línea declara el campo estático con la consulta compilada. La tercera línea compila la consulta. Las siguientes líneas declaran la consulta. La penúltima línea cierra la consulta. La última línea cierra la compilación. El método ObtenerPorEstado invoca la consulta compilada.

El proyecto AceriaData

En el proyecto AceriaData, se añaden Compiled Queries al repositorio en este punto. Se compilan las consultas más frecuentes y se comparan el tiempo de ejecución con las consultas no compiladas. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

Una Compiled Query se compila una sola vez y se reutiliza muchas veces.

El coste de compilación se paga cada vez que se ejecuta una consulta no compilada.

EF.CompileQuery compila consultas síncronas.

EF.CompileAsyncQuery compila consultas asíncronas.

Las Compiled Queries se almacenan en campos estáticos.

Aceptan parámetros que se traducen a parámetros SQL.

Se combinan con proyecciones y con AsNoTracking.

Aportan beneficios cuando la consulta se ejecuta muchas veces.

No aportan beneficios cuando la consulta se ejecuta pocas veces.

En aplicaciones de alta concurrencia reducen el uso de CPU.

En el proyecto AceriaData se añaden Compiled Queries al repositorio.

Anclaje en AceriaData

El concepto se implementa en M04/PROYECTO/4.9. En la práctica se restaura, compila y ejecuta sobre SQL Server LocalDB.

Ejemplo ejecutable del concepto

El siguiente archivo es la implementación real de Infrastructure correspondiente al punto 4.9.

CSHARP

```

using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    private static readonly Func<AceriaDbContext, string, IEnumerable<OrdenFabricacion>>
        ConsultaCompiladaPorEstadoM4 =
        EF.CompileQuery(

```

```

        (AceriaDbContext context, string estado) =>
            context.OrdenesFabricacion
                .AsNoTracking()
                .Where(o => o.Estado == estado)
                .OrderBy(o => o.FechaCreacion)
                .ThenBy(o => o.Id)
                .Select(o => o));

    public List<OrdenFabricacion> ObtenerPorEstadoNormalM4(string estado) =>
        _context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o => o.Estado == estado)
            .OrderBy(o => o.FechaCreacion)
            .ThenBy(o => o.Id)
            .ToList();

    public List<OrdenFabricacion> ObtenerPorEstadoCompiladoM4(string estado) =>
        ConsultaCompiladaPorEstadoM4(_context, estado).ToList();
}

```

Explicación línea a línea — Rendimiento49.cs

Línea 1	using AceriaData.Domain.Entities; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.Infrastructure.Persistence; → Importa tipos o extensiones requeridos por esta implementación.
Línea 3	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	namespace AceriaData.Infrastructure.Repositories; → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 7	public sealed partial class OrdenRepositorio → Declara la clase concreta que implementa la funcionalidad del punto.
Línea 8	{ → Delimita el bloque sintáctico asociado.
Línea 9	private static readonly Func<AceriaDbContext, string, IEnumerable<OrdenFabricacion>> → Declara un campo de solo lectura que conserva una dependencia o delegado reutilizable.
Línea 10	ConsultaCompiladaPorEstadoM4 = → Continúa la implementación del punto con esta expresión: ConsultaCompiladaPorEstadoM4 =
Línea 11	EF.CompileQuery(→ Prepara un delegado de compiled query de EF.
Línea 12	(AceriaDbContext context, string estado) => → Define la expresión lambda que EF Core o el caso de uso empleará en esta operación.
Línea 13	context.OrdenesFabricacion → Continúa la implementación del punto con esta expresión: context.OrdenesFabricacion
Línea 14	.AsNoTracking() → Desactiva tracking para esta consulta de lectura.
Línea 15	.Where(o => o.Estado == estado) → Añade el predicado de filtrado a la forma de consulta.
Línea 16	.OrderBy(o => o.FechaCreacion) → Forma parte del orden determinista.
Línea 17	.ThenBy(o => o.Id) → Forma parte del orden determinista.
Línea 18	.Select(o => o); → Proyecta la forma de resultado y controla datos materializados.
Línea 20	public List<OrdenFabricacion> ObtenerPorEstadoNormalM4(string estado) => → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.

Línea 21	<code>_context.OrdenesFabricacion</code> → Continúa la implementación del punto con esta expresión: <code>_context.OrdenesFabricacion</code>
Línea 22	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 23	<code>.Where(o => o.Estado == estado)</code> → Añade el predicado de filtrado a la forma de consulta.
Línea 24	<code>.OrderBy(o => o.FechaCreacion)</code> → Forma parte del orden determinista.
Línea 25	<code>.ThenBy(o => o.Id)</code> → Forma parte del orden determinista.
Línea 26	<code>.ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 28	<code>public List<OrdenFabricacion> ObtenerPorEstadoCompiladoM4(string estado) =></code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 29	<code>ConsultaCompiladaPorEstadoM4(_context, estado).ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 30	<code>}</code> → Delimita el bloque sintáctico asociado.

Qué debe observarse en ejecución

La lectura del código debe completarse con la evidencia de ejecución: SQL traducido cuando corresponda, número de comandos, cardinalidad, columnas materializadas, estado del ChangeTracker y las comprobaciones específicas del punto. El hecho de que el código compile no demuestra por sí solo una mejora de rendimiento; por eso cada punto termina en una comprobación observable.

Reto conceptual. Justificar cuándo el coste evitado por CompileQuery puede importar frente a red y base de datos.

Analogía operativa. CompiledQuery guarda una ruta de preparación en EF; no reserva una vía dentro de SQL Server.

Criterios de salida

- Relacionar LINQ con SQL o comandos ejecutados.
- Distinguir coste de servidor, transferencia, materialización y tracking.
- Justificar técnicas aplicadas y descartadas.
- Ejecutar el proyecto del punto y obtener 4.9 OK.

Punto 4.10 — Paginación eficiente: Skip/Take y keyset pagination

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en la paginación eficiente del proyecto **AceriaData**, comparando **Skip/Take** con **keyset pagination**, analizando el SQL generado por cada técnica y aplicando la paginación adecuada según el escenario.

Objetivos de aprendizaje

- Comprender qué es la paginación y por qué es importante.
- Aplicar Skip/Take para paginar resultados.
- Comprender el coste de Skip en tablas grandes.
- Aplicar keyset pagination para paginar de forma eficiente.
- Diferenciar entre offset pagination y keyset pagination.
- Analizar el SQL generado por cada técnica de paginación.

- Medir el impacto de la paginación en el rendimiento.
- Aplicar estas técnicas al proyecto AceriaData.

Consideraciones técnicas en EF Core 8

Toda paginación necesita orden totalmente determinista. AceriaData ordena por FechaCreacion e Id; keyset usa ambos valores como cursor. Offset es válido para saltos arbitrarios pero puede encarecerse con offsets altos.

Referencia oficial: <https://learn.microsoft.com/ef/core/querying/pagination>

Desarrollo teórico

Qué es la paginación

La paginación es la técnica que consiste en dividir un conjunto grande de resultados en páginas de tamaño fijo. En lugar de devolver todos los resultados de una consulta, se devuelve solo una página. La paginación es esencial en aplicaciones web y en cualquier escenario donde el número de resultados pueda ser elevado.

Ejemplo 1 (CSHARP).

```
var pagina = 1;
var tamanoPagina = 10;

var ordenes = context.OrdenesFabricacion
    .OrderBy(o => o.FechaCreacion)
    .Skip((pagina - 1) * tamanoPagina)
    .Take(tamanoPagina)
    .ToList();
```

CSHARP

Nota técnica. Este ejemplo usa una ordenación simplificada por fecha. Para paginación determinista, AceriaData añade ThenBy(o => o.Id), como se demuestra en 4.10.

La primera línea declara la página. La segunda línea declara el tamaño de la página. La tercera línea inicia la consulta. La cuarta línea ordena por fecha. La quinta línea salta las páginas anteriores. La sexta línea toma el tamaño de la página. La séptima línea materializa la consulta. Solo se devuelven diez órdenes.

Offset pagination con Skip/Take

La offset pagination es la técnica más común de paginación. Consiste en saltar un número fijo de filas y tomar las siguientes. Se implementa con Skip y Take. Es sencilla de implementar y funciona bien en tablas pequeñas o medianas.

Ejemplo 2 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .OrderBy(o => o.FechaCreacion)
    .Skip(20)
    .Take(10)
    .ToList();
```

CSHARP

Nota técnica. Este ejemplo usa una ordenación simplificada por fecha. Para paginación determinista, AceriaData añade ThenBy(o => o.Id), como se demuestra en 4.10.

La primera línea inicia la consulta. La segunda línea ordena por fecha. La tercera línea salta las primeras veinte filas. La cuarta línea toma las siguientes diez filas. La quinta línea materializa la consulta. El SQL generado tiene la siguiente forma:

Ejemplo 3 (SQL).

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [o].[Observaciones], [o].[IsDeleted], [o].[DeletedAt]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
```

SQL

```
ORDER BY [o].[FechaCreacion]
OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY
```

Nota técnica. Este SQL es ilustrativo. Para una consulta keyset con solo Take, el proveedor SQL Server puede generar TOP en lugar de OFFSET 0/FETCH. La forma que debe comprobarse es la salida real de ToQueryString().

La primera línea selecciona las columnas. La segunda línea indica la tabla. La tercera línea aplica el filtro global. La cuarta línea ordena por fecha. La quinta línea salta las primeras veinte filas y toma las siguientes diez.

El coste de Skip

El coste de Skip crece con el número de filas que se saltan. SQL Server debe leer y descartar todas las filas anteriores a la página solicitada. Si se salta un millón de filas, SQL Server lee un millón de filas y las descarta. Esto degrada el rendimiento de forma significativa en tablas grandes.

Ejemplo 4 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .OrderBy(o => o.FechaCreacion)
    .Skip(1000000)
    .Take(10)
    .ToList();
```

CSHARP

Nota técnica. Este ejemplo usa una ordenación simplificada por fecha. Para paginación determinista, AceriaData añade ThenBy(o => o.Id), como se demuestra en 4.10.

La primera línea inicia la consulta. La segunda línea ordena por fecha. La tercera línea salta un millón de filas. La cuarta línea toma las siguientes diez filas. La quinta línea materializa la consulta. SQL Server lee un millón de filas y las descarta antes de devolver las diez solicitadas.

Keyset pagination

La keyset pagination es una técnica de paginación que evita el coste de Skip. En lugar de saltar un número fijo de filas, se filtra por la clave de la última fila de la página anterior. Se implementa con un Where que compara la clave con el valor de la última fila.

Ejemplo 5 (CSHARP).

```
var ultimaFecha = new DateTime(2024, 5, 12);
var ultimoId = 5;

var ordenes = context.OrdenesFabricacion
    .Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id > ultimoId))
    .OrderBy(o => o.FechaCreacion)
    .ThenBy(o => o.Id)
    .Take(10)
    .ToList();
```

CSHARP

La primera línea declara la fecha de la última fila de la página anterior. La segunda línea declara el Id de la última fila. La tercera línea inicia la consulta. La cuarta línea filtra por la clave compuesta. La quinta línea ordena por fecha. La sexta línea ordena por Id. La séptima línea toma las siguientes diez filas. La octava línea materializa la consulta.

El SQL generado tiene la siguiente forma:

Ejemplo 6 (SQL).

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [o].[Observaciones], [o].[IsDeleted], [o].[DeletedAt]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
      AND ([o].[FechaCreacion] > @_ultimaFecha_0 OR ([o].[FechaCreacion] = @_ultimaFecha_0 AND [o].[Id] > @_ultimoId_1))
ORDER BY [o].[FechaCreacion], [o].[Id]
OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY
```

SQL

Nota técnica. Este SQL es ilustrativo. Para una consulta keyset con solo Take, el proveedor SQL Server puede generar TOP en lugar de OFFSET 0/FETCH. La forma que debe comprobarse es la salida real de ToQueryString().

La primera línea selecciona las columnas. La segunda línea indica la tabla. La tercera línea aplica el filtro global. La cuarta línea filtra por la clave compuesta. La quinta línea ordena por fecha y Id. La sexta línea toma las siguientes diez filas. El OFFSET es cero porque no se salta ninguna fila. Para que el seek compuesto sea eficiente conviene un índice cuyo orden empiece por FechaCreacion e Id. Este módulo no añade un índice nuevo, por lo que el plan real debe verificarse con el esquema existente.

Diferencia entre offset pagination y keyset pagination

La offset pagination salta un número fijo de filas. La keyset pagination filtra por la clave de la última fila. La offset pagination es sencilla pero costosa en tablas grandes. La keyset pagination es más compleja pero eficiente en tablas grandes.

Ejemplo 7 (CSHARP).

```
// Offset pagination
var ordenes1 = context.OrdenesFabricacion
    .OrderBy(o => o.FechaCreacion)
    .Skip(10000000)
    .Take(10)
    .ToList();

// Keyset pagination
var ordenes2 = context.OrdenesFabricacion
    .Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id > ultimoId))
    .OrderBy(o => o.FechaCreacion)
    .ThenBy(o => o.Id)
    .Take(10)
    .ToList();
```

La primera usa offset y la segunda keyset. Con un índice adecuado y navegación secuencial, keyset evita el coste creciente de saltar filas; el plan real sigue dependiendo de índices y selectividad.

La clave de ordenación en keyset pagination

La keyset pagination requiere una clave de ordenación única. Si la clave no es única, se debe combinar con otra columna para garantizar el orden. Lo más habitual es usar la clave primaria como segunda columna de ordenación.

Ejemplo 8 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .OrderBy(o => o.FechaCreacion)
    .ThenBy(o => o.Id)
    .ToList();
```

La primera línea inicia la consulta. La segunda línea ordena por fecha. La tercera línea ordena por Id. La cuarta línea materializa la consulta. La combinación de fecha e Id garantiza un orden único.

La dirección de la paginación

La keyset pagination puede paginar hacia delante y hacia atrás. Para paginar hacia delante, se filtra por la clave mayor que la última fila. Para paginar hacia atrás, se filtra por la clave menor que la primera fila y se invierte el orden.

Ejemplo 9 (CSHARP).

```
// Paginación hacia delante
var siguientes = context.OrdenesFabricacion
    .Where(o => o.FechaCreacion > ultimaFecha || (o.FechaCreacion == ultimaFecha && o.Id > ultimoId))
    .OrderBy(o => o.FechaCreacion)
    .ThenBy(o => o.Id)
    .Take(10)
    .ToList();

// Paginación hacia atrás
var anteriores = context.OrdenesFabricacion
    .Where(o => o.FechaCreacion < primeraFecha || (o.FechaCreacion == primeraFecha && o.Id < primerId))
    .OrderByDescending(o => o.FechaCreacion)
    .ThenByDescending(o => o.Id)
    .Take(10)
    .ToList();
```

La primera sección pagina hacia delante. La segunda sección pagina hacia atrás. La segunda invierte el orden para obtener las filas anteriores.

El total de páginas

La paginación suele incluir el número total de páginas. El total se calcula con una consulta de conteo. Es importante que el conteo se haga con los mismos filtros que la consulta de paginación.

Ejemplo 10 (CSHARP).

```
var total = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .Count();

var totalPaginas = (int)Math.Ceiling((double)total / tamanoPagina);
```

CSHARP

La primera línea cuenta las órdenes pendientes. La segunda línea calcula el total de páginas. La tercera línea redondea hacia arriba. El total de páginas se usa para mostrar la navegación.

La paginación con filtros

La paginación se combina con filtros para devolver solo las filas que cumplen una condición. Los filtros se aplican antes de la paginación.

Ejemplo 11 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.FechaCreacion)
    .Skip((pagina - 1) * tamanoPagina)
    .Take(tamanoPagina)
    .ToList();
```

CSHARP

Nota técnica. Este ejemplo usa una ordenación simplificada por fecha. Para paginación determinista, `AceriaData` añade `ThenBy(o => o.Id)`, como se demuestra en 4.10.

La primera línea inicia la consulta. La segunda línea filtra por estado. La tercera línea ordena por fecha. La cuarta línea salta las páginas anteriores. La quinta línea toma el tamaño de la página. La sexta línea materializa la consulta.

La paginación con proyecciones

La paginación se combina con proyecciones para reducir el volumen de datos transferidos. Las proyecciones se aplican después de la paginación.

Ejemplo 12 (CSHARP).

```
var resmenes = context.OrdenesFabricacion
    .OrderBy(o => o.FechaCreacion)
    .Skip((pagina - 1) * tamanoPagina)
    .Take(tamanoPagina)
    .Select(o => new OrdenResumenDto
    {
        NumeroOrden = o.NumeroOrden,
        Cliente = o.Cliente,
        Estado = o.Estado,
        FechaCreacion = o.FechaCreacion
    })
    .ToList();
```

CSHARP

Nota técnica. Este ejemplo usa una ordenación simplificada por fecha. Para paginación determinista, `AceriaData` añade `ThenBy(o => o.Id)`, como se demuestra en 4.10.

La primera línea inicia la consulta. La segunda línea ordena por fecha. La tercera línea salta las páginas anteriores. La cuarta línea toma el tamaño de la página. La quinta línea proyecta al DTO. Las siguientes líneas asignan las propiedades. La penúltima línea cierra la proyección. La última línea materializa la consulta.

La paginación con `AsNoTracking`

La paginación se combina con `AsNoTracking` para reducir el consumo de memoria. `AsNoTracking` se aplica antes de la paginación.

Ejemplo 13 (CSHARP).

```
var ordenes = context.OrdenesFabricacion
    .AsNoTracking()
    .OrderBy(o => o.FechaCreacion)
    .Skip((pagina - 1) * tamanoPagina)
    .Take(tamanoPagina)
    .ToList();
```

CSHARP

Nota técnica. Este ejemplo usa una ordenación simplificada por fecha. Para paginación determinista, `AceriaData` añade `ThenBy(o => o.Id)`, como se demuestra en 4.10.

La primera línea inicia la consulta. La segunda línea aplica `AsNoTracking`. La tercera línea ordena por fecha. La cuarta línea salta las páginas anteriores. La quinta línea toma el tamaño de la página. La sexta línea materializa la consulta.

El proyecto `AceriaData`

En el proyecto `AceriaData`, se profundiza en la paginación eficiente en este punto. Se añaden métodos al repositorio que usan `Skip/Take` y `keyset pagination`. Se comparan el SQL generado y el tiempo de ejecución. La base de datos sigue siendo `AceriaDB` en `SQL Server LocalDB`.

Resumen de la teoría

La paginación divide un conjunto grande de resultados en páginas.

`Skip/Take` implementa la `offset pagination`.

El coste de `Skip` crece con el número de filas saltadas.

La `keyset pagination` filtra por la clave de la última fila.

La `keyset pagination` suele escalar mejor para navegación siguiente/anterior cuando existe una ordenación única e índices adecuados.

La clave de ordenación debe ser única.

La paginación puede ser hacia delante o hacia atrás.

El total de páginas se calcula con una consulta de conteo.

La paginación se combina con filtros, proyecciones y `AsNoTracking`.

En el proyecto `AceriaData` se comparan `Skip/Take` y `keyset pagination`.

Anclaje en `AceriaData`

El concepto se implementa en `M04/PROYECTO/4.10`. En la práctica se restaura, compila y ejecuta sobre `SQL Server LocalDB`.

Ejemplo ejecutable del concepto

El siguiente archivo es la implementación real de `Infrastructure` correspondiente al punto 4.10.

```
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    public PaginaOrdenesDto ObtenerPaginaOffsetM4(int pagina, int tamano)
    {
        if (pagina < 1) throw new ArgumentOutOfRangeException(nameof(pagina));
        if (tamano < 1) throw new ArgumentOutOfRangeException(nameof(tamano));

        var consulta = _context.OrdenesFabricacion
            .AsNoTracking()
            .OrderBy(o => o.FechaCreacion)
            .ThenBy(o => o.Id)
            .Skip((pagina - 1) * tamano)
            .Take(tamano)
```

CSHARP

```

        .Select(o => new OrdenPaginaDto
        {
            Id = o.Id,
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        });

        return new PaginaOrdenesDto
        {
            Sql = consulta.ToQueryString(),
            Elementos = consulta.ToList()
        };
    }

    public PaginaOrdenesDto ObtenerPaginaKeysetM4(
        DateTime ultimaFecha,
        int ultimoId,
        int tamano)
    {
        if (tamano < 1) throw new ArgumentOutOfRangeException(nameof(tamano));

        var consulta = _context.OrdenesFabricacion
            .AsNoTracking()
            .Where(o =>
                o.FechaCreacion > ultimaFecha ||
                (o.FechaCreacion == ultimaFecha && o.Id > ultimoId))
            .OrderBy(o => o.FechaCreacion)
            .ThenBy(o => o.Id)
            .Take(tamano)
            .Select(o => new OrdenPaginaDto
            {
                Id = o.Id,
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            });

        return new PaginaOrdenesDto
        {
            Sql = consulta.ToQueryString(),
            Elementos = consulta.ToList()
        };
    }
}

```

Explicación línea a línea — Rendimiento410.cs

Línea 1	using AceriaData.Application.Dtos; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	namespace AceriaData.Infrastructure.Repositories; → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 6	public sealed partial class OrdenRepositorio → Declara la clase concreta que implementa la funcionalidad del punto.
Línea 7	{ → Delimita el bloque sintáctico asociado.
Línea 8	public PaginaOrdenesDto ObtenerPaginaOffsetM4(int pagina, int tamano) → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 9	{ → Delimita el bloque sintáctico asociado.
Línea 10	if (pagina < 1) throw new ArgumentOutOfRangeException(nameof(pagina)); → Comprueba una condición necesaria para considerar correcto el resultado.
Línea 11	if (tamano < 1) throw new ArgumentOutOfRangeException(nameof(tamano)); → Comprueba una condición necesaria para considerar correcto el resultado.

Línea 13	<code>var consulta = _context.OrdenesFabricacion</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 14	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 15	<code>.OrderBy(o => o.FechaCreacion)</code> → Forma parte del orden determinista.
Línea 16	<code>.ThenBy(o => o.Id)</code> → Forma parte del orden determinista.
Línea 17	<code>.Skip((pagina - 1) * tamano)</code> → Aplica el desplazamiento de la paginación offset.
Línea 18	<code>.Take(tamano)</code> → Limita el número máximo de elementos.
Línea 19	<code>.Select(o => new OrdenPaginaDto</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 20	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 21	<code>Id = o.Id,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 22	<code>NumeroOrden = o.NumeroOrden,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 23	<code>Cliente = o.Cliente,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 24	<code>Estado = o.Estado,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 25	<code>FechaCreacion = o.FechaCreacion</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 26	<code>});</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 28	<code>return new PaginaOrdenesDto</code> → Devuelve el resultado calculado al llamador.
Línea 29	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 30	<code>Sql = consulta.ToQueryString(),</code> → Obtiene la representación SQL sin materializar la consulta.
Línea 31	<code>Elementos = consulta.ToList()</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 32	<code>};</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 33	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 35	<code>public PaginaOrdenesDto ObtenerPaginaKeysetM4(</code> → Continúa la implementación del punto con esta expresión: <code>public PaginaOrdenesDto ObtenerPaginaKeysetM4(</code>
Línea 36	<code>DateTime ultimaFecha,</code> → Aporta un argumento o componente intermedio a la construcción multilínea en curso.
Línea 37	<code>int ultimoId,</code> → Aporta un argumento o componente intermedio a la construcción multilínea en curso.
Línea 38	<code>int tamano)</code> → Continúa la implementación del punto con esta expresión: <code>int tamano)</code>
Línea 39	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 40	<code>if (tamano < 1) throw new ArgumentOutOfRangeException(nameof(tamano));</code> → Comprueba una condición necesaria para considerar correcto el resultado.

Línea 42	<code>var consulta = _context.OrdenesFabricacion</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 43	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 44	<code>.Where(o =></code> → Añade el predicado de filtrado a la forma de consulta.
Línea 45	<code>o.FechaCreacion > ultimaFecha </code> → Continúa una condición compuesta usada para validar la equivalencia del resultado.
Línea 46	<code>(o.FechaCreacion == ultimaFecha && o.Id > ultimoId))</code> → Continúa la implementación del punto con esta expresión: <code>(o.FechaCreacion == ultimaFecha && o.Id > ultimoId))</code>
Línea 47	<code>.OrderBy(o => o.FechaCreacion)</code> → Forma parte del orden determinista.
Línea 48	<code>.ThenBy(o => o.Id)</code> → Forma parte del orden determinista.
Línea 49	<code>.Take(tamano)</code> → Limita el número máximo de elementos.
Línea 50	<code>.Select(o => new OrdenPaginaDto</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 51	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 52	<code>Id = o.Id,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 53	<code>NumeroOrden = o.NumeroOrden,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 54	<code>Cliente = o.Cliente,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 55	<code>Estado = o.Estado,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 56	<code>FechaCreacion = o.FechaCreacion</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 57	<code>});</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 59	<code>return new PaginaOrdenesDto</code> → Devuelve el resultado calculado al llamador.
Línea 60	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 61	<code>Sql = consulta.ToQueryString(),</code> → Obtiene la representación SQL sin materializar la consulta.
Línea 62	<code>Elementos = consulta.ToList()</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 63	<code>};</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 64	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 65	<code>}</code> → Delimita el bloque sintáctico asociado.

Qué debe observarse en ejecución

La lectura del código debe completarse con la evidencia de ejecución: SQL traducido cuando corresponda, número de comandos, cardinalidad, columnas materializadas, estado del ChangeTracker y las comprobaciones específicas del punto. El hecho de que el código compile no demuestra por sí solo una mejora de rendimiento; por eso cada punto termina en una comprobación observable.

Reto conceptual. Construir la condición seek para un orden compuesto FechaCreacion + Id.

Analogía operativa. Offset cuenta cajas desde el principio; keyset continúa desde la etiqueta exacta de la última caja vista.

Criterios de salida

- Relacionar LINQ con SQL o comandos ejecutados.
- Distinguir coste de servidor, transferencia, materialización y tracking.
- Justificar técnicas aplicadas y descartadas.
- Ejecutar el proyecto del punto y obtener 4.10 OK.

Punto 4.11 — Diagnóstico con logs, métricas y herramientas

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en el diagnóstico del proyecto **AceriaData**, configurando logs detallados, midiendo métricas de rendimiento y usando herramientas de análisis para identificar problemas de rendimiento en EF Core.

Objetivos de aprendizaje

- Comprender por qué es importante el diagnóstico en EF Core.
- Configurar logs detallados con LogTo y ILoggerFactory.
- Filtrar los logs por categoría, nivel y evento.
- Usar DiagnosticSource para capturar eventos de EF Core.
- Medir el tiempo de ejecución de las consultas.
- Contar el número de consultas ejecutadas.
- Detectar consultas lentas con el logging.
- Analizar el plan de ejecución en SQL Server.
- Usar las herramientas de diagnóstico de EF Core.
- Aplicar estas técnicas al proyecto **AceriaData**.

Consideraciones técnicas en EF Core 8

Un tiempo aislado no prueba rendimiento. El diagnóstico reproducible combina SQL, comandos, filas, tracking y tiempo, y usa TagWith para correlación.

Referencia oficial: <https://learn.microsoft.com/ef/core/performance/efficient-querying>

Desarrollo teórico

Por qué es importante el diagnóstico

El diagnóstico es el conjunto de técnicas que permiten observar el comportamiento de la aplicación en tiempo de ejecución. En EF Core, el diagnóstico permite ver las consultas que se ejecutan, el tiempo que tardan, los parámetros que reciben y los errores que producen. Sin diagnóstico, los problemas de rendimiento son difíciles de identificar y de resolver.

Ejemplo 1 (CSHARP).

```
optionsBuilder.LogTo(
    Console.WriteLine,
    new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
    LogLevel.Information);
```

CSHARP

La primera línea habilita el logging. La segunda línea especifica el destino. La tercera línea especifica las categorías. La cuarta línea especifica el nivel mínimo. A partir de este momento, todas las sentencias SQL se registran en la consola.

El método LogTo

LogTo es el método que configura el logging en EF Core. Acepta un delegado que recibe el mensaje como cadena de texto. Se puede usar Console.WriteLine para escribir en la consola, o cualquier otro método que acepte una cadena.

Ejemplo 2 (CSHARP).

```
optionsBuilder.LogTo(Console.WriteLine);
```

CSHARP

La primera línea configura el logging en la consola. Todos los mensajes de EF Core se escriben en la salida estándar.

Ejemplo 3 (CSHARP).

```
var writer = new StreamWriter("efcore.log", append: true);
optionsBuilder.LogTo(writer.WriteLine);
```

CSHARP

La primera línea crea un escritor de archivo. La segunda línea configura el logging para que escriba en el archivo. Los mensajes se acumulan en el archivo efcore.log.

Las categorías de logging

EF Core organiza los mensajes en categorías. Las más habituales son Microsoft.EntityFrameworkCore.Database.Command, que incluye las sentencias SQL, Microsoft.EntityFrameworkCore.Query, que incluye información sobre la traducción de consultas, y Microsoft.EntityFrameworkCore.Update, que incluye información sobre las operaciones de escritura.

Ejemplo 4 (CSHARP).

```
optionsBuilder.LogTo(
    Console.WriteLine,
    new[]
    {
        "Microsoft.EntityFrameworkCore.Database.Command",
        "Microsoft.EntityFrameworkCore.Query",
        "Microsoft.EntityFrameworkCore.Update"
    },
    LogLevel.Information);
```

CSHARP

La primera línea habilita el logging. La segunda línea especifica el destino. La tercera línea inicia el array de categorías. La cuarta línea incluye la categoría de comandos. La quinta línea incluye la categoría de consultas. La sexta línea incluye la categoría de actualizaciones. La séptima línea especifica el nivel mínimo.

Los niveles de logging

EF Core usa los niveles de logging estándar de .NET: Trace, Debug, Information, Warning, Error y Critical. El nivel Information incluye las sentencias SQL. El nivel Debug incluye información adicional sobre la ejecución. El nivel Warning incluye advertencias. El nivel Error incluye errores.

Ejemplo 5 (CSHARP).

```
optionsBuilder.LogTo(
    Console.WriteLine,
    new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
    LogLevel.Information);
```

CSHARP

La primera línea habilita el logging. La segunda línea especifica el destino. La tercera línea especifica las categorías. La cuarta línea especifica el nivel mínimo. Solo se muestran los mensajes de nivel Information o superior.

El logging con ILoggerFactory

ILoggerFactory es la interfaz estándar de .NET para el logging. EF Core se integra con ella y permite usar los proveedores de logging de .NET, como Console, Debug, EventSource y Serilog.

Ejemplo 6 (CSHARP).

```
var loggerFactory = LoggerFactory.Create(builder =>
{
    builder
        .AddFilter("Microsoft.EntityFrameworkCore.Database.Command", LogLevel.Information)
```

CSHARP

```
.AddConsole();
});

optionsBuilder.UseLoggerFactory(loggerFactory);
```

La primera línea crea la fábrica de loggers. La segunda línea inicia la configuración. La tercera línea filtra los mensajes de la categoría de comandos. La cuarta línea añade el proveedor de consola. La quinta línea cierra la configuración. La sexta línea configura EF Core para usar la fábrica.

El método `EnableSensitiveDataLogging`

Por defecto, EF Core oculta los valores de los parámetros en los mensajes de logging para evitar exponer datos sensibles. El método `EnableSensitiveDataLogging` permite mostrar los valores reales de los parámetros.

Ejemplo 7 (CSHARP).

```
optionsBuilder
    .UseSqlServer(connectionString)
    .EnableSensitiveDataLogging();
```

CSHARP

La primera línea configura el proveedor. La segunda línea habilita el logging de datos sensibles. Los parámetros de las consultas se muestran con sus valores reales. Es útil en desarrollo, pero no se recomienda en producción.

El método `EnableDetailedErrors`

Por defecto, EF Core no incluye información detallada en los mensajes de error para evitar exponer la estructura interna. El método `EnableDetailedErrors` permite mostrar información más detallada en los errores.

Ejemplo 8 (CSHARP).

```
optionsBuilder
    .UseSqlServer(connectionString)
    .EnableDetailedErrors();
```

CSHARP

La primera línea configura el proveedor. La segunda línea habilita los errores detallados. Los mensajes de error incluyen información sobre las propiedades y las entidades implicadas.

El método `ConfigureWarnings`

`ConfigureWarnings` permite configurar el comportamiento de EF Core ante determinadas advertencias. Se puede hacer que una advertencia se convierta en error, que se ignore o que se registre con un nivel distinto.

Ejemplo 9 (CSHARP).

```
optionsBuilder.ConfigureWarnings(warnings =>
    warnings.Throw(RelationalEventId.MultipleCollectionIncludeWarning));
```

CSHARP

La primera línea configura las advertencias. La segunda línea convierte la advertencia `MultipleCollectionIncludeWarning` en excepción. A partir de este momento, cualquier consulta que incluya varias colecciones sin `AsSplitQuery` lanza una excepción.

El método `DiagnosticSource`

`DiagnosticSource` es el mecanismo estándar de .NET para publicar eventos de diagnóstico. EF Core publica eventos a través de `DiagnosticSource` que se pueden capturar con un observador. Los eventos incluyen el inicio y el fin de las consultas, el tiempo de ejecución y los parámetros.

Ejemplo 10 (CSHARP).

```
var observer = new DiagnosticObserver();
DiagnosticListener.AllListeners.Subscribe(observer);
```

CSHARP

La primera línea crea el observador. La segunda línea suscribe el observador a todos los listeners. A partir de este momento, el observador recibe los eventos de EF Core.

El observador de diagnóstico

El observador de diagnóstico implementa la interfaz `IObserver<DiagnosticListener>` y la interfaz `IObserver<KeyValuePair<string, object>>`. Recibe los eventos de EF Core y puede registrar la información.

Ejemplo 11 (CSHARP).

```
public class DiagnosticObserver : IObservable<DiagnosticListener>
{
    public void OnNext(DiagnosticListener listener)
    {
        if (listener.Name == "Microsoft.EntityFrameworkCore")
        {
            listener.Subscribe(new EfCoreObserver());
        }
    }

    public void OnError(Exception error) { }
    public void OnCompleted() { }
}

public class EfCoreObserver : IObservable<KeyValuePair<string, object>>
{
    public void OnNext(KeyValuePair<string, object> value)
    {
        if (value.Key == "Microsoft.EntityFrameworkCore.Database.Command.CommandExecuted")
        {
            Console.WriteLine($"Comando ejecutado: {value.Value}");
        }
    }

    public void OnError(Exception error) { }
    public void OnCompleted() { }
}
```

CSHARP

La primera línea declara el observador. La segunda línea implementa el método `OnNext`. La tercera línea comprueba el nombre del listener. La cuarta línea suscribe el observador interno. La quinta línea implementa `OnError`. La sexta línea implementa `OnCompleted`. La séptima línea declara el observador interno. La octava línea implementa `OnNext`. La novena línea comprueba el nombre del evento. La décima línea muestra el comando ejecutado.

La medición del tiempo de ejecución

El tiempo de ejecución de una consulta se puede medir con `Stopwatch`. Se inicia el cronómetro antes de la consulta y se detiene después. La diferencia es el tiempo de ejecución.

Ejemplo 12 (CSHARP).

```
var cronometro = Stopwatch.StartNew();
var ordenes = context.OrdenesFabricacion.ToList();
cronometro.Stop();

Console.WriteLine($"Tiempo: {cronometro.ElapsedMilliseconds} ms");
```

CSHARP

La primera línea inicia el cronómetro. La segunda línea ejecuta la consulta. La tercera línea detiene el cronómetro. La cuarta línea muestra el tiempo.

El conteo de consultas

El número de consultas ejecutadas se puede contar con un observador de diagnóstico o con el logging. El logging registra cada consulta. Se puede contar el número de líneas de log que contienen `Executed DbCommand`.

Ejemplo 13 (CSHARP).

```
var contador = 0;
optionsBuilder.LogTo(message =>
{
    if (message.Contains("Executed DbCommand"))
    {
        contador++;
    }
});
```

CSHARP

La primera línea declara el contador. La segunda línea configura el logging. La tercera línea comprueba si el mensaje contiene `Executed DbCommand`. La cuarta línea incrementa el contador.

La detección de consultas lentas

Las consultas lentas se detectan con el logging y con la medición del tiempo. El logging registra el tiempo de ejecución de cada consulta. Se pueden filtrar las consultas que superan un umbral.

Ejemplo 14 (CSHARP).

```
optionsBuilder.LogTo(message =>
{
    if (message.Contains("Executed DbCommand"))
    {
        Console.WriteLine(message);
    }
});
```

CSHARP

La primera línea configura el logging. La segunda línea comprueba si el mensaje contiene Executed DbCommand. La tercera línea muestra el mensaje. Los mensajes incluyen el tiempo de ejecución de cada consulta.

El plan de ejecución en SQL Server

El plan de ejecución muestra cómo SQL Server ejecuta una consulta y puede inspeccionarse desde SSMS u otras herramientas de plan. SET STATISTICS IO ON aporta métricas de E/S, pero no sustituye al plan de ejecución.

Ejemplo 15 (SQL).

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente';

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```

SQL

La primera línea activa las estadísticas de E/S. La segunda línea activa las estadísticas de tiempo. La tercera línea ejecuta la consulta. La cuarta línea desactiva las estadísticas de E/S. La quinta línea desactiva las estadísticas de tiempo. Las estadísticas muestran el número de lecturas lógicas, físicas y el tiempo de CPU.

Las herramientas de diagnóstico de EF Core

EF Core ofrece varias herramientas de diagnóstico: ToQueryString, LogTo, EnableSensitiveDataLogging, EnableDetailedErrors, DiagnosticSource y los contadores de rendimiento. Estas herramientas permiten observar el comportamiento de EF Core en tiempo de ejecución.

Ejemplo 16 (CSHARP).

```
var sql = consulta.ToQueryString();
Console.WriteLine(sql);
```

CSHARP

La primera línea obtiene el SQL sin ejecutarlo. La segunda línea imprime el SQL. Es útil para inspeccionar una consulta antes de ejecutarla.

El proyecto AceriaData

En el proyecto AceriaData, se profundiza en el diagnóstico en este punto. Se configuran los logs detallados, se mide el tiempo de ejecución, se cuentan las consultas y se detectan las consultas lentas. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

El diagnóstico permite observar el comportamiento de la aplicación.

LogTo configura el logging en EF Core.

Las categorías de logging permiten filtrar los mensajes.

Los niveles de logging indican la importancia del mensaje.

ILoggerFactory integra EF Core con el logging de .NET.

EnableSensitiveDataLogging muestra los valores de los parámetros.

EnableDetailedErrors muestra información detallada en los errores.

ConfigureWarnings configura el comportamiento ante advertencias.

DiagnosticSource publica eventos de diagnóstico.

El tiempo de ejecución se mide con Stopwatch.

El número de consultas se cuenta con el logging o con un observador.

Las consultas lentas se detectan con el logging y la medición del tiempo.

El plan de ejecución en SQL Server identifica operaciones costosas.

En el proyecto AceriaData se configuran los logs y se miden las métricas.

Anclaje en AceriaData

El concepto se implementa en M04/PROYECTO/4.11. En la práctica se restaura, compila y ejecuta sobre SQL Server LocalDB.

Ejemplo ejecutable del concepto

El siguiente archivo es la implementación real de Infrastructure correspondiente al punto 4.11.

```
using System.Diagnostics;
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    public Diagnostic RendimientoDto DiagnosticarPendientesM4()
    {
        _context.ChangeTracker.Clear();
        SqlCommandCounterInterceptor.Instance.Reset();

        var consulta = _context.OrdenesFabricacion
            .TagWith("M4.11-DIAGNOSTICO")
            .AsNoTracking()
            .Where(o => o.Estado == "Pendiente")
            .OrderBy(o => o.FechaCreacion)
            .ThenBy(o => o.Id)
            .Select(o => new OrdenResumenDto
            {
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            })
            .Take(10);

        var sql = consulta.ToQueryString();

        var sw = Stopwatch.StartNew();
        var filas = consulta.ToList();
        sw.Stop();

        return new Diagnostic RendimientoDto
        {
            Filas = filas.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            EntidadesRastreadas = _context.ChangeTracker.Entries().Count(),
            Ticks = sw.ElapsedTicks,
            Sql = sql
        };
    }
}
```

CSHARP

Explicación línea a línea — Rendimiento411.cs

Línea 1	using System.Diagnostics; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using AceriaData.Application.Dtos; → Importa tipos o extensiones requeridos por esta implementación.

Línea 3	<code>using Microsoft.EntityFrameworkCore;</code> → Importa tipos o extensiones requeridos por esta implementación.
Línea 5	<code>namespace AceriaData.Infrastructure.Repositories;</code> → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 7	<code>public sealed partial class OrdenRepositorio</code> → Declara la clase concreta que implementa la funcionalidad del punto.
Línea 8	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 9	<code>public DiagnosticoRendimientoDto DiagnosticarPendientesM4()</code> → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 10	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 11	<code>_context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 12	<code>SqlCommandCounterInterceptor.Instance.Reset();</code> → Mide comandos SQL reales ejecutados.
Línea 14	<code>var consulta = _context.OrdenesFabricacion</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 15	<code>.TagWith("M4.11-DIAGNOSTICO")</code> → Etiqueta el SQL para correlacionarlo con logs.
Línea 16	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 17	<code>.Where(o => o.Estado == "Pendiente")</code> → Añade el predicado de filtrado a la forma de consulta.
Línea 18	<code>.OrderBy(o => o.FechaCreacion)</code> → Forma parte del orden determinista.
Línea 19	<code>.ThenBy(o => o.Id)</code> → Forma parte del orden determinista.
Línea 20	<code>.Select(o => new OrdenResumenDto</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 21	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 22	<code>NumeroOrden = o.NumeroOrden,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 23	<code>Cliente = o.Cliente,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 24	<code>Estado = o.Estado,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 25	<code>FechaCreacion = o.FechaCreacion</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 26	<code>})</code> → Continúa la implementación del punto con esta expresión: <code>})</code>
Línea 27	<code>.Take(10);</code> → Limita el número máximo de elementos.
Línea 29	<code>var sql = consulta.ToQueryString();</code> → Obtiene la representación SQL sin materializar la consulta.
Línea 31	<code>var sw = Stopwatch.StartNew();</code> → Participa en la medición temporal observacional.
Línea 32	<code>var filas = consulta.ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo <code>IQueryable</code> .

Línea 33	<code>sw.Stop();</code> → Detiene el cronómetro inmediatamente después del bloque que se está midiendo.
Línea 35	<code>return new DiagnosticoRendimientoDto</code> → Devuelve el resultado calculado al llamador.
Línea 36	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 37	<code>Filas = filas.Count,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 38	<code>ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),</code> → Mide comandos SQL reales ejecutados.
Línea 39	<code>EntidadesRastreadas = _context.ChangeTracker.Entries().Count(),</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 40	<code>Ticks = sw.ElapsedTicks,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 41	<code>Sql = sql</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 42	<code>};</code> → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 43	<code>}</code> → Delimita el bloque sintáctico asociado.
Línea 44	<code>}</code> → Delimita el bloque sintáctico asociado.

Qué debe observarse en ejecución

La lectura del código debe completarse con la evidencia de ejecución: SQL traducido cuando corresponda, número de comandos, cardinalidad, columnas materializadas, estado del `ChangeTracker` y las comprobaciones específicas del punto. El hecho de que el código compile no demuestra por sí solo una mejora de rendimiento; por eso cada punto termina en una comprobación observable.

Reto conceptual. Definir qué métrica distinguiría roundtrips de materialización.

Analogía operativa. Diagnosticar es instrumentar la línea antes de cambiar la máquina.

Criterios de salida

- Relacionar LINQ con SQL o comandos ejecutados.
- Distinguir coste de servidor, transferencia, materialización y tracking.
- Justificar técnicas aplicadas y descartadas.
- Ejecutar el proyecto del punto y obtener 4.11 OK.

Punto 4.12 — Estrategias de optimización y checklist de rendimiento

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se consolidan las estrategias de optimización del proyecto *AceriaData*, aplicando un checklist de rendimiento que recorre todas las técnicas del Módulo 4 y cerrando el módulo con un caso práctico completo de optimización.

Objetivos de aprendizaje

- Comprender qué es una estrategia de optimización y por qué es necesaria.
- Conocer el checklist de rendimiento de EF Core.
- Aplicar el checklist al proyecto *AceriaData*.

- Identificar los problemas de rendimiento más comunes.
- Resolver los problemas de rendimiento con las técnicas del Módulo 4.
- Comprender el orden de aplicación de las técnicas de optimización.
- Medir el impacto de las optimizaciones.
- Cerrar el Módulo 4 con una visión consolidada.

Consideraciones técnicas en EF Core 8

El checklist no impone una clasificación universal. Primero se define la forma necesaria, después se observa SQL, roundtrips, materialización y tracking, y solo entonces se eligen o descartan técnicas.

Desarrollo teórico

Qué es una estrategia de optimización

Una estrategia de optimización es un conjunto ordenado de pasos que se aplican para mejorar el rendimiento de una aplicación. No se trata de aplicar técnicas al azar, sino de seguir un proceso estructurado: medir, identificar, aplicar, verificar y documentar. La optimización sin medición es adivinación. La optimización sin verificación es fe. La optimización sin documentación es olvido.

Ejemplo 1 (CSHARP).

```
// 1. Medir
var cronometro = Stopwatch.StartNew();
var ordenes = context.OrdenesFabricacion.ToList();
cronometro.Stop();
Console.WriteLine($"Tiempo base: {cronometro.ElapsedMilliseconds} ms");

// 2. Identificar el problema (over-fetching, N+1, etc.)
// 3. Aplicar la solución (proyección, Include, etc.)
// 4. Verificar la mejora
// 5. Documentar la decisión
```

CSHARP

La primera sección mide el tiempo base. La segunda sección indica que se debe identificar el problema. La tercera sección indica que se debe aplicar la solución. La cuarta sección indica que se debe verificar la mejora. La quinta sección indica que se debe documentar la decisión.

El checklist de rendimiento

El checklist de rendimiento es una lista de comprobaciones que se aplican a cada consulta y a cada operación de escritura. El objetivo es detectar los problemas de rendimiento más comunes antes de que lleguen a producción. El checklist se aplica en orden de impacto: primero los problemas más graves, después los menos graves.

Ejemplo 2 (TEXT).

- 1\ ¿Se está usando AsNoTracking en consultas de solo lectura?
- 2\ ¿Se están proyectando solo las columnas necesarias?
- 3\ ¿Se está evitando el problema N+1 con Include?
- 4\ ¿Se está evitando el producto cartesiano con AsSplitQuery?
- 5\ ¿Se están aplicando filtros y paginación en el servidor?
- 6\ ¿Se están evitando funciones en Where que impidan el uso de índices?
- 7\ ¿Se están usando Compiled Queries en consultas frecuentes?
- 8\ ¿Se están midiendo los tiempos y contando las consultas?

TEXT

El checklist enumera las comprobaciones. Cada una corresponde a una técnica del Módulo 4. El orden va de mayor a menor impacto: el tracking y el over-fetching son los más impactantes, la paginación y las funciones en Where son los siguientes, y las Compiled Queries son los últimos.

El orden de aplicación de las técnicas

El orden de aplicación de las técnicas sigue el principio de mayor impacto primero. Las técnicas que afectan a todas las consultas se aplican primero. Las técnicas que afectan a consultas concretas se aplican después.

Ejemplo 3 (TEXT).

- 1\ AsNoTracking en consultas de solo lectura (afecta a todas las consultas)
- 2\ Proyecciones (afecta a todas las consultas que no necesitan la entidad completa)
- 3\ Include para evitar N+1 (afecta a consultas con relaciones)
- 4\ AsSplitQuery para evitar producto cartesiano (afecta a consultas con varias colecciones)
- 5\ Filtros y paginación en el servidor (afecta a consultas con muchos resultados)
- 6\ Evitar funciones en Where (afecta a consultas con filtros)

TEXT

- 7\.. Compiled Queries en consultas frecuentes (afecta a consultas repetidas)
- 8\.. Diagnóstico continuo (afecta a todas las consultas)

El orden va de mayor a menor impacto. Las primeras técnicas afectan a más consultas. Las últimas afectan a menos.

Los problemas de rendimiento más comunes

Entre los problemas habituales están tracking innecesario, over-fetching, N+1, explosión cartesiana, fronteras cliente mal elegidas, expresiones poco sargables y paginación inadecuada. Las técnicas se eligen según evidencia; no existe una receta que deba aplicarse completa a cada consulta.

Ejemplo 4 (TEXT).

TEXT

- Tracking innecesario → AsNoTracking
- Over-fetching → Proyecciones
- N+1 → Include
- Producto cartesiano → AsSplitQuery
- Consultas en memoria → Expresiones traducibles
- Funciones en Where → Comparaciones directas o collation
- Falta de paginación → Skip/Take o keyset pagination
- Compilación repetida → Compiled Queries

Cada problema tiene su solución. El checklist permite detectarlos y aplicar la solución correspondiente.

La medición del impacto

La medición del impacto es esencial para verificar que las optimizaciones funcionan. Se mide el tiempo de ejecución antes y después de la optimización. Se cuentan las consultas antes y después. Se mide el volumen de datos transferidos antes y después.

Ejemplo 5 (CSHARP).

CSHARP

```
var cronometroAntes = Stopwatch.StartNew();
var ordenesAntes = context.OrdenesFabricacion.ToList();
cronometroAntes.Stop();

var cronometroDespues = Stopwatch.StartNew();
var ordenesDespues = context.OrdenesFabricacion
    .AsNoTracking()
    .Select(o => new { o.NumeroOrden, o.Cliente })
    .ToList();
cronometroDespues.Stop();

Console.WriteLine($"Antes: {cronometroAntes.ElapsedMilliseconds} ms");
Console.WriteLine($"Después: {cronometroDespues.ElapsedMilliseconds} ms");
```

La primera sección mide el tiempo antes de la optimización. La segunda sección mide el tiempo después de la optimización. La tercera sección muestra los tiempos. La diferencia es el impacto de la optimización.

La documentación de las decisiones

La documentación de las decisiones es esencial para que otros desarrolladores entiendan por qué se aplicó una técnica concreta. Se documenta en comentarios XML, en la documentación del proyecto o en un registro de decisiones de arquitectura.

Ejemplo 6 (CSHARP).

CSHARP

```
/// <summary>
/// Obtiene los resúmenes de las órdenes pendientes.
/// Usa proyección para reducir el volumen de datos transferidos.
/// Usa AsNoTracking porque es una consulta de solo lectura.
/// Usa filtro por estado para reducir el número de filas.
/// </summary>
public List<OrdenResumenDto> ObtenerResumenesPendientes()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado
        })
        .ToList();
}
```

La primera sección declara el comentario XML. Las siguientes líneas describen la técnica aplicada. La sexta línea declara el método. Las siguientes líneas implementan la consulta.

El ciclo de optimización

La optimización es un ciclo continuo, no una tarea puntual. Se mide, se identifica, se aplica, se verifica y se documenta. Después se vuelve a medir para detectar nuevos problemas. El ciclo se repite a lo largo de la vida de la aplicación.

Ejemplo 7 (TEXT).

Medir → Identificar → Aplicar → Verificar → Documentar → Medir...

TEXT

El ciclo se repite. Cada vuelta mejora el rendimiento. La optimización es un proceso continuo, no un evento único.

Los anti-patrones de optimización

Los anti-patrones de optimización son las prácticas que parecen optimizaciones pero no lo son. La optimización prematura es el anti-patrón más común: aplicar técnicas de optimización antes de medir. La optimización sin medición es otra: aplicar técnicas sin saber si mejoran el rendimiento. La optimización excesiva es otra: aplicar demasiadas técnicas y complicar el código sin beneficio.

Ejemplo 8 (CSHARP).

```
// Anti-patrón: optimización prematura
var ordenes = context.OrdenesFabricacion
    .AsNoTracking()
    .AsSplitQuery()
    .Select(o => new { o.NumeroOrden })
    .ToList();
```

CSHARP

La consulta combina técnicas sin demostrar que todas aporten valor. Si la proyección escalar elimina las navegaciones, SplitQuery deja de tener un grafo de colecciones que dividir; el checklist debe justificar técnicas aplicadas y descartadas.

Las métricas de rendimiento

Las métricas de rendimiento son los indicadores que se miden para evaluar la optimización. Las más habituales son: tiempo de ejecución, número de consultas, volumen de datos transferidos, uso de memoria y uso de CPU.

Ejemplo 9 (TEXT).

- Tiempo de ejecución: milisegundos por consulta
- Número de consultas: consultas por operación
- Volumen de datos: bytes transferidos
- Uso de memoria: entidades rastreadas
- Uso de CPU: tiempo de compilación de consultas

Cada métrica indica un aspecto del rendimiento. La combinación de todas permite evaluar la optimización de forma completa.

TEXT

El proyecto AceriaData

En el proyecto AceriaData, se consolidan las estrategias de optimización en este punto. Se aplica el checklist de rendimiento a todas las consultas del repositorio. Se miden los tiempos y se cuentan las consultas. Se documentan las decisiones. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

Una estrategia de optimización es un conjunto ordenado de pasos.

El checklist de rendimiento detecta los problemas más comunes.

El orden de aplicación va de mayor a menor impacto.

Los problemas más comunes son tracking, over-fetching, N+1 y producto cartesiano.

La medición del impacto verifica que las optimizaciones funcionan.

La documentación de las decisiones evita olvidos.

La optimización es un ciclo continuo.

Los anti-patrones son la optimización prematura y excesiva.

Las métricas de rendimiento son tiempo, consultas, datos, memoria y CPU.

En el proyecto AceriaData se aplica el checklist a todas las consultas.

Anclaje en AceriaData

El concepto se implementa en M04/PROYECTO/4.12. En la práctica se restaura, compila y ejecuta sobre SQL Server LocalDB.

Ejemplo ejecutable del concepto

El siguiente archivo es la implementación real de Infrastructure correspondiente al punto 4.12.

```
using AceriaData.Application.Dtos;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public sealed partial class OrdenRepositorio
{
    public ChecklistRendimientoDto EjecutarChecklistFinalM4()
    {
        _context.ChangeTracker.Clear();
        SqlCommandCounterInterceptor.Instance.Reset();

        var consulta = _context.OrdenesFabricacion
            .TagWith("M4.12-CHECKLIST-FINAL")
            .AsNoTracking()
            .Where(o => o.Estado == "Pendiente")
            .OrderBy(o => o.FechaCreacion)
            .ThenBy(o => o.Id)
            .Select(o => new OrdenPaginaDto
            {
                Id = o.Id,
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            })
            .Take(5);

        var sql = consulta.ToQueryString();
        var filas = consulta.ToList();

        return new ChecklistRendimientoDto
        {
            Filas = filas.Count,
            ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),
            EntidadesRastreadas = _context.ChangeTracker.Entries().Count(),
            Sql = sql,
            DecisionCompiledQuery =
                "No se aplica: esta consulta final no se ha demostrado como hot path; medir antes de introducir
complejidad.",
            DecisionLoading =
                "Proyección escalar: no se cargan navegaciones, por lo que Include/SplitQuery no aportan valor en
esta consulta."
        };
    }
}
```

Explicación línea a línea — Rendimiento412.cs

Línea 1	using AceriaData.Application.Dtos; → Importa tipos o extensiones requeridos por esta implementación.
Línea 2	using Microsoft.EntityFrameworkCore; → Importa tipos o extensiones requeridos por esta implementación.
Línea 4	namespace AceriaData.Infrastructure.Repositories; → Sitúa el archivo en la capa y espacio de nombres correspondiente.
Línea 6	public sealed partial class OrdenRepositorio → Declara la clase concreta que implementa la funcionalidad del punto.
Línea 7	{ → Delimita el bloque sintáctico asociado.
Línea 8	public ChecklistRendimientoDto EjecutarChecklistFinalM4() → Declara un método concreto de la implementación, con su tipo de retorno, nombre y parámetros.
Línea 9	{ → Delimita el bloque sintáctico asociado.

Línea 10	<code>_context.ChangeTracker.Clear();</code> → Limpia tracking antes de la demostración.
Línea 11	<code>SqlCommandCounterInterceptor.Instance.Reset();</code> → Mide comandos SQL reales ejecutados.
Línea 13	<code>var consulta = _context.OrdenesFabricacion</code> → Calcula y conserva el resultado que será validado o mostrado.
Línea 14	<code>.TagWith("M4.12-CHECKLIST-FINAL")</code> → Etiqueta el SQL para correlacionarlo con logs.
Línea 15	<code>.AsNoTracking()</code> → Desactiva tracking para esta consulta de lectura.
Línea 16	<code>.Where(o => o.Estado == "Pendiente")</code> → Añade el predicado de filtrado a la forma de consulta.
Línea 17	<code>.OrderBy(o => o.FechaCreacion)</code> → Forma parte del orden determinista.
Línea 18	<code>.ThenBy(o => o.Id)</code> → Forma parte del orden determinista.
Línea 19	<code>.Select(o => new OrdenPaginaDto</code> → Proyecta la forma de resultado y controla datos materializados.
Línea 20	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 21	<code>Id = o.Id,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 22	<code>NumeroOrden = o.NumeroOrden,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 23	<code>Cliente = o.Cliente,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 24	<code>Estado = o.Estado,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 25	<code>FechaCreacion = o.FechaCreacion</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 26	<code>})</code> → Continúa la implementación del punto con esta expresión: <code>})</code>
Línea 27	<code>.Take(5);</code> → Limita el número máximo de elementos.
Línea 29	<code>var sql = consulta.ToQueryString();</code> → Obtiene la representación SQL sin materializar la consulta.
Línea 30	<code>var filas = consulta.ToList();</code> → Materializa la consulta y ejecuta SQL si sigue siendo IQueryable.
Línea 32	<code>return new ChecklistRendimientoDto</code> → Devuelve el resultado calculado al llamador.
Línea 33	<code>{</code> → Delimita el bloque sintáctico asociado.
Línea 34	<code>Filas = filas.Count,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 35	<code>ConsultasSql = checked((int)SqlCommandCounterInterceptor.Instance.Count),</code> → Mide comandos SQL reales ejecutados.
Línea 36	<code>EntidadesRastreadas = _context.ChangeTracker.Entries().Count(),</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 37	<code>Sql = sql,</code> → Asigna el valor calculado a la propiedad o variable correspondiente del resultado.
Línea 38	<code>DecisionCompiledQuery =</code> → Continúa la implementación del punto con esta expresión: <code>DecisionCompiledQuery =</code>
Línea 39	<code>"No se aplica: esta consulta final no se ha demostrado como hot path; medir antes de introducir complejidad."</code> → Completa el mensaje diagnóstico que documenta la evidencia observada o el motivo del fallo.

Línea 40	DecisionLoading = → Continúa la implementación del punto con esta expresión: DecisionLoading =
Línea 41	"Proyeccion escalar: no se cargan navegaciones, por lo que Include/SplitQuery no aportan valor en esta consulta." → Continúa la implementación del punto con esta expresión: "Proyeccion escalar: no se cargan navegaciones, por lo que Include/SplitQuery no aportan valor en esta consulta."
Línea 42	}; → Cierra la llamada, expresión o inicializador abierto en las líneas anteriores.
Línea 43	} → Delimita el bloque sintáctico asociado.
Línea 44	} → Delimita el bloque sintáctico asociado.

Qué debe observarse en ejecución

La lectura del código debe completarse con la evidencia de ejecución: SQL traducido cuando corresponda, número de comandos, cardinalidad, columnas materializadas, estado del ChangeTracker y las comprobaciones específicas del punto. El hecho de que el código compile no demuestra por sí solo una mejora de rendimiento; por eso cada punto termina en una comprobación observable.

Reto conceptual. Auditar una consulta y justificar tanto técnicas aplicadas como descartadas.

Analogía operativa. El checklist final no cambia todas las piezas de la máquina, solo las que la medición justifica.

Criterios de salida

- Relacionar LINQ con SQL o comandos ejecutados.
- Distinguir coste de servidor, transferencia, materialización y tracking.
- Justificar técnicas aplicadas y descartadas.
- Ejecutar el proyecto del punto y obtener 4.12 OK.